



中国研究生创新实践系列大赛  
“华为杯”第二十一届中国研究生  
数学建模竞赛

学 校 西安邮电大学

---

参赛队号 24116640167

---

1.李明浩

---

队员姓名 2.向 洋

---

3.周夏怡

---

中国研究生创新实践系列大赛

# “华为杯”第二十一届中国研究生 数学建模竞赛

题 目                      WLAN 组网中网络吞吐量建模

## 摘                      要：

随着无线局域网(WLAN)的普及,在高密度部署环境中,由于节点密度的不断增大,导致了网络中的干扰、冲突等问题变得更加严重,实际的部署带宽和数据传输速率都会大幅度降低,可支撑的系统吞吐量依然受到限制,这就要求对 WLAN 系统进行进一步的优化。对 WLAN 系统优化的核心问题是对吞吐量预测,根据 WLAN 实测数据的基础上,研究无线局域网的网络拓扑结构、节点之间的 RSSI、信道接入机制和干扰等因素对无线局域网的数据传输、传输速率的影响,进而实现对无线局域网的吞吐量的准确预测。对吞吐量的准确预测有助于在工业、教育、医疗等新兴应用领域取得突破,能够为用户带来最优服务体验。

本文通过机器学习和深度学习的相关算法,对 WLAN 实测数据进行网络拓扑分析、对节点间接受信号的强度(RSSI)分析以及其他干扰因素分析来评估对 WLAN 的影响,根据附件提供的大量数据进行组合筛选,利用Pearson相关性研究、随机森林、决策树等机器学习与深度学习数学建模理论的知识,通过Python、Spss、matlab 等工具组合使用来解决问题。

针对问题一,根据问题的需求,对相关实测数据进行处理,根据 WLAN 网络实测训练数据,对网络拓扑、业务流量、门限、节点间 RSSI 等因素对 AP 发送机会的影响进行分析,并按影响强弱的顺序进行排序,通过Python训练随机森林回归模型和 LSTM 模型,验证两种模型在多个参数下的最优情况,筛选出最优训练模型,并对每个任务的 AP 发送机会进行预测,得出最终的预测结果,预测结果保存于附录和附件中。

针对问题二,根据实测数据中的 RSSI 和门限信息,结合问题一对 AP 发送机会的分析,通过建立模型,通过相关性分析选择与预测数据相关影响因素特征,并使用Python编写三种模型,即随机森林回归模型,多元线性回归模型和 LSTM 模型,使用训练数据训练模型,采用梯度下降法对模型训练进行优化,最终预测出 AP 在传输过程中的 PHY Rate,并通过(MCS,NSS)组合表映射得出预测的(MCS,NSS),预测结果与相关计算信息保存于附录和附件中。

针对问题三,结合问题一和问题二而通过构建合适的模型,通过Python使用三种模型进行预测 WLAN 系统的吞吐量。我们综合考虑问题一和问题二中预测的数据,以及训练集中的其他数据,将所有数据整合后,并通过相关性系数对整合数据进行分析,得出网络吞吐量的影响因数,筛选特征信息,从提高模型的预测精度。最后,构建随机森林回归模型、三种核函数的支持向量机模型和 LSTM 模型对筛选出的相关特征信息进行学习,并通过验证集得出最优模型,对提供的测试集进行网络吞吐量的预测,预测结果及相关计算信息保存于附录和附件中。

**关键词：**随机森林；LSTM 神经网络；机器学习；梯度下降法；相关性分析；

## 目录

|                               |    |
|-------------------------------|----|
| 一、问题背景与问题重述.....              | 4  |
| 1.1 问题重述.....                 | 4  |
| 1.2 课题背景.....                 | 4  |
| 二、模型假设及关键性符号说明.....           | 5  |
| 2.1 模型假设.....                 | 5  |
| 2.2 符号说明.....                 | 5  |
| 三、问题一模型建立与求解.....             | 6  |
| 3.1 问题背景及目标.....              | 6  |
| 3.2 问题分析与数据预处理.....           | 6  |
| 3.2.1 AP 发送机会影响因素分析.....      | 8  |
| 3.2.2 相关性分析.....              | 8  |
| 3.2.3 影响性强弱排序.....            | 11 |
| 3.2.4 数据预处理.....              | 12 |
| 3.3 模型建立.....                 | 17 |
| 3.3.1 建模目标.....               | 17 |
| 3.3.2 特征选择.....               | 18 |
| 3.3.3 随机森林回归模型.....           | 18 |
| 3.3.3 长短时记忆网络模型.....          | 19 |
| 3.4 模型求解结果.....               | 21 |
| 四、问题二模型建立与求解.....             | 25 |
| 4.1 问题背景及目标.....              | 25 |
| 4.2 问题分析与建模思路.....            | 25 |
| 4.2.1 MCS 和 NSS 选择影响因素分析..... | 25 |
| 4.2.2 相关性分析.....              | 26 |
| 4.3 模型建立.....                 | 27 |
| 4.3.1 建模目标.....               | 27 |
| 4.3.2 数学模型详解.....             | 28 |
| 4.3.3 特征选择.....               | 31 |
| 4.3.4 梯度下降法与多元线性回归模型.....     | 31 |
| 4.4 模型求解结果.....               | 34 |
| 五、问题三模型建立与求解.....             | 38 |
| 5.1 问题背景及目标.....              | 38 |
| 5.2 问题分析与建模思路.....            | 39 |
| 5.2.1 WLAN 网络数据传输分析.....      | 39 |
| 5.2.2 网络吞吐量影响因素分析.....        | 39 |
| 5.2.3 相关性分析.....              | 40 |
| 5.3 模型建立.....                 | 42 |
| 5.3.1 建模目标.....               | 42 |
| 5.3.2 数据查询.....               | 42 |
| 5.3.3 吞吐量计算.....              | 43 |
| 5.3.4 支持向量机预测.....            | 44 |
| 5.4 模型求解结果.....               | 45 |

|                    |    |
|--------------------|----|
| 六、模型的总结与评价.....    | 51 |
| 6.1 模型优缺点分析 .....  | 51 |
| 6.1.1 模型的优点.....   | 51 |
| 6.1.2 模型的缺点.....   | 52 |
| 6.2 模型的改进与推广 ..... | 52 |
| 参考文献.....          | 53 |
| 附录.....            | 54 |

# 一、问题背景与问题重述

## 1.1 问题重述

根据附件中实测数据训练集和测试集以及相关知识和问题要求，拟建立 **AP 发送时间**的预测模型，AP 发送数据时选用(MCS, NSS)的预测模型，WLAN 系统吞吐量的预测模型，分别完成下列问题：

**问题 1：根据实际测量的 WLAN 网络数据，分析 WLAN 网络中各种数据信息，给出影响性强弱的顺序，预测每个 AP 发送数据帧序列的总时长。**

将给定的数据集中节点间 RSSI、信道接入机制、网络拓扑、业务流量等特征作为输入，影响因素重要性排序，将每个 AP 的发送时长(seq\_time)作为输出，从而建立预测模型。

**问题 2：对 AP 发送数据的 (MCS, NSS)进行预测。**

MCS 和 NSS 共同决定了数据发送时所选用的 PHY Rate(物理层速率)，而物理层速率是除了发送机会以外的另一个影响 WLAN 系统吞吐量的因素。将 RSSI 信息、门限信息以及问题 1 中分析的特征作为输入，输出是对 AP 发送数据的预测的(MCS, NSS)组合。

**问题 3：对 WLAN 系统吞吐量进行建模。**

将给定数据集中测试集的网络拓扑、RSSI 等信息作为输入，结合问题 1 对 AP 发送时常的预测和问题 2 对(MCS, NSS)的预测进而对各个 AP 的吞吐量进行预测，并建立模型。

## 1.2 课题背景

WLAN(Wireless Local Area Network)，即无线局域网，能够在一定的范围内实现设备和设备之间的连接从而形成局域网，WLAN 中常见的技术为 Wi-Fi，WLAN 由于其网络使用自由、网络部署灵活、安装便捷、节约成本等特点受到了广泛地应用，部署领域从范围较小的区域到了高密的场景，如会议室、企业网、工业制造等，在这些高密的部署场景中节点更加密集且覆盖范围可能重叠，随之带来的问题是高密的环境下造成 WLAN 系统吞吐量受限，使得数据传输效率和部署带宽下降，进而对 WLAN 系统进行优化是非常必要的<sup>[1]</sup>。

无线局域网优化问题的关键在于对无线局域网的吞吐量进行预测，即每一次传输的比特数目。准确、快速地预测无线局域网的吞吐量，是构建可靠、高效的无线局域网系统的关键技术。为此，近年来许多研究采用机器学习的相关技术，对 WLAN 系统进行特征抽取，并对其进行建模。主要特征内容包括：信道、带宽、流量、传输功率、信道访问机制等**基础信息**、**架构信息**包括节点间信号强度(RSSI)、信干噪比(SINR)、传输时间等，以及节点动态位置和环境噪声等**临时信息**。但是，在实际应用中，无线局域网的通信行为会受信道环境、干扰等因素的影响，导致训练的模型预测精度不高，难以实现商业应用。然而，本项目拟将影响因子与实际测量数据相结合，构建无线局域网的吞吐量预测模型，不但可以得到更加精确的无线网络系统的吞吐量预测，而且可以有效地指导无线网络的优化与性能提升。

在此基础上，本项目将研究利用深度学习与机器学习相结合的方法，并使用最优化方法求解，通过采集各节点的时间序列吞吐率，并对其进行预处理，从而实现对节点吞吐量的预测。拟采用数据预处理、特征工程、机器学习、统计分析、模型评价等多种手

段，对无线局域网的吞吐量进行精确预测，为无线局域网的性能优化提供理论依据。通过对模型、算法进行优化，并兼顾其泛化能力与实时性能，从而保证模型在实际部署场景中的应用效果。

## 二、模型假设及关键性符号说明

### 2.1 模型假设

为了能够方便和准确的预测 WLAN 组网中的各种数据，我们做出如下假设：

(1)假设在确定 AP 和 STA 数量后，统计在内的设备不会受到范围外的设备进行信号干扰。

(2)假设所有的实验都具有代表性、不存在特殊或极端的条件，环境噪声保持稳定，且可用一个固定的常量数值表示，附件以及收集来的数据都真实可靠。

(3)假设信号无法检测时，无信号强度数据设为-100Bdm，且信号传送过程中，强度不会受到任何因素(如传输距离，穿透障碍物等)而造成衰减。

(4)假设信干噪比波动值大于 0.1 时，AP 传输模由同步传输转换为异步传输。

### 2.2 符号说明

| 符号                 | 含义                    |
|--------------------|-----------------------|
| $x_t$              | 输入特征向量集合              |
| $RSSI_t$           | t 时刻节点间接收信号强度         |
| $PD_t$             | 物理检测门限                |
| $ED_t$             | 能量检测门限                |
| $NAV_t$            | 网络分配矢量                |
| $Protocol_t$       | 业务流量协议类型              |
| $Pkt\_len_t$       | 数据包长度                 |
| $EIRP_t$           | AP 发送功率               |
| $SINR_{STA}$       | STA 接收数据时的信干噪比        |
| $P_{signal}$       | AP 到 STA 的信号功率        |
| $P_{noise}$        | 环境噪声                  |
| $RSSI_{Ass\_AP}$   | 关联 AP 到该 STA 的 RSSI   |
| $RSSI_{Other\_AP}$ | 邻区其他 AP 到该 STA 的 RSSI |

### 三、问题一模型建立与求解

#### 3.1 问题背景及目标

**背景：**无线局域网(WLAN)系统中，网络拓扑、业务流量、门限、节点间的 RSSI 等多种因素影响 AP(接入点)的发送机会，即 AP 在单位时间内能够发送数据的时长。为了优化 WLAN 系统，精确预测 AP 的发送机会和吞吐量至关重要。现有研究大多基于仿真数据进行建模，但实际部署中的环境复杂多变，导致仿真模型难以准确反映真实场景。该问题需要基于实测数据，分析和建模这些影响因素对 AP 发送机会的影响。

**目标：AP 发送机会的影响分析和预测。**基于 WLAN 网络实测训练数据，分析网络拓扑、业务流量、门限、节点间 RSSI 等因素对 AP 发送机会的影响，并按影响强弱排序。通过训练模型，预测每个 AP 的发送机会，即 AP 发送数据帧序列的总时长(seq\_time)，并根据测试数据进行预测验证。

#### 3.2 问题分析与数据预处理

在 WLAN 网络中，AP 的发送机会(即发送数据帧序列的总时长 seq\_time)受多个因素影响。为了更好的分析理解各参数对其的影响，决定使用模拟器 eNSP 根据不同 AP 个数情况搭建相应的 WLAN 网络拓扑图。

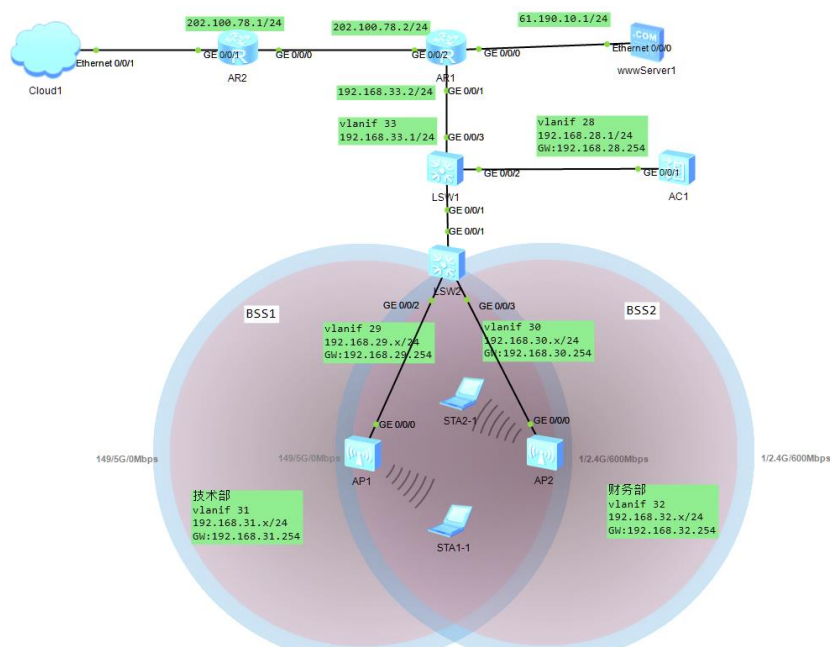


图 3.1 eNSP 模拟 2AP 下 WLAN 网络拓扑图

上图为采用两个 AP 模拟企业无线网络设计，考虑实际情况下，WLAN 组网在企业内不同部门使用不同 AP 提供无线网络服务，因此会有多个 AP，故使用 AC 进行统一管

理。

在 2 个 AP 组网情况下，其中，AP1 为技术部门提供服务，命名为 BSS1。AP2 为财务部门提供服务，命名为 BSS2。STA1-1 和 STA2-1 分别为技术部和财务部使用的主机终端，由图可知，在 STA 处于 AP1 和 AP2 信号重叠范围内时，STA1-1 与 AP1 通信会受到 AP2 的信号影响，STA2-1 同理。

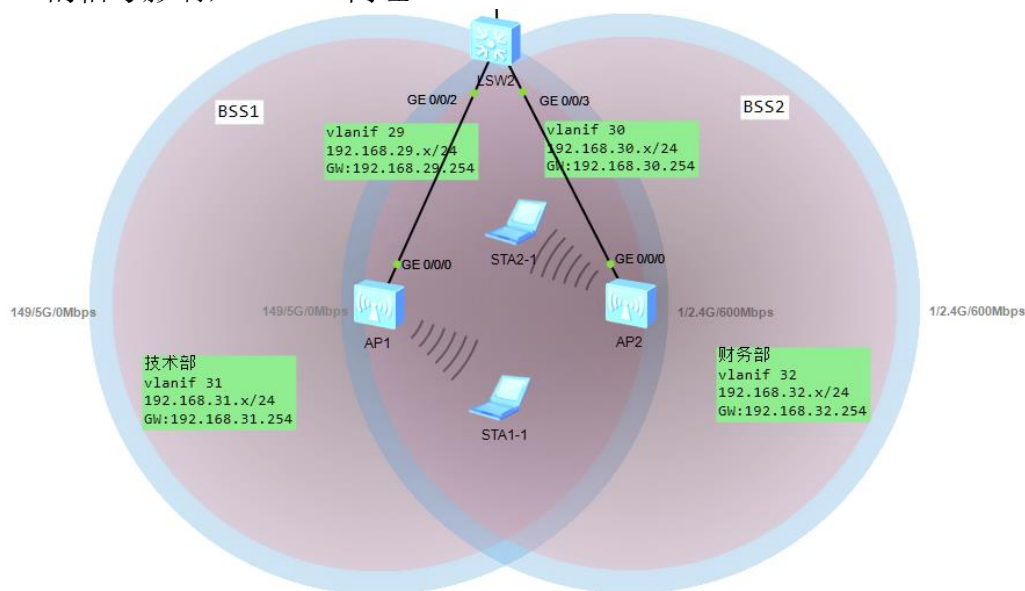


图 3.2 eNSP 模拟 2AP 下 WLAN 网络拓扑图

在 3 个 AP 组网情况下，新增添了一个 AP3 为销售部门提供无线网络服务，命名为 BSS3。STA1-1、STA1-2 和 STA2-1、STA2-2 以及 STA3-1、STA3-2 分别为技术部、财务部和销售部使用的主机终端，由图可知，在 STA 处于 AP1、AP2 和 AP3 信号重叠范围内时，STA1-2 与 AP1 通信会受到 AP2 和 AP3 的信号影响，STA2-2 和 STA3-2 同理。

因此 AP 信号重叠区域，STA 的通信会受到其他 AP 信号的干扰。当其他 AP 信号强度所映射的范围很广时，会影响到通信 AP 的发送机会。在异步传输模式，节点之间的信号强度会产生信号噪声，会对信噪比大小产生影响。

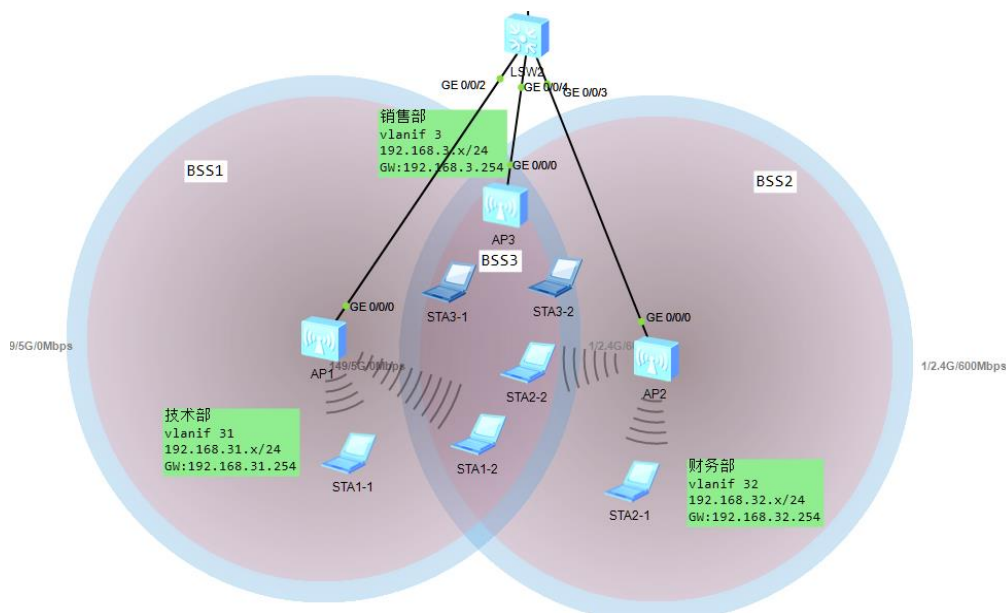


图 3.3 eNSP 模拟 3AP 下 WLAN 网络拓扑图

以下根据提供的实测训练数据和 WLAN 网络示意图,分析得出 AP 发送机会的影响参数主要有以下几点:

- 节点间 RSSI(接收信号强度)
- CCA 门限(PD 门限和 ED 门限)
- NAV 门限
- 业务流量特性(协议类型、数据包长度)
- AP 发送功率(EIRP)
- 网络拓扑(AP 和 STA 的位置)

### 3.2.1 AP 发送机会影响因素分析

以下逐一分析上述参数对 AP 发送机会的影响:

**1.节点间 RSSI(接收信号强度):** RSSI 直接影响 STA 和 AP 之间的通信质量。当 STA 处于多个 AP 的信号覆盖范围时,其他较强的干扰 AP 信号会影响与 STA 通信的 AP 的发送机会,导致通信速率下降。因此, RSSI 是 seq\_time 的重要影响因素之一。

**2.CCA 门限(载波感知门限,包括 PD 门限和 ED 门限):** CCA 机制通过物理载波检测(PD)和能量检测(ED)来感知信道是否可用。如果门限设置过低,AP 会过早认为信道繁忙,降低其传输机会;而设置过高可能导致碰撞增加<sup>[2]</sup>。合理的 CCA 门限设置能平衡 AP 发送机会与碰撞风险。

**3. NAV 门限:** NAV 值用于控制 STA 何时可以开始传输数据,较高的 NAV 值会使 STA 等待更长时间再传输,从而减少 AP 的发送机会。NAV 值的配置需要考虑多 AP 场景下的干扰,特别是信号重叠区域。

**4.业务流量特性(协议类型、数据包长度等):** 不同的协议类型和数据包长度对 AP 的发送机会会有不同的需求。较大的数据包需要占用更多时间,增加了 seq\_time。此外,不同的流量类型(如媒体流量、网页流量等),对时延和带宽的需求也影响着 AP 的负载。

**5. AP 发送功率(EIRP):** 发送功率决定了 AP 的覆盖范围和信号强度,较高的发送功率可以覆盖更大的区域,但也会增加相邻 AP 之间的干扰。在信号重叠的情况下,高功率 AP 可能会对邻近 AP 造成更大的干扰,从而影响它们的 seq\_time。

**6.网络拓扑(AP 和 STA 的位置):** AP 和 STA 的位置布局决定了网络中各设备的信号覆盖范围和干扰情况。在网络规划中,合理的拓扑布局能够减少 AP 间的相互干扰,优化整体性能。

### 3.2.2 相关性分析

(1) 皮尔森相关系数(Pearson correlation coefficient)<sup>[9]</sup>,又称皮尔逊积矩相关系数(Pearson product-moment correlation coefficient,简称 PPMCC 或 PCCs)是一种常用的线性相关系数,用于反映两个变量  $X$  和  $Y$  的线性相关程度,其值介于-1 到 1 之间。如图 3.1 所示,系数值  $r$  介于-1 到 1 之间, $r$  为负值,呈负相关, $r$  为正值,则呈正相关。系数  $r$  绝对值越大,相关性越强, $r$  接近于 0,相关性越弱。

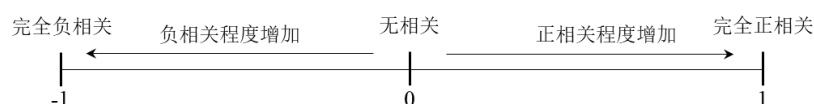


图 3.4 皮尔森相关系数

两个变量之间的皮尔逊相关系数被定义为两个变量之间的协方差除和标准差的商，即将两个变量  $X$  和  $Y$  之间的协方差和各自标准差乘积的比值记为总体的相关系数  $\rho$ ，其计算方式如下：

$$\rho_{X,Y} = \frac{cov(X,Y)}{\sigma_X \sigma_Y} = \frac{E[(X-\mu_X)(Y-\mu_Y)]}{\sigma_X \sigma_Y} \quad (3-1)$$

上式定义了总体相关系数，常用希腊小写字母  $\rho_{X,Y}$  作为系数代表符号

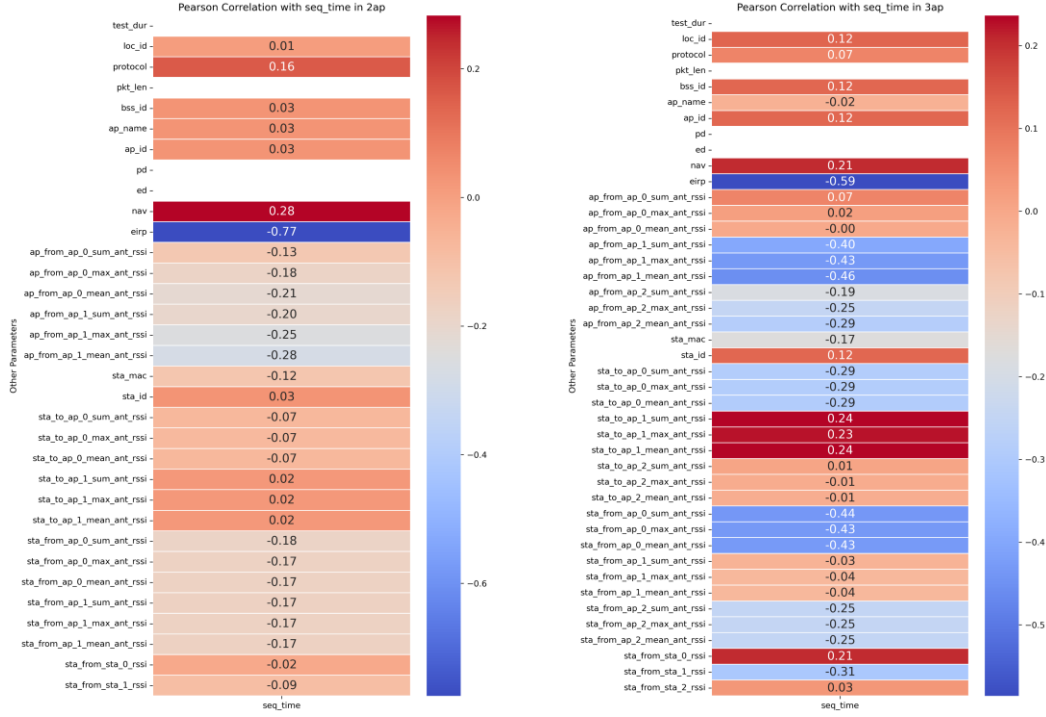


图 3.5 发送机会的皮尔森相关系数

(2) 斯皮尔曼秩相关系数(The Spearman's rank coefficient of correlation)，简称斯皮尔曼相关系数，是秩相关(rank correlation)的一种非参数度量(nonparametric measure)，通常记为  $\rho_S$ 。斯皮尔曼相关可以看作皮尔逊相关的非参数版本。斯皮尔曼相关考察的是两者单调关系的强度，即两者在变大或变小的趋势上保持步调一致的程度，哪怕没有保持比例关系。。

斯皮尔曼相关系数是一种无参数(与分布无关)的检验方法，用于度量变量之间联系的强弱。如果数据中没有重复值，并且当两个变量完全单调相关时，斯皮尔曼相关系数则为+1 或-1，称变量完全 *Spearman* 秩相关。*Spearman* 相关性的基本思想是：分别对两个变量  $X$ 、 $Y$  做等级变换(rank transformation)，用等级  $R_X$  和  $R_Y$  表示；然后按 Pearson 相关性分析的方法计算  $R_X$  和  $R_Y$  的相关性<sup>[12]</sup>。

$$\rho_S = \frac{\sum_{i=1}^n (R_{X_i} - \overline{R_X})(R_{Y_i} - \overline{R_Y})}{\sqrt{\sum_{i=1}^n (R_{X_i} - \overline{R_X})^2 \sum_{i=1}^n (R_{Y_i} - \overline{R_Y})^2}} \quad (3-2)$$

其中  $R_{X_i}$  和  $R_{Y_i}$  分别是观测值  $i$  的取值的等级， $\overline{R_X}$  和  $\overline{R_Y}$  分别是变量  $X$  和变量  $Y$  的平均等级，更简单化后，公式可变为：

$$\rho_S = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)} \quad (3-3)$$

$n$  是观测值的总数量,  $d_i = R_{X_i} - R_{Y_i}$  表示二列成对变量的等级差数。

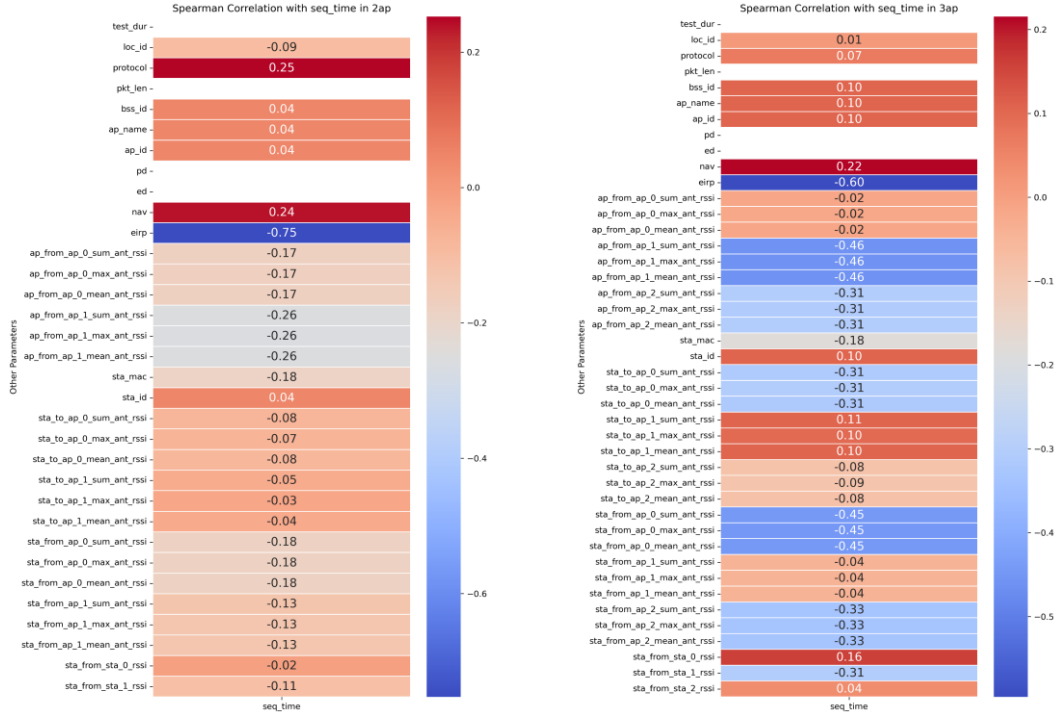


图 3.6 发送机会的斯皮尔曼相关系数

(3) 肯德尔相关性系数(Kendall's correlation coefficient), 又称肯德尔系数(Kendall's Tau), 它也是一种用于衡量两个变量之间相关性的非参数统计量, 但所计算的对象为分类变量。它不需要假设数据的分布形式, 因此在小样本或有离群值的情况下更为稳健。肯德尔相关系数的取值范围在-1 到 1 之间, 当  $Tau$  为 1 时, 表示两个随机变量拥有一致的相关性; 当  $Tau$  为-1 时, 表示两个随机变量拥有完全相反的相关性; 当  $Tau$  为 0 时, 表示两个随机变量没有相关性, 即相互独立。

公式一:

$$Tau - a = \frac{2(n_c - n_d)}{n(n-1)} \quad (3-4)$$

其中  $n_c$  表示  $XY$  中拥有一致性的元素对数(两个元素为一对);  $n_d$  表示  $XY$  中拥有不一致性的元素对数, 注意: 这一公式仅适用于集合  $X$  与  $Y$  中均不存在相同元素的情况(集合中各个元素唯一)

公式二:

$$Tau - b = \frac{2(n_c - n_d)}{(n_0 - n_1)(n_0 - n_2)} \quad (3-5)$$

其中,  $n_0 = \frac{n(n-1)}{2}$ ,  $n_1$  是与第一个变量相关的重复分组的调整项,  $n_2$  是与第二个变量相关的重复分组的调整项。注意: 这一公式适用于集合  $X$  或  $Y$  中存在相同元素的情况(当然, 如果  $X$  或  $Y$  中均不存在相同的元素时, 公式二便等同于公式一)。

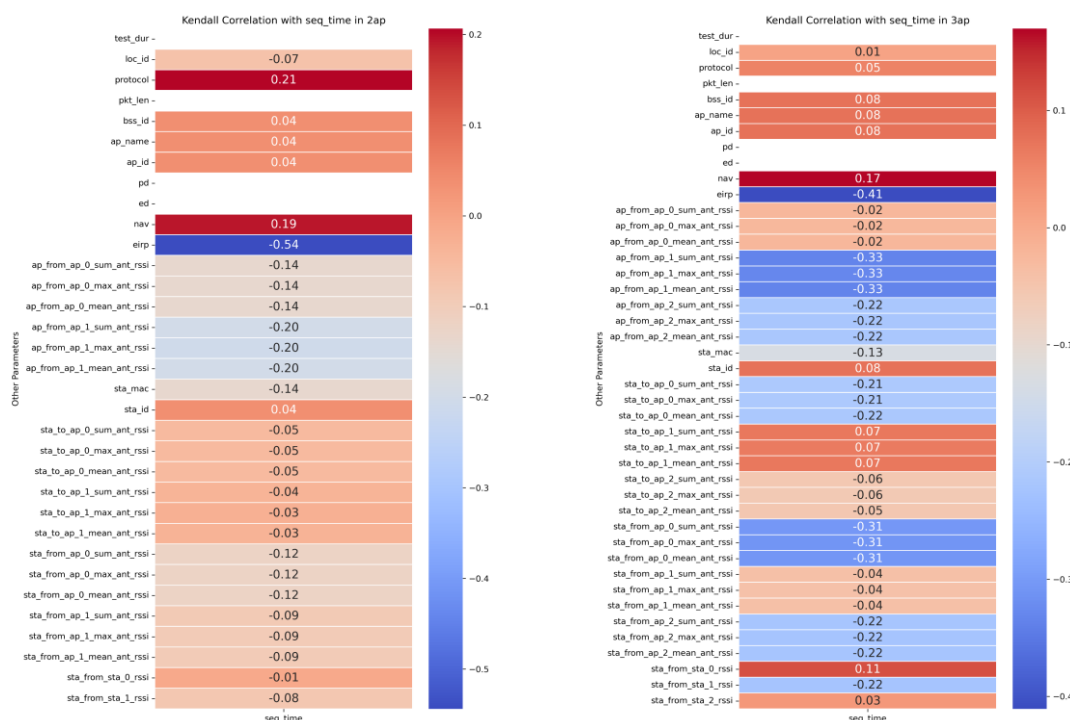


图 3.7 发送机会的肯德尔相关系数

由于非参数相关(指 Spearman 和 Kendall)在计算中使用的信息较少,所以它们的表达能力相对较弱。在 Pearson 的情况下,相关性计算采用均方差和标准差等信息,而非参数相关性仅使用序数信息和成对分数。在非参数相关的情况下,X 和 Y 值可能是连续的且有序的,并且不需要 X 和 Y 的近似正态分布。但在 Pearson 相关的情况下,它假定 X 和 Y 的分布应该是正态分布,并且也应该是连续的。

在正常情况下, Kendall 相关性比 Spearman 相关性更强健和有效。这意味着当样本量较小或存在一些异常值时,首选 Kendall 相关。在所有情况下, Kendall 相关系数的绝对值均小于其他绝对值。

### 3.2.3 影响性强弱排序

结合 3.2.2 的三大相关系数图及问题文档中所给背景信息,我们发现 AP 的发送机会(即数据帧序列的总时长 seq\_time)受网络拓扑、业务流量、门限、节点间 RSSI 的测试基本信息等多个因素影响。通过分析 3 个 AP 与 2 个 AP 情况下的三大相关系数图,我们发现节点信息中的发送功率 erip 为影响 AP 发送机会的关键因素。随着 erip 的增加,AP 的发送机会逐渐减少,这表明 AP 发送数据帧序列的功率增大,STA 能够更加有效的检测到 AP 所送的信号,从而减少所需的总时长。其次,NAV 门限和 AP 与 STA 检测各 AP 的 RSSI 为次要影响因素,随着 NAV 门限的增加,判决要求增高,导致低于门限的 RSSI 信号被直接丢弃,进而减少 AP 的发送机会。随着在三种计算方式下 STA 检测到 AP 的 RSSI 增加,STA 可以接受到更加微弱的发送信号,即使存在干扰信号的情况下,仍能较好地接收到指定 AP 发送的信号,这进一步降低了 AP 的发送机会,缩短了总时长。系数图中其他信息具有较弱的影响强度。在题目附件所给数据中,由于数据长度、包检测门限和能量检测门限的数据一直保持不变,因此,根据系数图无法得出则三种信息对 AP 发送机会的影响强弱。但根据题目材料可以看出,以上三种信息影响着发送信息的强弱和信息的丢包率,因此对 AP 发送机会的影响必然存在。

综上所述,我们对 AP 发送机会的综合影响程度由强到弱排序如下:

1. AP 的发送功率 `erip`
2. AP 各天线检测到来自 AP\_x 的 RSSI，即 `ap_from_ap_x_sum_ant_rssi`、`ap_from_ap_x_max_ant_rssi` 和 `ap_from_ap_x_mean_ant_rssi`
3. STA 各天线收到来自 AP\_x 的 RSSI，即 `sta_from_ap_x_sum_ant_rssi`、`sta_from_ap_x_max_ant_rssi` 和 `sta_from_ap_x_mean_ant_rssi`
4. NAV 门限
5. AP 发送的流量类型 `protocol`，为 TCP 或 UDP。
6. 聚合前的数据长度 `pkt_len`
7. STA 各天线收到 STA\_x 的 RSSI 之和，即 `sta_from_sta_x_rssi`
8. 能量检测门限 `ed`、包检测门限 `pd`
9. 网络拓扑信息，即 `bss_id`、`ap_id` 等

### 3.2.4 数据预处理

#### (1)异常值检测及处理

首先对数据中的异常值进行检测，我们得到所给测试数据中四个测试集和12个训练集中均不存在数据异常状况，而`training_set_2ap_loc2_nav82`训练集中存在两行数据中的`rssi`信息、`sta_mac`和`sta_id`出现异常，对于此类异常数据，我们进行删除处理。

表3.1为检测出的异常数据展示(红色字体为数据异常值)，我们对`training_set_2ap_loc2_nav82`训练集的此类异常数据进行整行删除操作。

表 3.1 检测出的数据异常值

| <code>eirp</code>                      | <code>ap_from_ap_0_sum_ant_rssi</code>  | <code>ap_from_ap_0_max_ant_rssi</code> | <code>ap_from_ap_0_mean_ant_rssi</code> | <code>ap_from_ap_1_sum_ant_rssi</code> |
|----------------------------------------|-----------------------------------------|----------------------------------------|-----------------------------------------|----------------------------------------|
| 29                                     | 471f                                    | sta_1                                  | 2                                       | 11                                     |
| 29                                     | 471f                                    | sta_1                                  | 2                                       | 11                                     |
| <code>ap_from_ap_1_max_ant_rssi</code> | <code>ap_from_ap_1_mean_ant_rssi</code> | <code>sta_mac</code>                   | <code>sta_id</code>                     | <code>sta_to_ap_0_sum_ant_rssi</code>  |
| 0.12                                   | 16                                      | 0.003477                               | 0                                       | 33.22                                  |
| 0.11                                   | 14                                      | 0.003764                               | 0                                       | 37.12                                  |

#### (2)缺失值检测及处理

处理完数据异常之后，我们对训练集和测试集中空缺的数据进行检测和处理。数据集中空缺数值全位于 `ap` 间信号或 `sta` 接受信号的 `rssi` 列中，即为为接收到发送信号，根据 WLAN 信号的常规知识，我们选择了一个比数据集中任何发送信号都要小的信号强度进行填充，其值设置为-100dBm，用于表示无信号强度。检测缺失值和填补后数据如表 3.2 和表 3.3 所示：

表 3.2 检测出的缺失数据

| nav | eirp | ap_from_ap_0 |     |     | ap_from_ap_0 |     |     | ap_from_ap_0 |     |     |
|-----|------|--------------|-----|-----|--------------|-----|-----|--------------|-----|-----|
|     |      | sum          | ant | rss | max          | ant | rss | mean         | ant | rss |
| -82 | 9    |              |     |     |              |     |     |              |     |     |
| -82 | 10   |              |     |     |              |     |     |              |     |     |

表 3.3 填补缺失数据

| nav | eirp | ap_from_ap_0 |     |     | ap_from_ap_0 |     |     | ap_from_ap_0 |     |     |
|-----|------|--------------|-----|-----|--------------|-----|-----|--------------|-----|-----|
|     |      | sum          | ant | rss | max          | ant | rss | mean         | ant | rss |
| -82 | 9    | -100         |     |     | -100         |     |     | -100         |     |     |
| -82 | 10   | -100         |     |     | -100         |     |     | -100         |     |     |

### (3)合并相同 AP 数量训练集及处理 RSSI 数据

处理完缺失数据及异常数据后，我们根据训练集中的AP数量将5个AP数为2和8个AP数为3的数据集进行合并，整合为一个AP数为2的训练集和一个 AP数为3的训练集为不同AP数量的测试集预测做准备。

其次，由于训练数据中所给的RSSI数据为字符串数组，我们将训练集中每个任务的不同RSSI数组进行求和取均值，作为所对应的RSSI位的特征数据。

如下为整合后数据统计数量：training\_set\_3ap\_loc30\_nav82

表 3.4 整合数据统计表

| AP 数量 | 任务数量 |
|-------|------|
| 2     | 392  |
| 3     | 858  |

以下为以training\_set\_2ap\_loc0\_nav82为例的AP数量为2的RSSI数据处理分析图，其中展示了20个测试用例数据：

#### 1. AP数量为2，AP间的信号强度数据示意图：

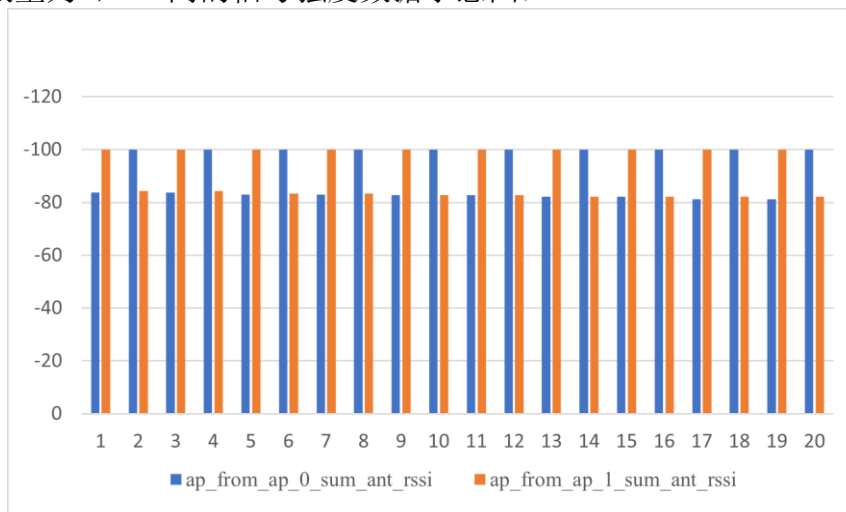


图 3.8 2AP 下 AP 检测到 AP 的 RSSI 之和

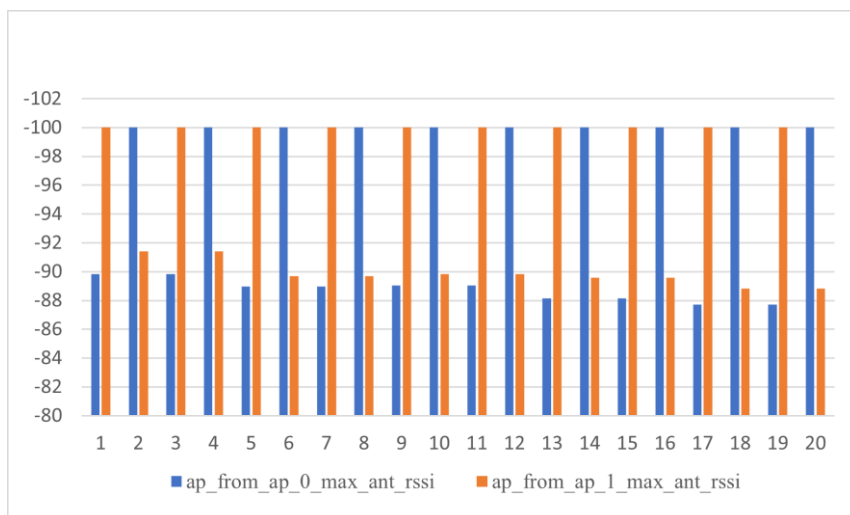


图 3.9 2AP 下 AP 检测到 AP 的 RSSI 最大值

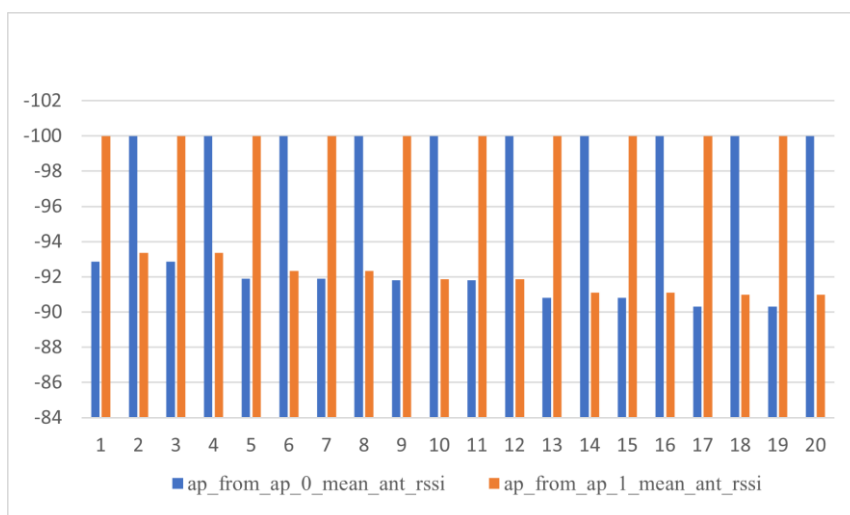


图 3.10 2AP 下 AP 检测到 AP 的 RSSI 均值

2. AP数量为2，STA与AP的信号强度数据示意图：

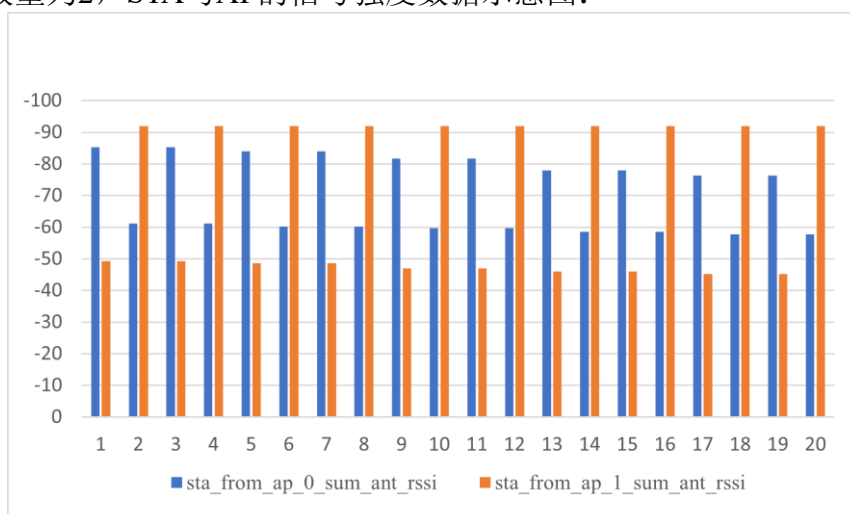


图 3.11 2AP 下 STA 接收到 AP 的 RSSI 之和

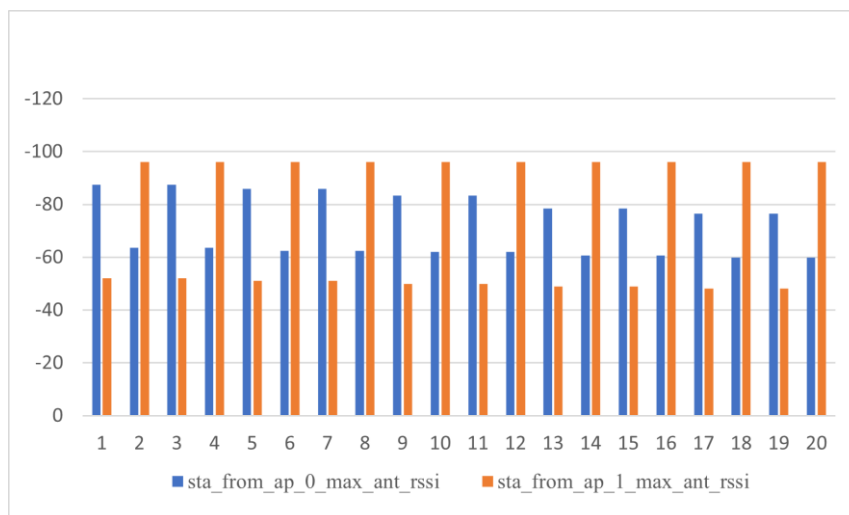


图 3.12 2AP 下 STA 接收到 AP 的 RSSI 最大值

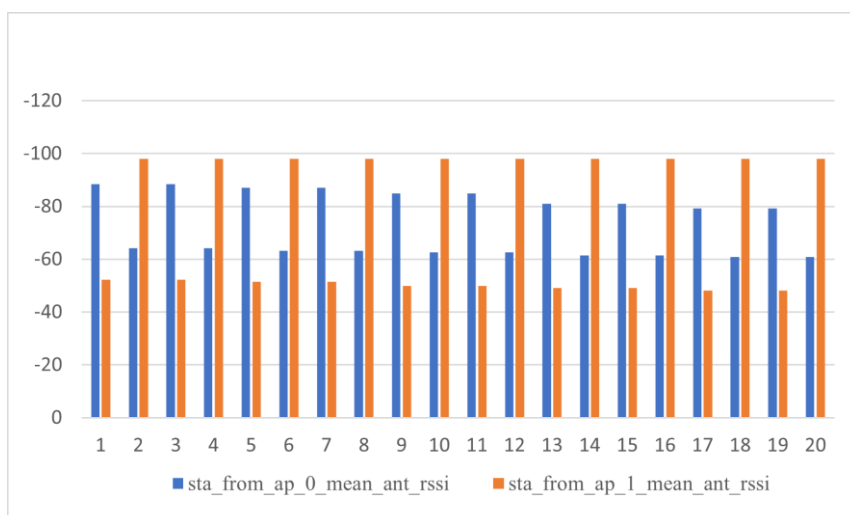


图 3.13 2AP 下 STA 接收到 AP 的 RSSI 均值

以下为以 training\_set\_2ap\_loc0\_nav82 为例的 AP 数量为 3 的 RSSI 数据处理分析图，其中展示了 20 个测试用例数据：

1. AP 数量为 3，AP 间的信号强度数据示意图：

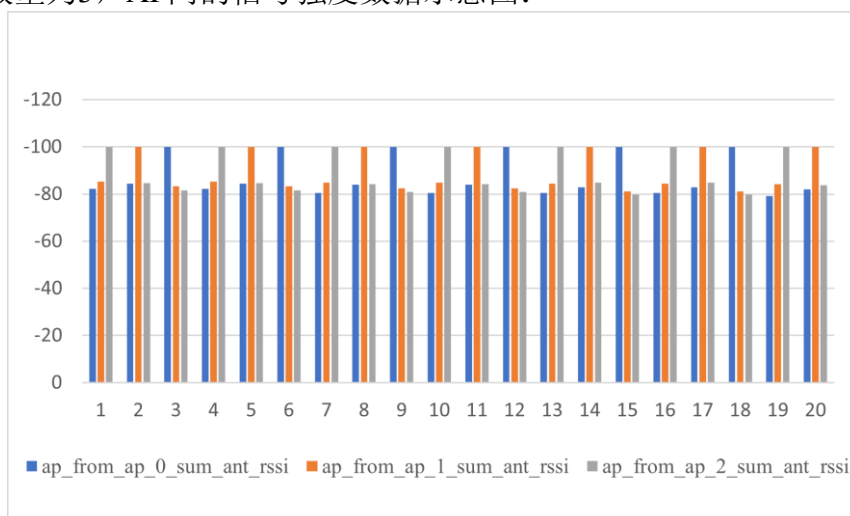


图 3.14 3AP 下 AP 检测到 AP 的 RSSI 之和

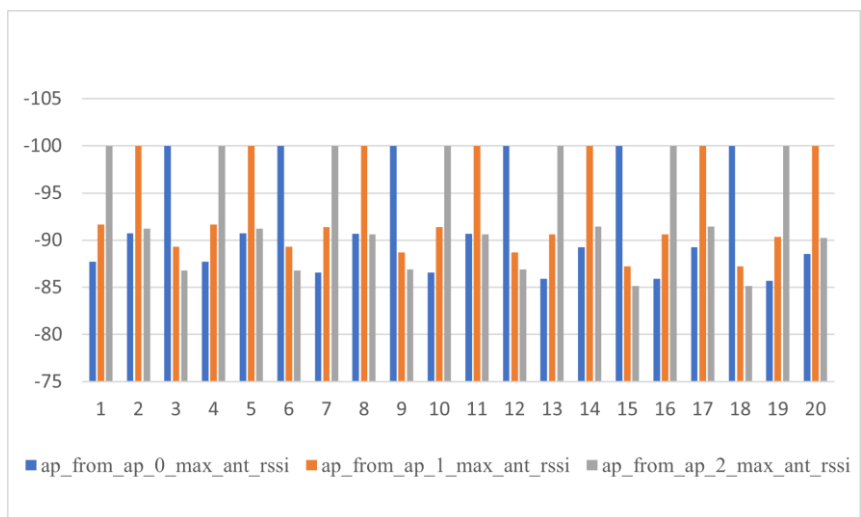


图 3.15 3AP 下 AP 检测到 AP 的 RSSI 最大值

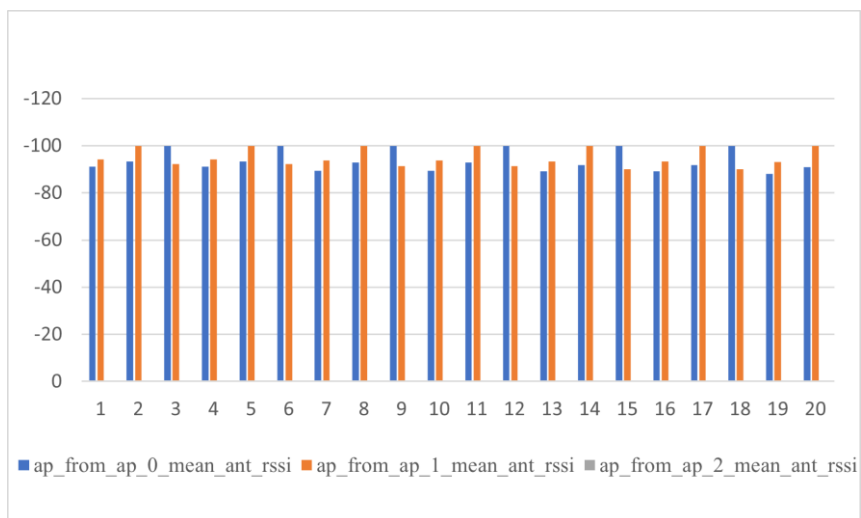


图 3.16 3AP 下 AP 检测到 AP 的 RSSI 均值

## 2. AP数量为3，STA与AP的信号强度数据示意图：

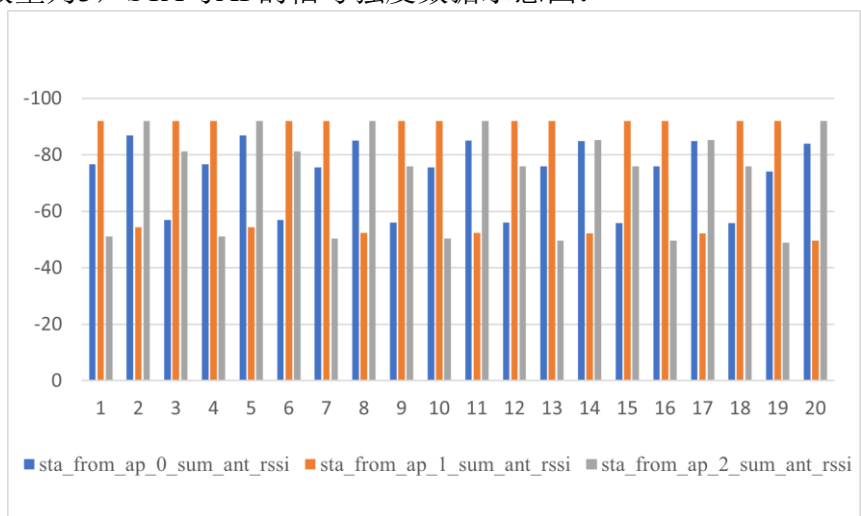


图 3.17 3AP 下 STA 接收到 AP 的 RSSI 之和

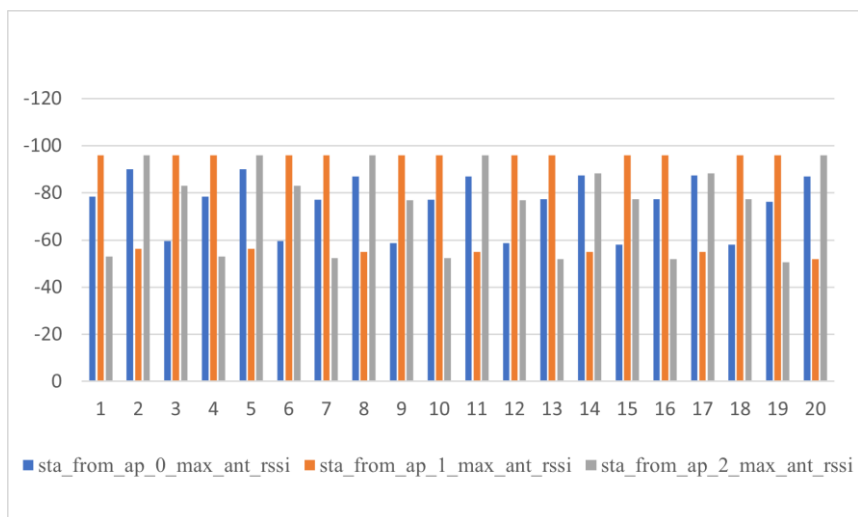


图 3.18 3AP 下 STA 接收到 AP 的 RSSI 最大值

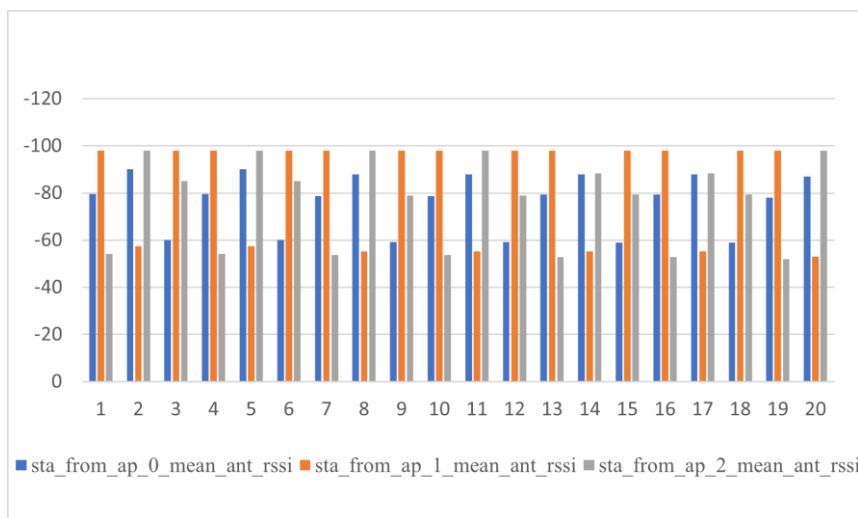


图 3.19 3AP 下 STA 接收到 AP 的 RSSI 均值

其余数据集中 RSSI 数据均以上述处理方案相同，故不做演示。

### 3.3 模型建立

#### 3.3.1 建模目标

建立一个数学模型，根据选择的特征数据，预测每个 AP 的 seq\_time，即发送数据帧序列的总时长。

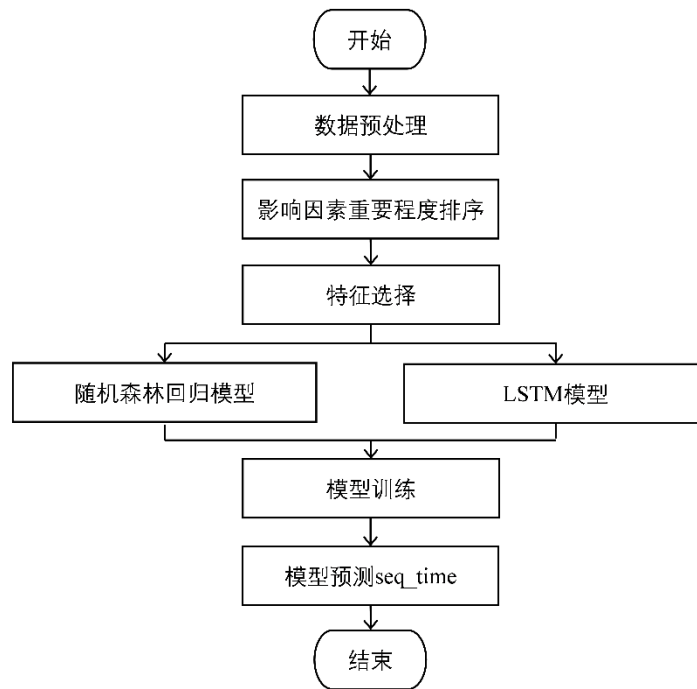


图 3.20 问题一的模型建立流程图

### 3.3.2 特征选择

根据 3.2.2 相关性分析和 3.2.3 影响性强弱排序，我们选择以下特征作为预测总时长 seq\_time 的模型的输入数据：

#### (1) RSSI 相关特征：

- erip: AP 的发送功率
- ap\_from\_ap\_x\_max\_ant\_rssi: AP 各天线检测到来自其他 AP 的 RSSI 最大值
- ap\_from\_ap\_x\_mean\_ant\_rssi: AP 各天线检测到来自其他 AP 的 RSSI 平均值
- sta\_from\_ap\_x\_sum\_ant\_rssi: STA 各天线接收到来自其他 AP 的 RSSI 之和
- sta\_from\_ap\_x\_max\_ant\_rssi: STA 各天线接收到来自其他 AP 的 RSSI 最大值
- sta\_from\_ap\_x\_mean\_ant\_rssi: STA 各天线接收到来自其他 AP 的 RSSI 平均值

#### (2) 门限相关特征：

- nav: NAV 门限

#### (3) 业务流量特征：

- protocol: AP 发送的流量类型(UDP/TCP)

#### (4) 网络拓扑特征：

- loc\_id: AP 和 STA 部署在某一位置的编号
- bss\_id: BSS 编号

### 3.3.3 随机森林回归模型

随机森林算法是目前最常用到的强大监督学习方法之一，能够同时解决回归和分类问题<sup>[3]</sup>。其核心思想基于集成学习，通过将多棵决策树组合起来提高预测效果。对于分

类问题，随机森林通过多数投票的方式确定最终分类结果；而在回归问题中，则将各棵决策树的预测结果取平均值来得到最终的回归预测。通常情况下，决策树的数量越多，随机森林的鲁棒性和准确性也越高。

$$P_i = \frac{|C_{i,D}|}{|D|} \quad (3-6)$$

$$Gini(D) = 1 - \sum_{i=1}^m P_i^2 \quad (3-7)$$

$D$  是 WLAN 网络实测训练数据集， $P_i$  是数据集  $D$  中记录属于  $C_i$  类的概率， $|D|$  是数据集  $D$  中记录的总条数， $|C_{i,D}|$  是数据集  $D$  中属于  $C_i$  类的记录的条数， $m$  是数据集的个数。针对 WLAN 实测信号的数据集，假设里面含有  $K$  个特征。对于随机森林而言我们有以下几个步骤：

**Step1:** 第一步采用 *Bagging* 的想法，从 WLAN 网络实测数据集中进行随机挑选和放回操作，并进行  $n$  次过程，生成与原始数据集规模相同的  $n$  个训练子集  $\{D_1, D_2, \dots, D_n\}$ 。

**Step2:** 从第一步得到的  $n$  个 WLAN 网络实测的训练子集中，选择一个训练子集构建决策树，每棵决策树在构建过程中，随机选取  $k$  个特征条件 ( $k < K$ )，利用这  $k$  个特征选取进行最佳分裂点的选择，不断重复该过程，直到满足停止条件。

**Step3:** 在上述的步骤中会产生  $n$  个决策树，这些决策树共同构成了随机森林模型。

**Step4:** 当输入新的测试数据时，随机森林会通过每棵决策树的预测结果，进行投票或取平均值，从而输出最终的预测结果。

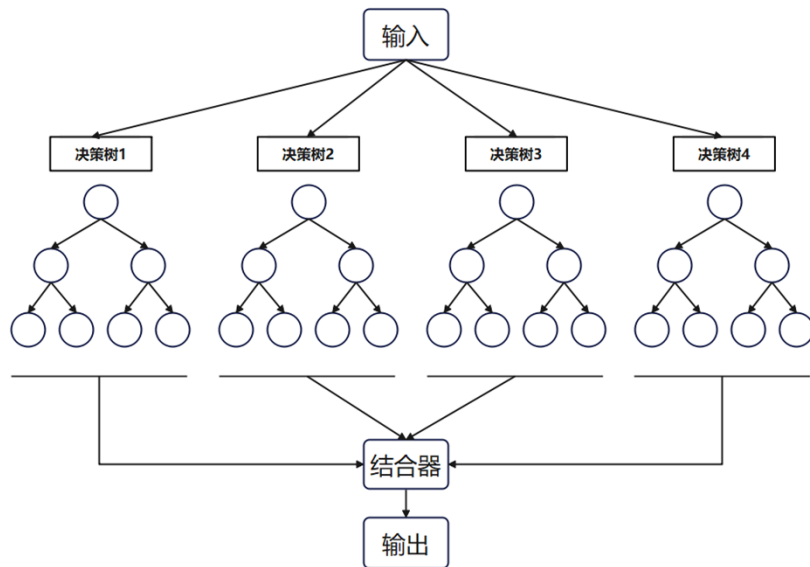


图 3.21 随机森林概念图

随机森林采用多个决策树组合，对每个样本采用随机的有放回策略进行选择，并从样本中随机选择一些特征，上图展示了随机森林演算法的特定过程，在此过程中，回归问题则是将多个回归结果的平均。

### 3.3.3 长短时记忆网络模型

长短期记忆网络(LSTM)是一种改进的递归神经网络(RNN)，主要针对传统 RNN 中

的梯度消失问题设计。在处理序列数据时，RNN 的训练难以捕捉长距离依赖，因为在反向传播过程中，随着时间步长的增加，梯度会逐渐减小，导致无法有效更新远距离的权重。LSTM 通过引入“记忆单元”和三个门(输入门、输出门、遗忘门)来控制信息流动，解决了这个问题。其具体结构如下图所示。

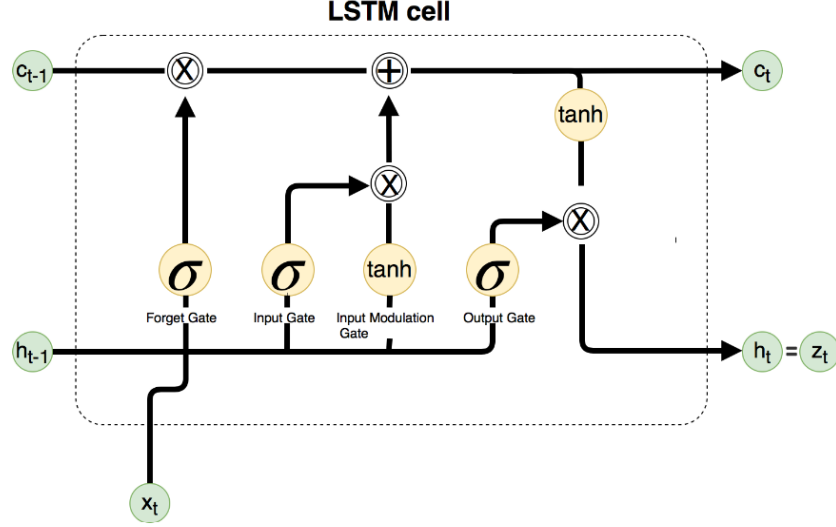


图 3.22 LSTM 概念图

与简单 RNN 不同的是，LSTM 引入了一个专门用于线性循环信息传递的内部状态。这一内部状态用于处理长期依赖，同时网络也通过非线性方式将信息传递给外部的隐藏状态。LSTM 模型中的“门”机制是一种软门，其取值范围介于 0 到 1 之间，控制信息传递的比例。

遗忘门负责决定从前一时刻的内部状态中遗忘多少信息；输入门则调节当前时刻的候选状态应该保留多少新信息；输出门则决定从内部状态中输出多少信息给外部的隐藏层。如果遗忘门的输出接近 0 而输入门的输出接近 1，则表示记忆单元会清除旧信息并写入新的候选状态。相反，如果遗忘门输出为 1 而输入门为 0，表示记忆单元保留先前的内容而不更新。在 RNN 中，隐状态包含过去的历史信息，可以看作是短期记忆，因为每个时刻隐状态都会被更新。而长期记忆则体现在网络的参数中，这些参数在训练过程中逐步积累经验，但其更新频率远远低于短期记忆。LSTM 的记忆单元可以在某一时刻捕捉到重要信息，并利用这些单元在一定的时间范围内保存这些信息。与短期记忆相比，LSTM 中的记忆单元保存信息的时间更长，但仍短于长期记忆，因此被称为长的短期记忆。

使用 LSTM 模型预测 WLAN 中 AP 发送机会(seq\_time)时，在每个时间步  $t$ ，有一组影响因素  $x_t$ ，模型通过这些输入来预测下一个时间步的发送机会<sup>[4]</sup>。在时间步  $t$  的输入特征向量  $x_t$  可以表示为：

$$x_t = [RSSI_t, PD_t, ED_t, NAV_t, Protocol_t, Pkt\_len_t, EIRP_t]$$

其中， $RSSI_t$  是节点间接收信号强度， $PD_t, ED_t$  分别是物理和能量检测门限， $NAV_t$  是网络分配矢量， $Protocol_t$  是业务流量协议类型， $Pkt\_len_t$  为数据包长度， $EIRP_t$  为 AP 发送功率。

LSTM 单元的状态更新计算公式为：

$$\begin{aligned} f_t &= \sigma(W_{fx}x_t + W_{fh}h_{t-1} + b_f) \\ i_t &= \sigma(W_{ix}x_t + W_{ih}h_{t-1} + b_i) \\ g_t &= \tanh(W_{gx}x_t + W_{gh}h_{t-1} + b_g) \\ o_t &= \sigma(W_{ox}x_t + W_{oh}h_{t-1} + b_o) \end{aligned} \quad (3-8)$$

$$o_t = \sigma(W_{ox}x_t + W_{oh}h_{t-1} + b_o)$$

$$st = gt \cdot it + s_{t-1} \cdot ft$$

$$ht = \phi(st) \cdot ot$$

并且,  $x_t \in R^k$  表示  $t$  时刻的向量的输入:  $W_{fx}$ 、 $W_{fh}$ 、 $W_{ix}$ 、 $W_{ih}$ 、 $W_{gx}$ 、 $W_{gh}$ 、 $W_{gx}$ 、 $W_{oh}$  是权重矩阵:  $b_f$ 、 $b_i$ 、 $b_g$ 、 $b_o$  是对应权重的偏向量。 $\sigma$  是 *sigmoid* 函数:  $\phi$  是 *tanh* 函数;  $\cdot$  表示点积。 $ht$  是  $t$  时刻以及之前时刻储存了所有有用信息的隐状态向量。

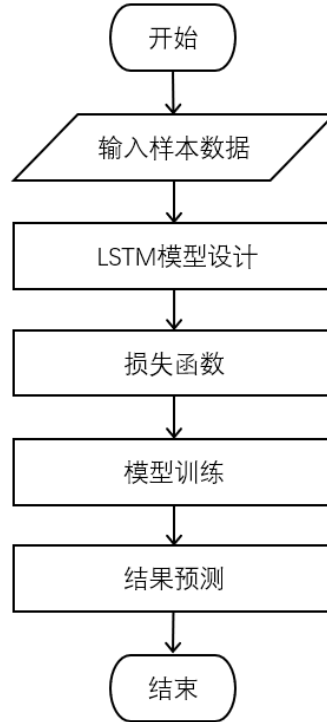


图 3.23 LSTM 模型流程图

如上图所示, LSTM 模型流程为样本数据的输入、LSTM 模型的设计、损失函数的计算、对模型进行训练、预测最终结果。

### 3.4 模型求解结果

#### (1) 随机森林模型求解:

我们使用 python 将整合后的 2AP 和 3AP 训练集以 8:2 的比例划分训练集和验证集, 分别将训练集放入随机森林模型进行训练, 通过验证集对模型性能进行评判分析, 以下是验证集下的模型效果分析, 图中展示了随机森林回归模型对 50 个测试用例的预测值与真实值的对比:

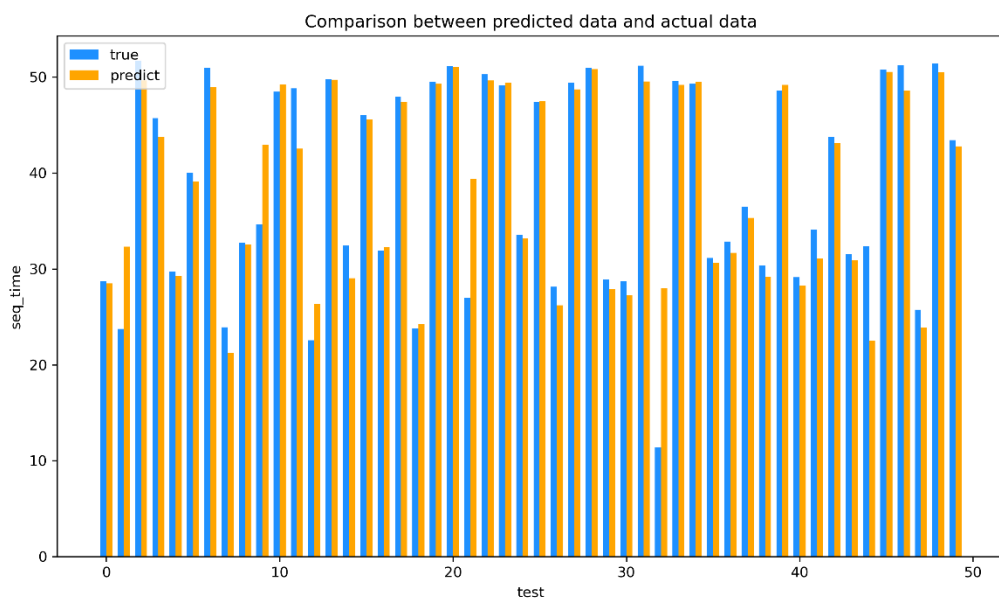


图 3.24 2AP 下随机森林预测与真实 seq\_time 对比图

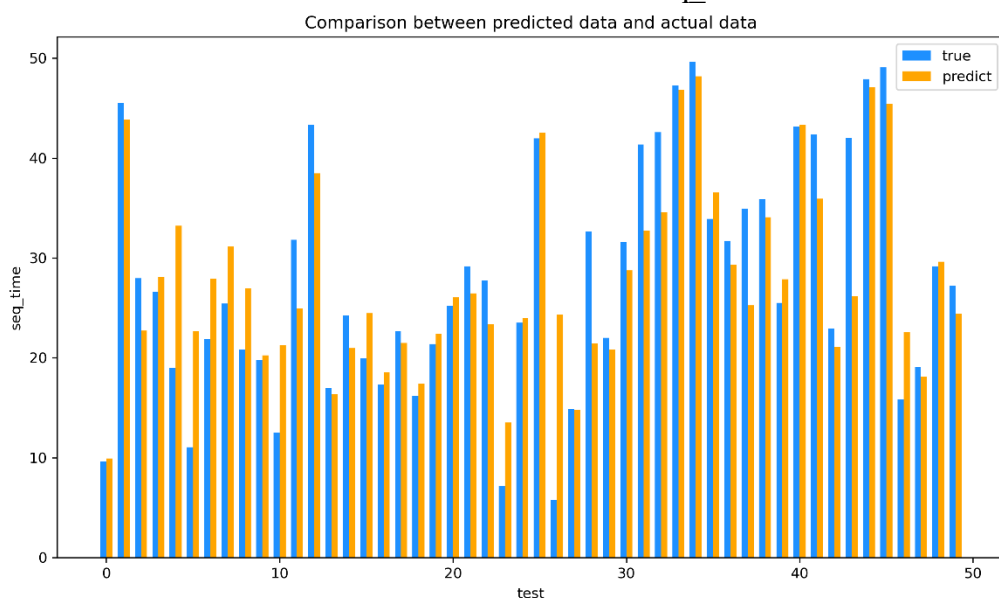


图 3.25 3AP 下随机森林预测与真实 seq\_time 对比图

表 3.5 随机森林效果分析

| AP 数量 | n   | MSE   | MAE  |
|-------|-----|-------|------|
| 2     | 79  | 11.12 | 1.76 |
| 3     | 172 | 42.51 | 4.76 |

其中,  $n$  为验证集的样本数, 体现在图上为柱状图中蓝色或橙色的验证总数量; MSE 为均方误差, 是所有预测误差的平方和的平均值; MAE 为平均绝对误差, 为所有预测误差的绝对值之和的平均值。

## (2) LSTM 模型求解:

我们采用与随机森林相同比例划分训练集和验证集, 使用 python 将训练集放入 LSTM 模型进行训练, 使用 Adam 优化器进行模型优化, 设置学习率为 0.005, 通过验

证集对 200 趟训练模型进行择优选择，并评判分析，以下是验证集下的模型效果分析，图中展示了 LSTM 模型对 50 个测试用例的预测值与真实值的对比：

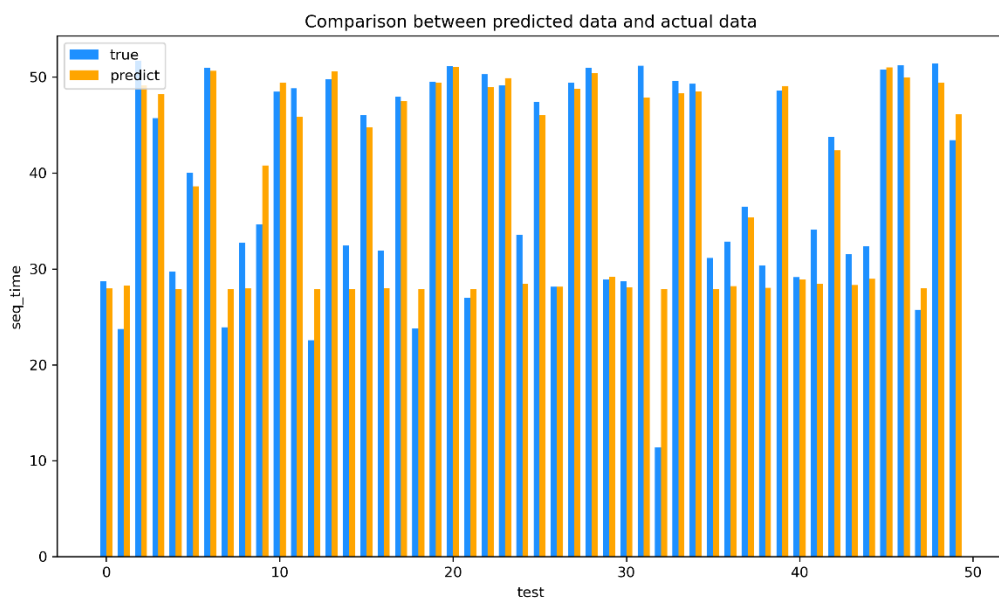


图 3.26 2AP 下 LSTM 预测与真实 seq\_time 对比图

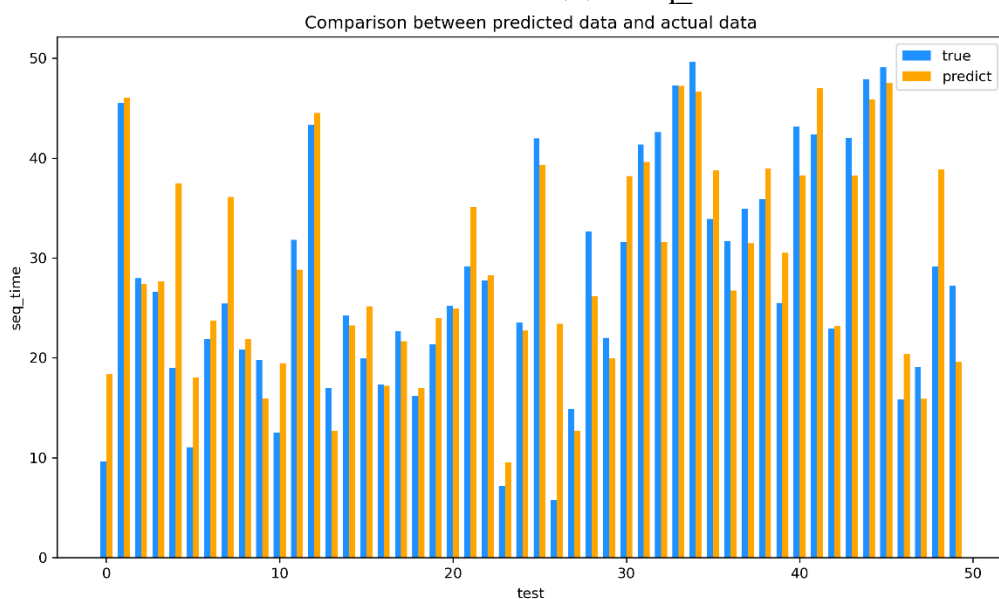


图 3.27 3AP 下 LSTM 预测与真实 seq\_time 对比图

表 3.6 LSTM 效果分析

| AP 数量 | LSTM 层数   | 隐藏神经元     | MSE         |
|-------|-----------|-----------|-------------|
| 2     | 1         | 50        | 10.04       |
|       | 5         | 50        | 8.99        |
|       | <b>10</b> | <b>50</b> | <b>8.12</b> |
| 3     | 1         | 50        | 41.81       |

|          |           |              |
|----------|-----------|--------------|
| <b>5</b> | <b>50</b> | <b>37.90</b> |
| 10       | 50        | 42.76        |

根据两种模型训练后通过验证集得出的 MSE 指标，我们选择用具有 10 层 50 个隐藏神经元的 LSTM 模型预测 test\_set\_1\_2ap，用具有 5 层 50 个隐藏神经元的 LSTM 模型预测 test\_set\_1\_3ap。以下为测试集 test\_set\_1\_2ap 和 test\_set\_1\_3ap 中部分预测 seq\_time，全部预测数据请查看本文附录：

表 3.7 test\_set\_1\_2ap 部分预测总时长

| test_id | loc_id | protocol | bss_id | predict seq_time |
|---------|--------|----------|--------|------------------|
| 1       | loc3   | tcp      | 1      | 48.69            |
| 1       | loc3   | tcp      | 0      | 47.21            |
| 2       | loc3   | udp      | 1      | 49.59            |
| 2       | loc3   | udp      | 0      | 49.59            |
| 3       | loc3   | tcp      | 1      | 48.66            |
| 3       | loc3   | tcp      | 0      | 47.27            |
| 4       | loc3   | udp      | 1      | 49.56            |
| 4       | loc3   | udp      | 0      | 49.58            |

表 3.8 test\_set\_1\_3ap 部分预测总时长

| test_id | loc_id | protocol | bss_id | predict seq_time |
|---------|--------|----------|--------|------------------|
| 1       | loc33  | tcp      | 2      | 21.32            |
| 1       | loc33  | tcp      | 1      | 42.2             |
| 1       | loc33  | tcp      | 0      | 38.53            |
| 2       | loc33  | udp      | 2      | 29.63            |
| 2       | loc33  | udp      | 1      | 35.52            |
| 2       | loc33  | udp      | 0      | 38.12            |
| 3       | loc33  | tcp      | 2      | 21.19            |

## 四、问题二模型建立与求解

### 4.1 问题背景及目标

**背景：**在无线局域网中，AP 的传输速率自适应调整过程通过 AMC(自适应调制编码)算法实现。AMC 算法基于环境中的 SINR(信干噪比)动态调整 AP 的传输速率，而不同的 MCS(调制和编码方案)和 NSS(空间流数)决定了 PHY Rate(物理层速率)的选择。由于 WLAN 通信环境复杂，AP 在传输过程中可能多次切换 MCS 和 NSS 组合。选用次数最多的 MCS 和 NSS 能够反映 AP 所处环境的 SINR 水平，因此准确预测 AP 发送数据时使用的(MCS,NSS)对 WLAN 系统的性能和吞吐量具有关键影响。

**目标：**AP 发送数据的(MCS,NSS)建模和预测。基于实测数据中的 RSSI 和门限信息，结合问题一对 AP 发送机会的分析，建立模型，预测 AP 在传输过程中使用的 MCS 和 NSS 组合。使用训练数据训练模型并在测试数据上进行预测(MCS,NSS)。

### 4.2 问题分析与建模思路

在 WLAN 系统中，AP 的 PHY 速率是由传输过程中所选用的调制编码方案 MCS 和空间流数量 NSS 决定的。PHY 速率不仅决定了每个 AP 的吞吐量，还受信道条件、干扰情况、节点间 RSSI 等因素的影响。问题二要求通过 WLAN 实测数据，特别是节点间 RSSI 和信号门限信息，对 AP 在发送数据时采用的(MCS, NSS)组合进行建模和预测。

以下根据提供的实测训练数据和 WLAN 网络示意图，分析得出 AP 发送数据的(MCS,NSS)的影响参数主要有以下几点：

- 信干噪比(SINR)
- 节点间 RSSI
- 传输方式(同步/异步)
- 门限参数

#### 4.2.1 MCS 和 NSS 选择影响因素分析

下面对预测 AP 发送数据的(MCS,NSS)主要的影响参数进行一一分析：

**1.信干噪比(SINR):** SINR 是信号强度与干扰和噪声强度的比值，直接影响调制编码方案的选择。较高的 SINR 允许 AP 选择更高的 MCS 等级(更高的调制阶数和更密集的编码)，从而提高 PHY 速率。相反，低 SINR 则要求选择低阶的 MCS 组合来确保数据传输的可靠性。因此，SINR 是(MCS, NSS)组合的关键影响因素。

**2.节点间 RSSI:** RSSI 表示接收信号的强度，通常反映了通信链路的质量。虽然 RSSI 本身不能直接决定 MCS 的选择，但较强的 RSSI 通常伴随较高的 SINR，从而影响 MCS 的选择。在 RSSI 较高的情况下，AP 可以使用较高的调制编码等级和多个空间流(NSS)，提升传输效率。

**3.传输方式(同步/异步):** 在同步传输中，多个 AP 和 STA 之间的传输协同工作，减少了信道的竞争和干扰，从而提高了有效的 SINR，允许采用更高的 MCS 和更多的 NSS 组合。而在异步传输中，信道竞争和干扰较为严重，可能导致更低的 MCS 等级和更少的空间流数<sup>[5]</sup>。

**4.门限参数(CCA、PD、ED 门限等):** 门限参数决定了 AP 对信道繁忙状态的判断。当门限设置较为敏感(门限值较低)时，AP 更容易认为信道繁忙，导致更少的传输机会和

更低的 PHY 速率。因此，门限参数也会间接影响 MCS 和 NSS 的选择。

## 4.2.2 相关性分析

### (1) 皮尔森相关系数

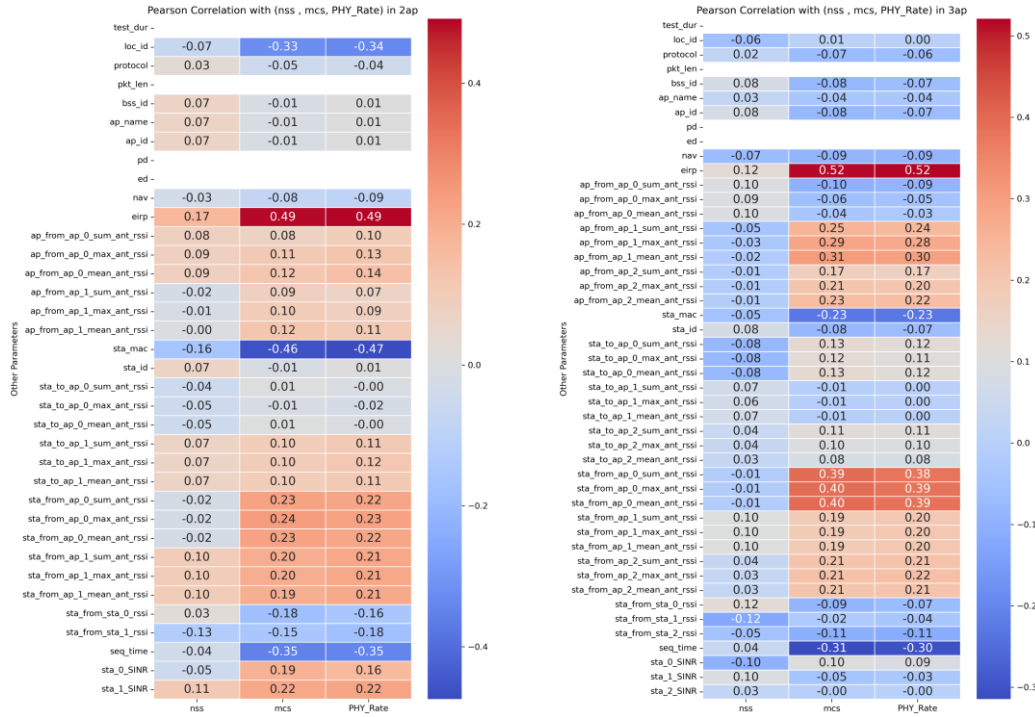


图 4.1 NSS、MCS 及 PHY Rate 的皮尔森相关系数

### (2) 斯皮尔曼秩相关系数

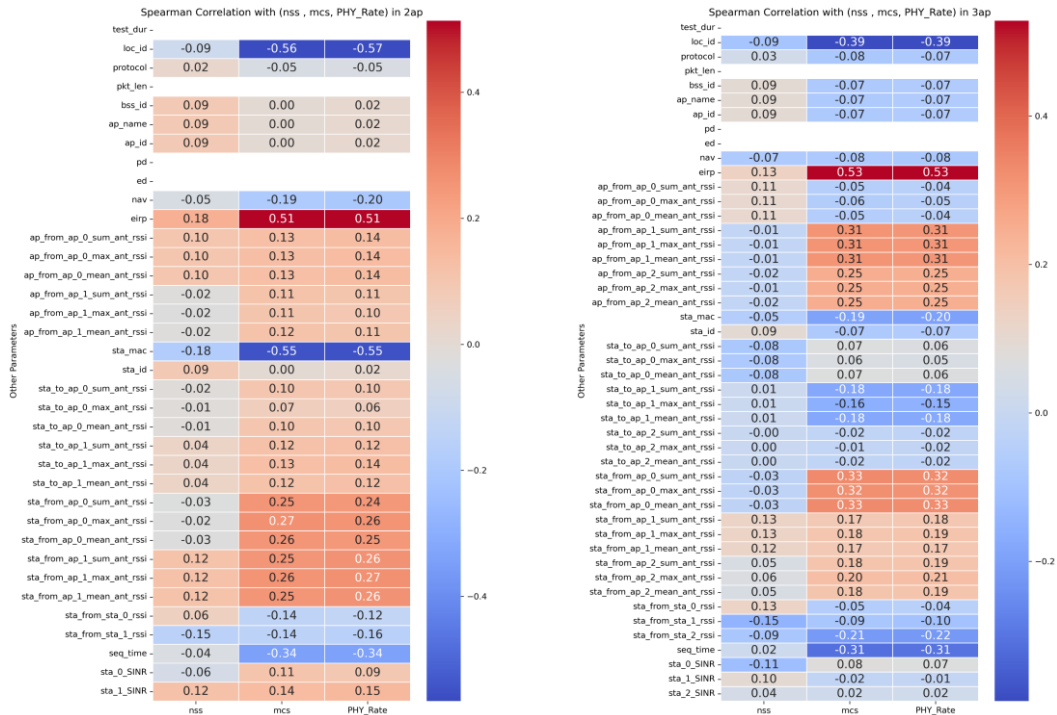


图 4.2 NSS、MCS 及 PHY Rate 的斯皮尔曼相关系数

### (3)肯德尔相关性系数

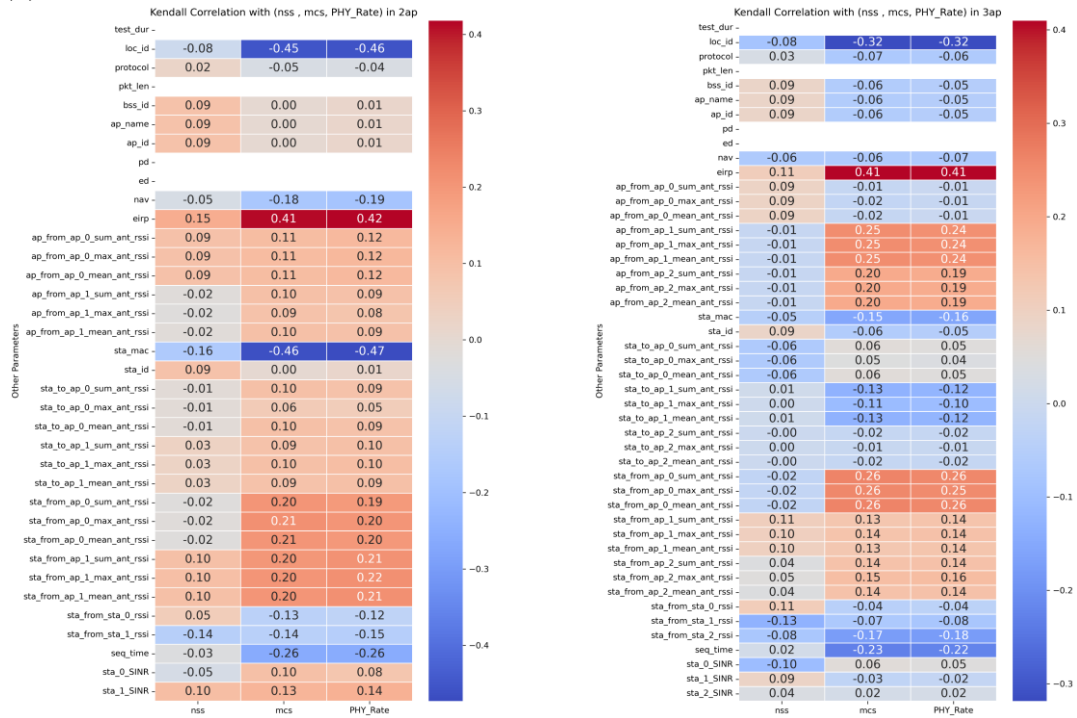


图 4.3 NSS、MCS 及 PHY Rate 的肯德尔相关系数

根据以上三大相关系数图，我们可以得出(NSS, MCS)组合，即 PHYRate，的主要因素为发射功率 erip、STA 接收到的 AP\_x 的 RSSI 和 AP 检测到 AP\_x 的 RSSI。在固定 20MHz 带宽时，随着这三类数据的增加，MCS 等级增加，从而导致发送数据的 PHY Rate 升高。

## 4.3 模型建立

### 4.3.1 建模目标

建立一个模型，利用训练集中的测试基本信息，特别是节点间 RSSI 和门限信息，预测 AP 发送数据时最常用的 (MCS, NSS) 组合。

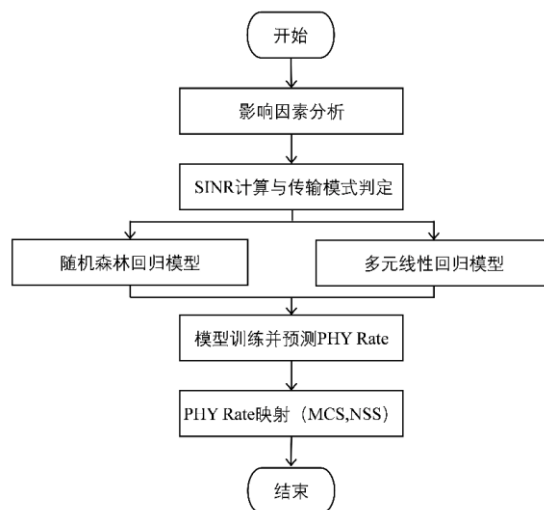


图 4.4 问题二模型建立流程图

### 4.3.2 数学模型详解

#### 1. SINR 计算

考虑到传输模式对 SINR 计算的影响，以下分为三种情况计算 SINR：

(1) 仅有同步传输时，AP 在绝大多数情况下交替使用信道发送信号，偶然同时发送信号。因此，STA 接收信号时的干扰主要来自环境底噪，对于 AP 发送给其关联 STA 的数据，STA 接收时的 SINR 可以表示为：

$$S_{STA} = \frac{P_{signal}}{P_{noise}} \quad (4-1)$$

- $S_{STA}$ ：即  $SINR_{STA}$  表示 STA 接收数据时的信干噪比。
- $P_{signal}$ ：AP 到 STA 的信号功率。
- $P_{noise}$ ：环境噪声。

RSSI 转换为信号功率值公式：

$$P_{signal} = 10^{\frac{R_{AP-STA}}{10}} \quad (4-2)$$

(2) 当 AP 处于异步模式，且 AP 总是不互相侦听信道时，STA 接收 AP 数据时的 SINR 主要来自其他 AP 的信号干扰，其环境底噪可忽略不计，则 STA 的 SINR 为关联 AP 到该 STA 的 RSSI 与邻区其他 AP 到该 STA 的 RSSI 之差，即：

$$S_{STA} = R_{Ass\_AP} - R_{Other\_AP} \quad (4-3)$$

- $R_{Ass\_AP}$ ：即  $RSSI_{Ass\_AP}$  表示关联 AP 到该 STA 的 RSSI。
- $R_{Other\_AP}$ ：即  $RSSI_{Other\_AP}$  表示邻区其他 AP 到该 STA 的 RSSI。

(3) 当 AP 的 RSSI 处于 [PD, ED] 或 [NAV, PD] 时，同步传输与异步传输混合，即混合传输。在同步传输阶段，STA 接收数据的干扰只有环境底噪，在异步传输阶段，干扰有邻区其他 AP 的信号和环境干扰。STA 接收数据时的 SINR 可以表示为：

$$S_{STA}^1 = \frac{P_{signal}}{P_{noise}} \quad (4-4)$$

$$S_{STA}^2 = R_{Ass\_AP} - R_{Other\_AP} \quad (4-5)$$

$$S_{STA}^{Total} = S_{STA}^1 + S_{STA}^2 \quad (4-6)$$

- $S_{STA}^1$ ：即  $SINR_{STA}^1$  表示同步传输阶段时 STA 接收数据的 SINR。
- $S_{STA}^2$ ：即  $SINR_{STA}^2$  表示异步传输阶段时 STA 接收数据的 SINR。
- $S_{STA}^{Total}$ ：即  $SINR_{STA}^{Total}$  表示混合传输阶段 STA 接收数据的 SINR。

若干扰 AP 的 RSSI 较低(低于设定门限)，其干扰可忽略不计；若没有噪声功率的具体值，将其设定为 -100dBm，表示无信号强度。

#### 2. AP 传输模式的判定

不同传输模式会影响模型预测(MCS, NSS)，但在实测训练数据中未明确说明具体的 AP 传输模式，因此需要通过设计特征工程说明传输模式。

由于在不同传输模式下，STA 的 SINR 会表现出不同的变化模式：

- 在同步传输时，SINR 较为稳定，受到的干扰主要是环境底噪。
- 在异步传输或混合传输时，SINR 会出现波动，受到邻区 AP 的干扰以及环境底噪。

基于此变化特点，可以设计一个表示 SINR 波动的特征，即：

$$\Delta S = \frac{S_{max} - S_{min}}{TD} \quad (4-7)$$

- $S_{max}$ : 即  $SINR_{max}$  表示 STA 接收数据的最大 SINR 值。
- $S_{min}$ : 即  $SINR_{min}$  表示 STA 接收数据的最小 SINR 值。
- $\Delta S$ : 即  $\Delta SINR$  表示 STA 接收数据的 SINR 波动值。
- $TD$ : 即  $Test\_dur$  表示测试时间。

较小的 SINR 波动，即  $\Delta SINR$  值较小，AP 对应为同步传输模式，而波动较高的情况，即  $\Delta SINR$  值较大，对应异步传输模式或混合传输模式。我们根据上述 SINR 计算公式计算实测数据的  $\Delta SINR$  值，并设置一个衡量波动幅度的阈值  $\Delta SINR=0.1$ ，大于设定阈值则判定为异步传输模式，小于或等于阈值判定为同步传输模式，下图分别展示了 2AP 和 3AP 情况下 STA 的  $\Delta SINR$  值：

1. AP 数量为 2 时，STA 的  $\Delta SINR$  值：

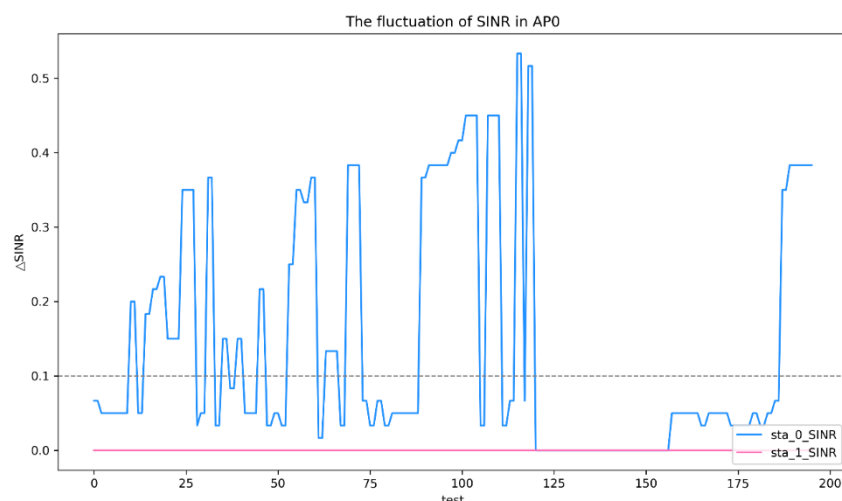


图 4.5 STA 接收 AP0 数据时 SINR 波动

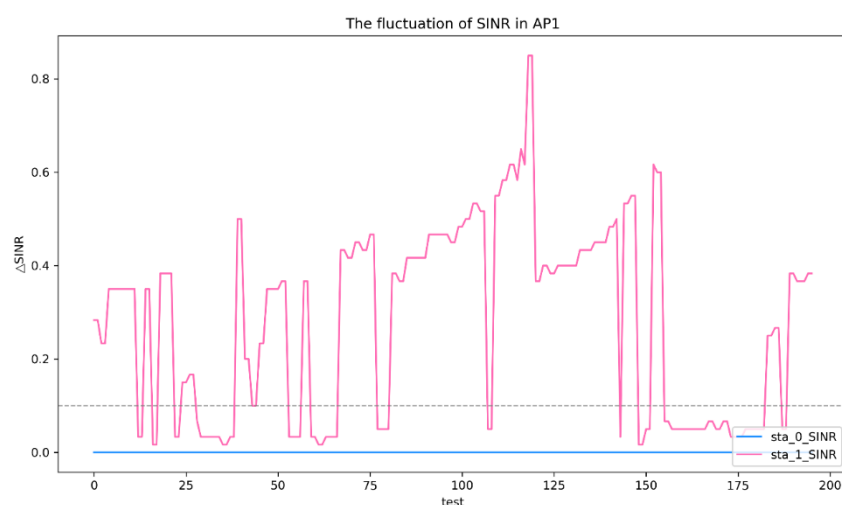


图 4.6 STA 接收 AP1 数据时 SINR 波动

1. AP 数量为 3 时，STA 的 $\Delta\text{SINR}$ 值：

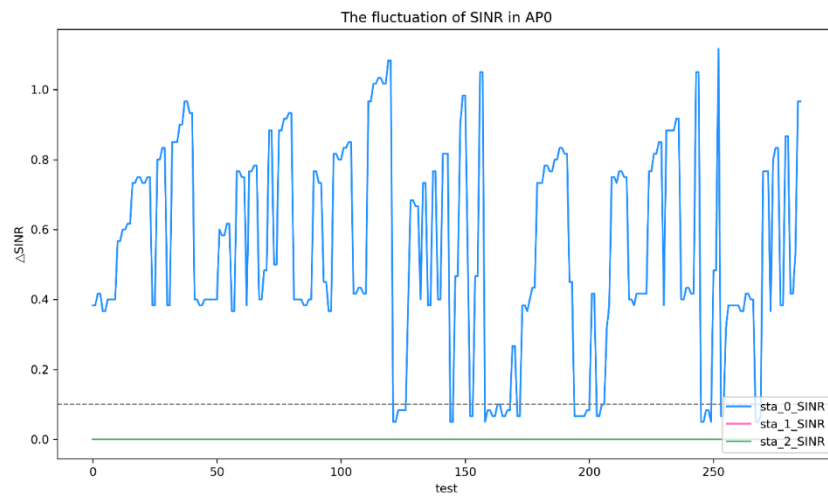


图 4.7 STA 接收 AP0 数据时 SINR 波动



图 4.8 STA 接收 AP1 数据时 SINR 波动

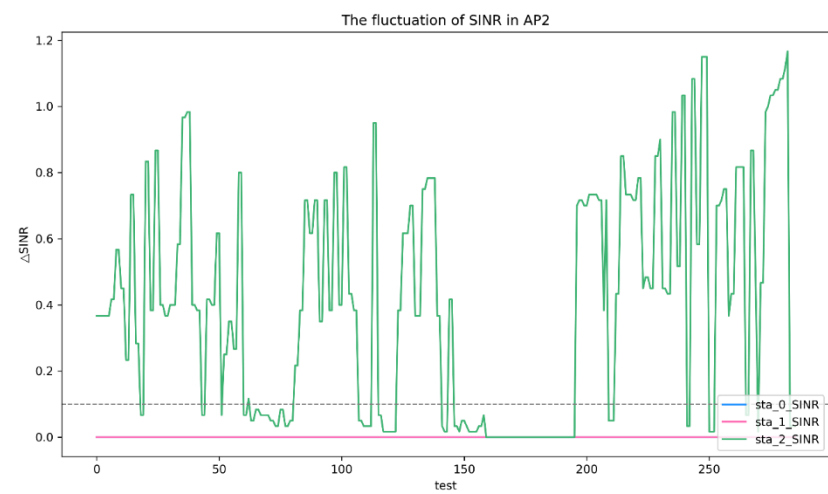


图 4.9 STA 接收 AP2 数据时 SINR 波动

### 3. (MCS, NSS) 与 SINR 的映射

根据 IEEE 802.11 协议标准,不同的(MCS, NSS)组合要求不同的最低信干噪比(SINR)才能保证稳定通信。以下是基于 IEEE 802.11ac 和 IEEE 802.11ax 标准在网络带宽为 20Hz 时, 给出的 MCS 和 NSS 组合与 PHY 速率的对应表:

表 4.1 (MCS, NSS)组合与 PHY 速率的对应关系

| MCS | NSS | PHY 速率(Mbps) |
|-----|-----|--------------|
| 0   | 1   | 8.6          |
| 1   | 1   | 17.2         |
| 2   | 1   | 25.8         |
| 3   | 1   | 34.4         |
| ... | ... | ...          |
| 11  | 2   | 286.8        |

**映射规则:** 根据计算得到的 PHY Rate 与表中 PHY 速率进行比较, 选择小于等于计算得到的实际 PHY Rate 所对应的最高(MCS, NSS)。

#### 4.3.3 特征选择

结合 4.2.2 相关性分析, 根据影响因素, 选择以下特征作为模型的输入:

##### 1. 信号功率相关:

- sta\_from\_ap\_x\_sum\_ant\_rssi: STA 接收到来自 AP 的 RSSI 之和
- sta\_from\_ap\_x\_mean\_ant\_rssi: STA 接收到来自 AP 的 RSSI 平均值

##### 2. 干扰功率相关:

- SINR: 信干噪比
- sta\_from\_ap\_x\_sum\_ant\_rssi: STA 接收到来自其他 AP 的 RSSI 之和
- ap\_from\_ap\_x\_sum\_ant\_rssi: AP 检测到来自 AP 的 RSSI 之和

##### 3. 传输方式:

- 同步或异步传输的概率

##### 4. 门限参数:

- pd: 包检测门限
- ed: 能量检测门限
- nav: NAV 门限

#### 4.3.4 梯度下降法与多元线性回归模型

*Step1.* 多元回归方程 :

对 AP 发送数据时最常用的 (MCS, NSS) 组合进行预测时会受到许多因素的影响, 可用数学语言表达为函数有多个变量, 通常将含有两个及两个以上自变量的回归模型称为多元回归<sup>[11]</sup>, 多元线性回归将所有自变量进行最优组合来进行因变量的预测, 常见的

多元线性回归方程如下：

$$f(x) = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \cdots + \omega_n x_n + \varepsilon \quad (4-8)$$

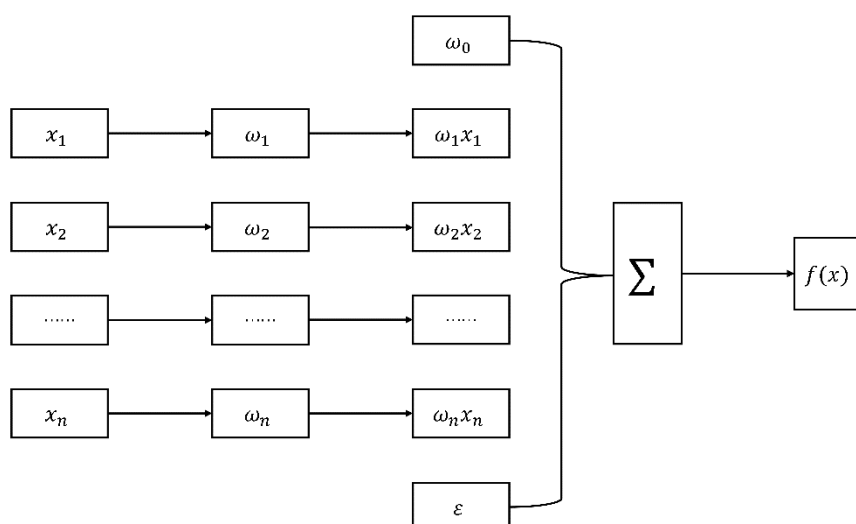


图 4.10 线性回归方程结构图

其中  $f(x)$  可以近似地表示为自变量  $x_1, x_2, \dots, x_n$  的线性函数， $\omega_0$  为常数项， $\omega_1, \omega_2, \dots, \omega_n$  为不同自变量对应的偏回归系数。

接下来通过主成分分析(PCA)对数据进行降维，其目的是规约数据集的部分属性或变量，下面对主成分分析 PCA 进行方法说明以及应用步骤描述：

(1) 方法说明：

找到一个坐标系使得数据维度降低，从而减少数据存储的空间，并且信息损失是最小的，主成分是原始数据集中所有属性的线性组合。

(2) 应用步骤：

首先是去中心化和找坐标系，其中去中心化是把坐标原点放在数据中心，找坐标系就是找到方差最大的方向作为新的坐标轴的方向。关键是怎么找到方差的最大方向，这就需要协方差矩阵，该矩阵的特征向量是坐标轴方向，特征值就是坐标轴方向上数据的方向。

**Step1:** 中心化，将原始的坐标系原点移动到整个样本点的中心，找到直线平移后的最佳位置，此步骤的本质也就是平移。

**Step2:** 样本点在直线上分布的越离散效果越好，评价离散指标的统计量就是方差，即需要找到某个角度使得样本点在这条直线上的投影的方差最大。

本题目中最终经过主成分分析所选择的特征参数为信号功率、干扰功率、传输方式、门限参数。

以二维数据降低到一维数据为例，旋转后的样本点投影到坐标轴上如果是恰好不相关的，即随着  $x^{\wedge}$  的增大  $y^{\wedge}$  可增可减，也就是  $x^{\wedge}$  和  $y^{\wedge}$  的协方差和相关系数都为 0，此时样本点在  $x^{\wedge}$  轴上的方差就是旋转过程中最大的，该  $x^{\wedge}$  坐标轴就是我们要寻找的那个最佳位置，将  $x^{\wedge}$  轴命名为“第一主成分 pc1”， $y^{\wedge}$  命名为“第二主成分 pc2”，于是将二维数据降低为一维数据，所以只需要 pc1 即可。

本题中通过主成分分析最终得到的多元线性回归方程的表达式为：

$$f(x) = \omega_0 + \omega_1 x_1 + \omega_2 x_2 \quad (4-9)$$

Step2. 确定损失函数：

一般的，损失值可以理解为一个样本经过模型处理得到的预测值和真实值的差值，差值越小即损失越小，就认为模型的性能越好，损失数可以通过损失函数来求，损失函数的种类也很丰富，但其本质都是计算真实值和预测值的差值，简单的损失值求解思想如下：

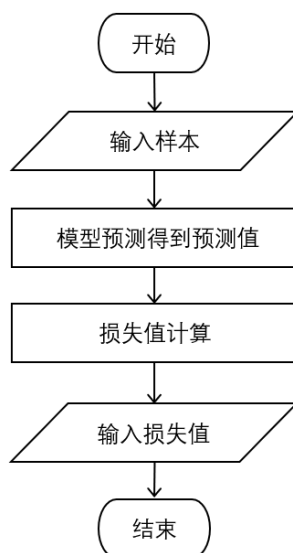


图 4.11 损失值求解流程图

本题中我们将损失函数定义为如下表达式：

$$L(x_1, x_2, \dots, x_n) = \sum_i^n (f_0(x_1, x_2, \dots, x_n) - y_i)^2 \quad (4-10)$$

Step3. 相关参数的初始化：

主要是对输入的变量 $x_1, x_2, \dots, x_n$ 进行初始化，设置算法的终止距离 $\epsilon$ ，步长为 $a$ ，假设在没有任何先验的情况下将 $x_1, x_2, \dots, x_n$ 初始化为 $0$ ，将初始步长设置为 $1$ ，在调用过程中再进行优化。

Step4. 梯度下降：

通过梯度下降法可以最小化损失函数输出的损失值，有利于训练更加精确的模型，梯度下降的主要概念包括步长、特征、拟合样本的假设函数、损失函数，梯度下降法的主要步骤为：

(1)确定当前损失函数的梯度。对于 $\omega$ 而言，将其梯度计算方式为：

$$\frac{\partial}{\partial \omega_i} J(\omega_1, \omega_2, \dots, \omega_n) \quad (4-11)$$

(2)求当前位置的上升距离。将步长 $a$ 与损失函数的梯度相乘，具体的运算方式如下：

$$a \frac{\partial}{\partial \omega_i} J(\omega_1, \omega_2, \dots, \omega_n) \quad (4-12)$$

(3)输出结果判断。对于所有的 $x_i$ 判断其梯度下降的距离是否都小于 $\epsilon$ ，如果满足此条件便可以输出结果，输出的结果为 $\omega_i (i=1, 2, 3, \dots, n)$

(4) $\omega_i$ 更新。将所有的 $\omega_i$ 进行更新，更新方式为：

$$\omega_i = \omega_i - a \frac{\partial}{\partial \omega_i} J(\omega_1, \omega_2, \dots, \omega_n) \quad (4-13)$$

更新完成之后回到第(1)步继续执行。

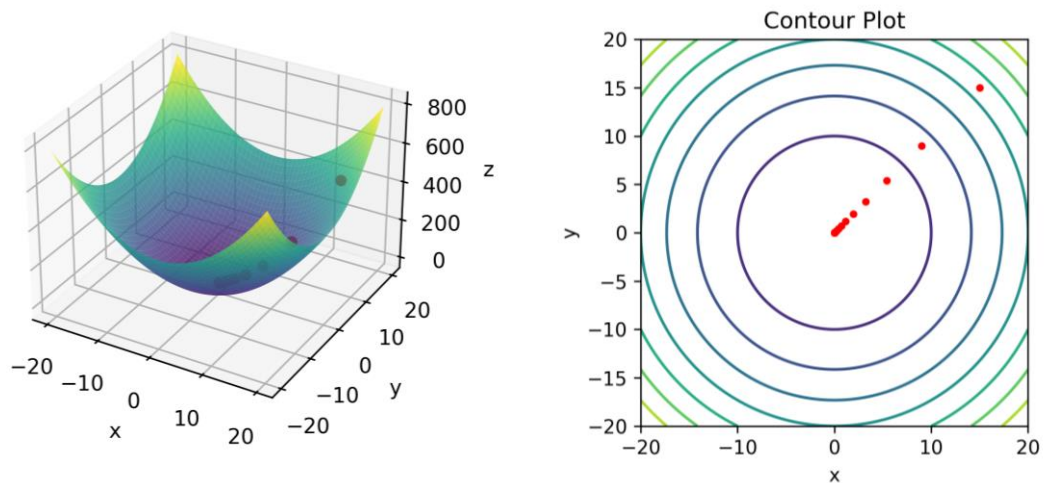


图 4.12 3D 图表（左）和等高线图（右）展示梯度下降的过程

梯度下降法是一种常见的使目标函数最小的优化方法。该方法常用于机器学习、深度学习等领域，对模型参数进行更新。在这个项目中，我们用 **Python** 来实现了一个梯度下降的算法，并且通过画出等高线图和 3D 图直观的显示了梯度下降的过程。

#### 4.4 模型求解结果

##### (1) 随机森林模型求解：

以下是通过 **python** 将 2/10 的训练集划分为验证集下的模型效果分析，图中展示了随机森林回归模型对 50 个测试用例的预测值与真实值的对比：

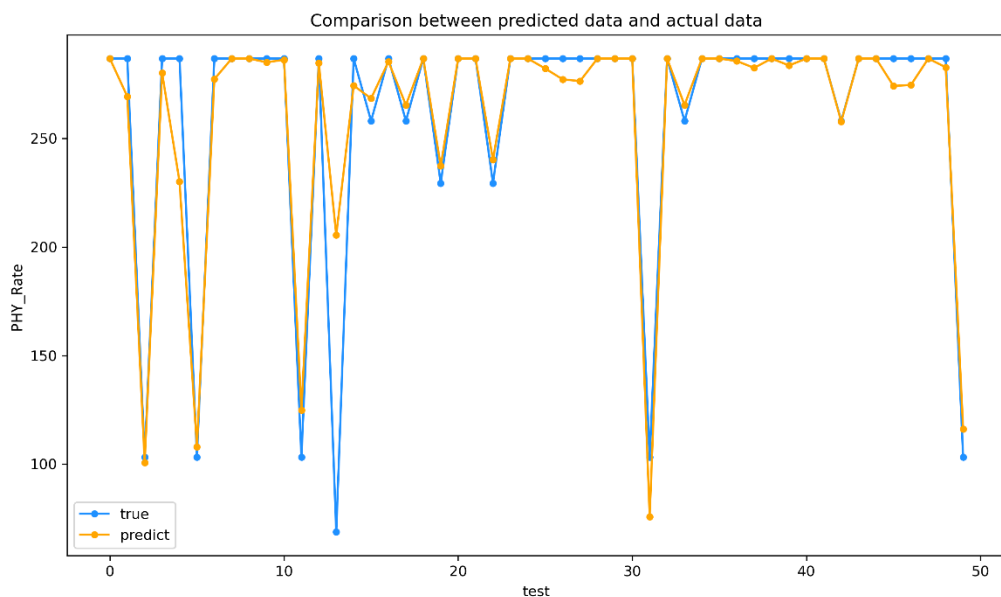


图 4.13 2AP 下随机森林预测与真实 PHY\_Rate 对比图

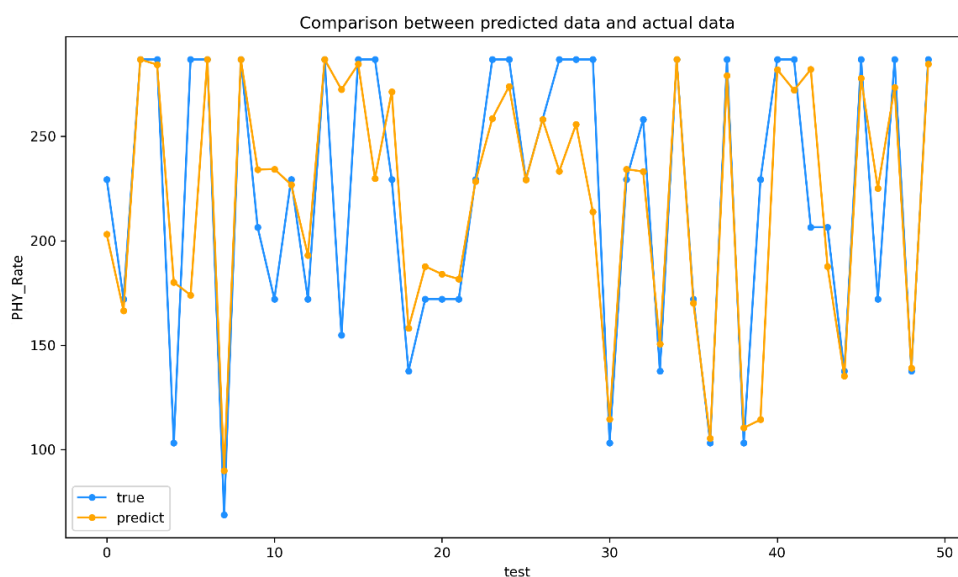


图 4.14 3AP 下随机森林预测与真实 PHY\_Rate 对比图

表 4.2 随机森林效果分析

| AP 数量 | n   | MSE     | MAE   |
|-------|-----|---------|-------|
| 2     | 79  | 698.35  | 11.11 |
| 3     | 172 | 1158.25 | 19.18 |

其中,  $n$  为验证集的样本数, 体现在图上为折线图中蓝色或橙色的验证总数量; MSE 为均方误差, 是所有预测误差的平方和的平均值; MAE 为平均绝对误差, 为所有预测误差的绝对值之和的平均值。

## (2) 多元线性回归模型求解:

我们通过 python 将标准化后的数据送入多元线性回归模型中训练, 设置梯度下降优化的学习率为 0.001, 迭代次数为 2000 次。随后通过验证集进行模型效果分析, 图中展示了多元线性回归模型对 50 个测试用例的预测值与真实值的对比:

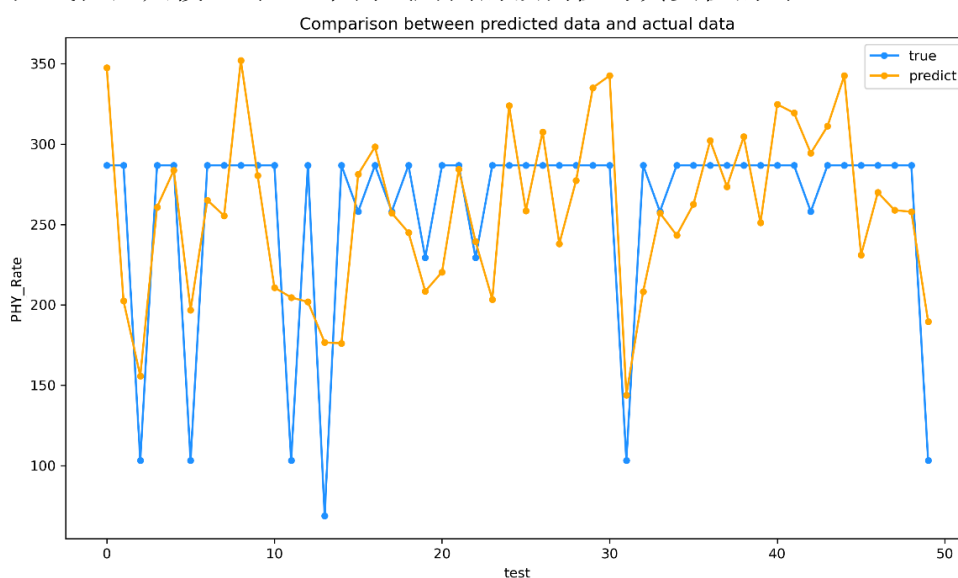


图 4.15 2AP 下多元线性回归预测与真实 PHY\_Rate 对比图

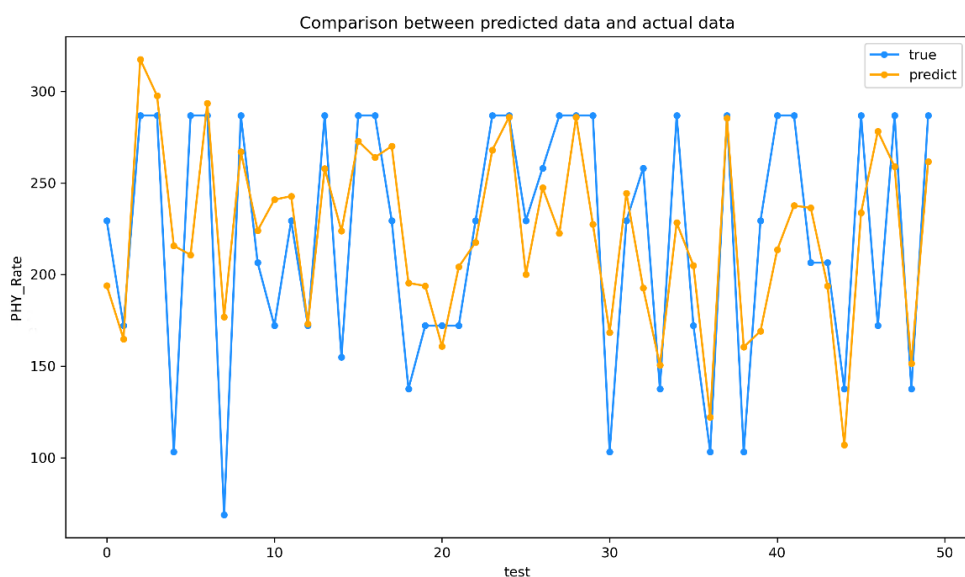


图 4.16 3AP 下多元线性回归预测与真实 PHY\_Rate 对比图

表 4.3 多元线性回归效果分析

| AP 数量 | n   | MSE     | MAE   |
|-------|-----|---------|-------|
| 2     | 79  | 3099.49 | 46.03 |
| 3     | 172 | 2469.18 | 40.19 |

其中，n 为验证集的样本数，我们按照 8:2 的比例将原训练集划分为训练集和验证集，通过验证集进行模型评测分析。

### (3) LSTM 模型求解：

我们采用与上述两种方法的相同方案划分原训练集，使用 python 将训练集放入 LSTM 模型进行训练，使用 Adam 优化器进行模型优化，设置学习率为 0.005，通过验证集对 500 趟训练模型进行择优选择及模型评判分析，以下为全部验证集下的 50 个测试数据体系出的模型效果分析：



图 4.17 2AP 下 LSTM 预测与真实 PHY\_Rate 对比图

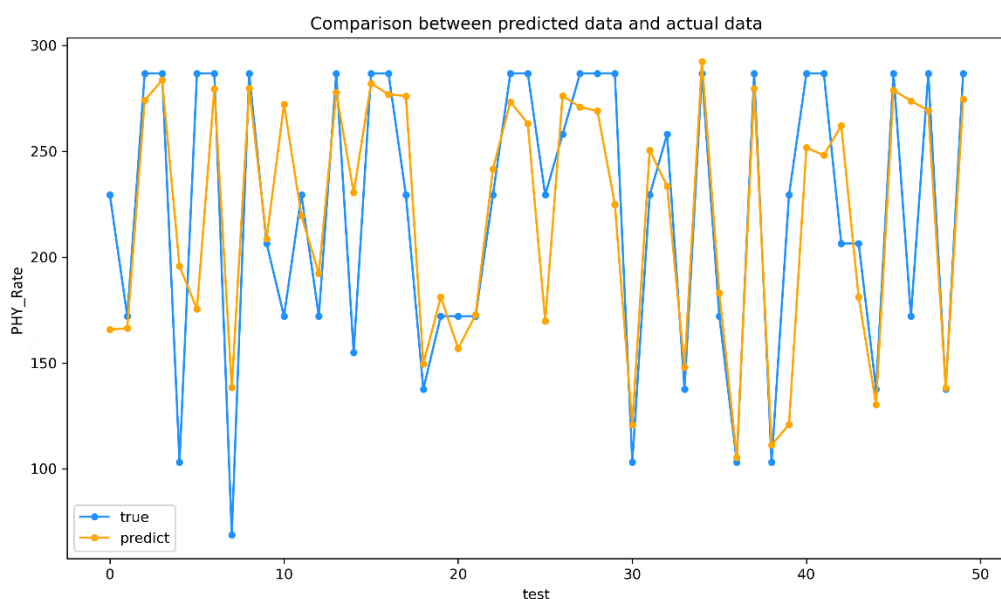


图 4.18 3AP 下 LSTM 预测与真实 PHY\_Rate 对比图

表 4.4 LSTM 效果分析

| AP 数量 | LSTM 层数   | 隐藏神经元     | MSE            |
|-------|-----------|-----------|----------------|
| 2     | 1         | 50        | 1518.62        |
|       | 5         | 50        | 865.95         |
|       | <b>10</b> | <b>50</b> | <b>544.64</b>  |
| 3     | 1         | 50        | 1902.55        |
|       | <b>5</b>  | <b>50</b> | <b>1507.85</b> |
|       | 10        | 50        | 1663.39        |

根据随机森林、多元逻辑回归、LSTM 的图示以及表格分析中的 MSE 来评判，在 AP 数量为 2 的情况下，10 层具有 50 个隐藏神经元的 LSTM 模型在三种方法中具有较好的预测效果；在 AP 数量为 3 的情况下，基于机器学习的随机森林模型具有较好的预测效果。因此，我们针对 AP 数量的不同，采用以上两种已经训练好的模型针对对应的 AP 数量测试集 test\_set\_2\_2ap 和 test\_set\_2\_3ap 中的 PHY\_Rate 进行预测，随后通过(MCS, NSS)组合表中的数据作为评判标准，对比得出每个任务中的(MCS, NSS)组合。以下为测试集中的部分预测数据，全部预测数据请查看本文附录：

表 4.5 test\_set\_1\_2ap 部分预测总时长

| test_id | loc_id | protocol | bss_id | NSS | MCS | PHY_Rate |
|---------|--------|----------|--------|-----|-----|----------|
| 1       | loc4   | tcp      | 1      | 2   | 8   | 212.7    |

|   |      |     |   |   |   |       |
|---|------|-----|---|---|---|-------|
| 1 | loc4 | tcp | 0 | 1 | 7 | 99.2  |
| 2 | loc4 | udp | 1 | 2 | 8 | 212.7 |
| 2 | loc4 | udp | 0 | 1 | 7 | 99.2  |
| 3 | loc4 | tcp | 1 | 2 | 6 | 168.9 |
| 3 | loc4 | tcp | 0 | 1 | 7 | 99.2  |
| 4 | loc4 | udp | 1 | 2 | 6 | 168.9 |
| 4 | loc4 | udp | 0 | 1 | 7 | 99.2  |

表 4.6 test\_set\_1\_3ap 部分预测总时长

| test_id | loc_id | protocol | bss_id | NSS | MCS | PHY_Rate |
|---------|--------|----------|--------|-----|-----|----------|
| 1       | loc34  | tcp      | 2      | 2   | 6   | 159.6    |
| 1       | loc34  | tcp      | 0      | 1   | 7   | 89.3     |
| 1       | loc34  | tcp      | 1      | 1   | 4   | 63.6     |
| 2       | loc34  | udp      | 2      | 2   | 6   | 159.6    |
| 2       | loc34  | udp      | 0      | 1   | 7   | 89.3     |
| 2       | loc34  | udp      | 1      | 1   | 4   | 63.6     |
| 3       | loc34  | tcp      | 2      | 2   | 6   | 165.3    |

## 五、问题三模型建立与求解

### 5.1 问题背景及目标

**背景：**WLAN 优化的核心问题是准确预测系统吞吐量，吞吐量直接影响用户体验，尤其在高密度部署场景下。实际网络环境中的信道干扰和节点间的复杂交互，使得吞吐量预测成为一个挑战。问题三基于前面对 AP 发送机会和(MCS, NSS)组合的分析，要求进一步对 WLAN 系统的吞吐量建模。其中 RSSI 作为一个关键特征反映信号强度，但由于信道的快速变化，仅凭 RSSI 可能难以精确预测吞吐量。可以引入问题二得出的(MCS, NSS)信息来增强模型的预测能力。

**目标：WLAN 系统吞吐量的建模和预测。**通过构建合适的模型，预测 WLAN 系统的吞吐量。模型应考虑同频 AP 个数的影响，利用 RSSI 和(MCS, NSS)组合来提高模型的预测精度，并基于提供的测试集，进行网络吞吐量的预测。

## 5.2 问题分析与建模思路

作为衡量网络性能指标之一的无线局域网 WLAN 系统的吞吐量,受到包括了 AP 发送机会、由(MCS, NSS)决定的 PHY 速率、数据帧丢包率等因素的影响,故而将问题一对 AP 发送机会影响因素分析、问题二对 AP 发送数据的(MCS, NSS)建模和预测以及丢包率进行综合考虑,有助于对 WLAN 系统的吞吐量进行精准的预测。

### 5.2.1 WLAN 网络数据传输分析

为了更好理解 WLAN 网络中的数据传输交互过程,我们设计了一个数据发送方与接收方以及其他站点间的示意图。该图展示了 WLAN 网络中 AP 与 STA 之间的数据传输过程,以及在同一信道下其他 AP 的干扰情况。

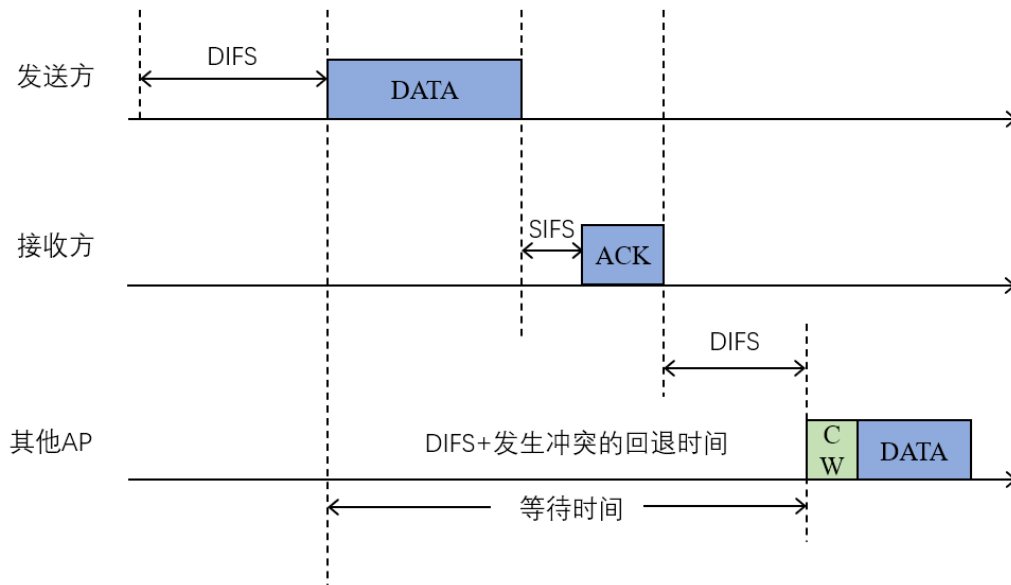


图 5.1WLAN 网络数据传输示意图

- DIFS: DCF 帧间间隔。表示一次网络传输完成后的间隔时间。
- SIFS: SIF 短帧间间隔。在传输报文分片与 ACK 间的间隔时间<sup>[6]</sup>。
- CW: Competitive Window 竞争窗口。CSMA/CA 协议要求等待的随机退避时间。

由图可知,数据发送方在 DIFS 间隔期间检查信道空闲后发送 DATA 帧,接收方在 SIFS 间隔后发送 ACK 确认帧,确保数据成功传输。WLAN 运行 CSMA/CA(载波监听多路访问/冲突避免)协议进行信道争用,当其他 AP 检测到信道占用时,它们会触发冲突避免机制,即二进制指数规避算法,并进入随机退避时间<sup>[7]</sup>。若信道仍占用,则延长回退时间。因此,信道冲突、退避时间、信道竞争、ACK 确认机制和网络丢包率都会增加信道占用时间,降低 WLAN 的整体吞吐量和传输效率。

### 5.2.2 网络吞吐量影响因素分析

WLAN 系统吞吐量定义为单位时间内节点能够成功传送数据的数量<sup>[8]</sup>,主要影响因素的定义如下:

Seq\_time: AP 的发送机会,指在总测试时间内,AP 能够成功占用信道的时间。

PHY Rate: 物理层速率,由 AP 在数据传输过程中的 MCS 和 NSS 的组合决定。

PER: 丢包率,丢包率越高代表实际传输成功的数据量越少,丢包率越低代表实际

传输成功的数据量越多，丢包率的大小决定了实际传输中数据量成功传输的多少。

基于以上定义，将 WLAN 系统中每个 AP 的吞吐量定义为如下公式，其中 Throughput 表示吞吐量，PHY Rate 可查表获得，由 (MCS, NSS) 决定；PER 表示丢包率，可由训练数据直接统计得到；seq\_time 表示 AP 发送时间，由问题一模型预测得到；Test\_dur 表示测试的总时间：

$$TP_{AP} = (PR - p \times PR) \times \frac{st}{TD} \quad (5-1)$$

- $TP_{AP}$ : 即  $Throughput_{AP}$  表示 AP 的网络吞吐量。
- $PR$ : 即  $PHY Rate$  表示发送数据的 PHY 层速率。
- $p$ : 即  $PER$  表示丢包率。
- $st$ : 即  $seq\_time$  表示 AP 发送时间。
- $TD$ : 即  $Test\_dur$  表示测试总时间。

WLAN 系统的网络吞吐量计算公式定义如下：

$$TP_{system} = \sum TP_{AP} \quad (5-2)$$

- $TP_{system}$ : 即  $Throughput_{system}$  表示 AP 的网络吞吐量。

### 5.2.3 相关性分析

#### (1) 皮尔森相关系数

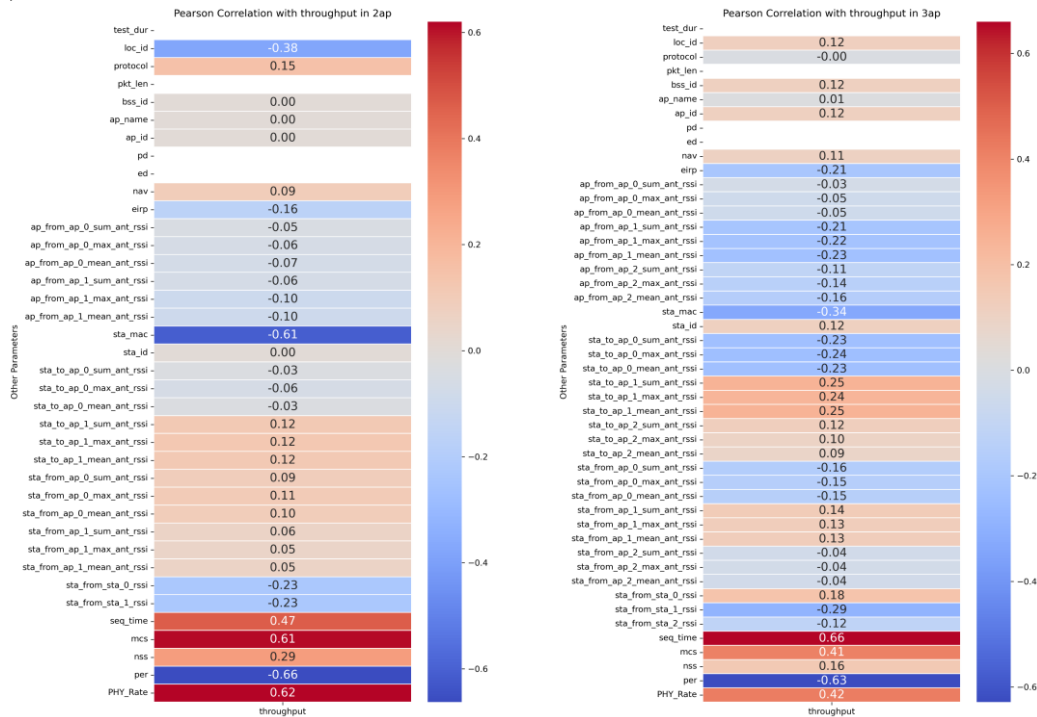


图 5.2 吞吐量的皮尔森相关系数

## (2)斯皮尔曼秩相关系数

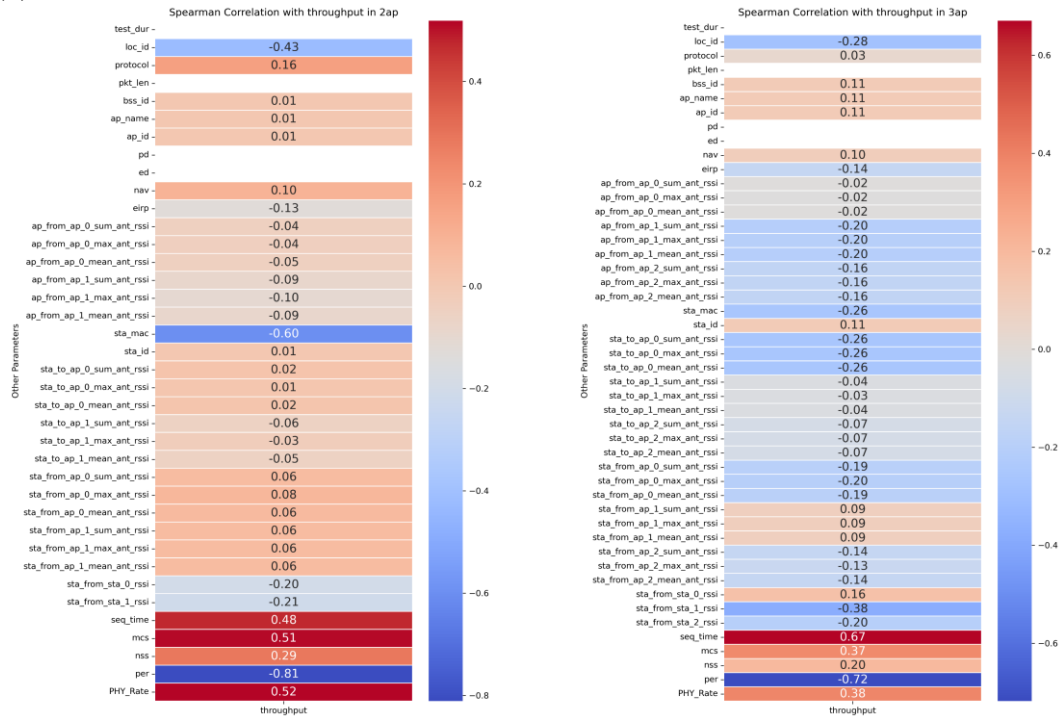


图 5.3 吞吐量的斯皮尔曼相关系数

## (3)肯德尔相关性系数

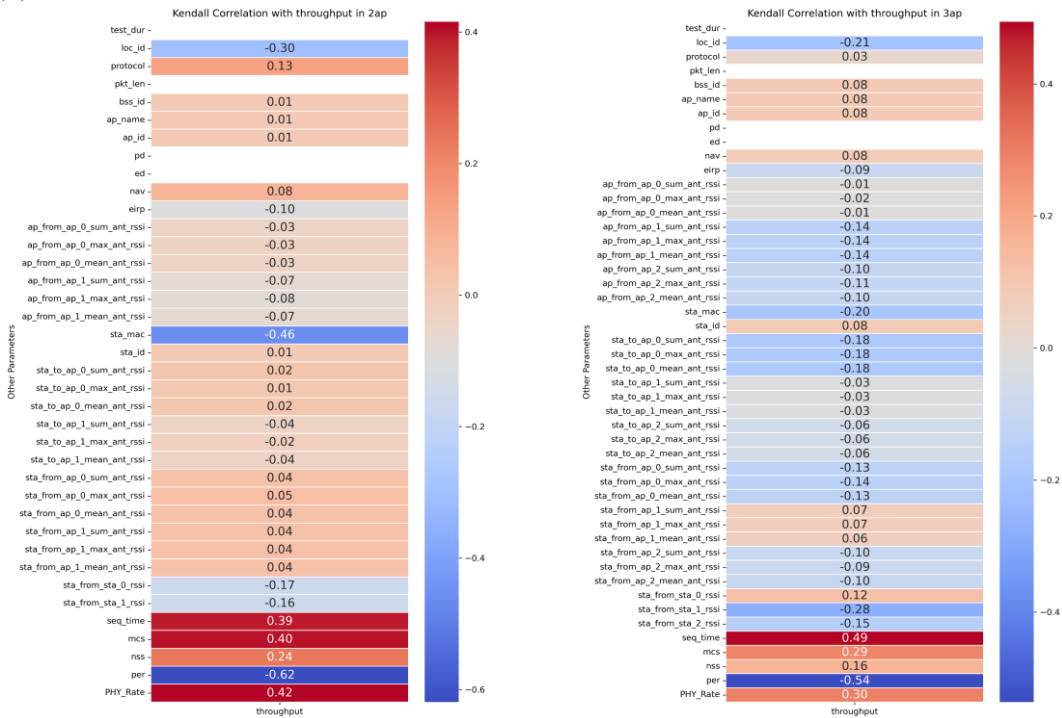


图 5.4 吞吐量的肯德尔相关系数

根据关于吞吐量的三大相关系数图表示，吞吐量 `throughput` 主要与问题一和问题二求解的发送机会(即总时长 `seq_time`)、(MCS, NSS)组合(即 `PHY Rate`)呈强正相关，当发送机会增多，`PHY Rate` 增强时，简单而言为 AP 与 STA 之间发送的信息增多增强时，网络吞吐量会随之增加。其次，吞吐量与丢包率 `per` 呈强负相关，当丢包率减少时，AP

和 STA 接收到的信号数量增多，网络吞吐量随之增加。

## 5.3 模型建立

### 5.3.1 建模目标

建模主要步骤包括特征输入和吞吐量计算，特征包括 PHY Rate、PER、seq\_time、Test\_dur。

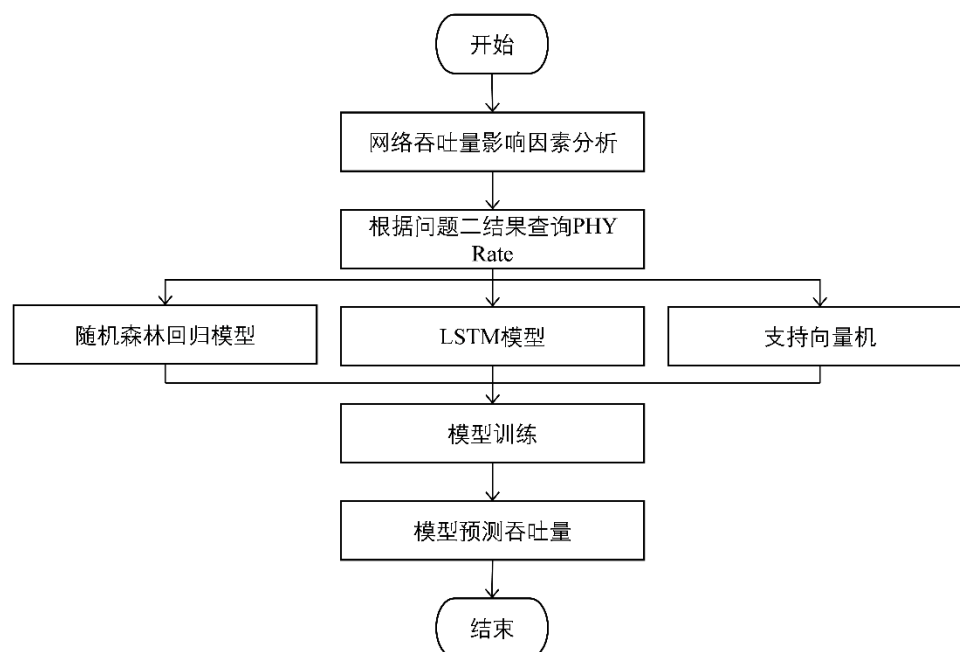


图 5.5 问题三模型建立流程图

### 5.3.2 数据查询

根据 MCS 和 NSS 组合表(即表 5.1)进行查询，我们获取到 WLAN 中 PHY 速率，随后我们根据 MCS 和 NSS 组合对，在数据集上添加名为 PHY\_Rate 的一列，用于存放每个任务的 PHY 速率，为预测网络吞吐量 throughput 提供数据依据。

表 5.1 (MCS, NSS)组合与 PHY 速率的对应关系

| NSS | MCS | PHY 速率 (Mbps) |
|-----|-----|---------------|
| 1   | 0   | 8.6           |
|     | 1   | 17.2          |
|     | ... | ...           |
|     | 10  | 129.0         |
|     | 11  | 143.4         |
| 2   | 0   | 17.2          |

|     |       |
|-----|-------|
| 1   | 34.4  |
| ... | ...   |
| 10  | 258.1 |
| 11  | 286.8 |

---

### 5.3.3 吞吐量计算

针对测试集 `test_set_1_2ap` 和 `test_set_1_3ap` 中的数据，我们根据 5.2.1 中的公式 5-1，将每个任务的(MCS, NSS)组合查表取出的 PHY Rate 和问题一中预测出的总时长 `seq_time` 和一次测试的时间进行计算，得出两个测试集中的近似实际吞吐量，并存入数据集中，为后续计算吞吐量预测误差做准备，以下为两个不同 AP 数量的测试集中前 20 个任务根据公式计算出的近似实际吞吐量统计图：

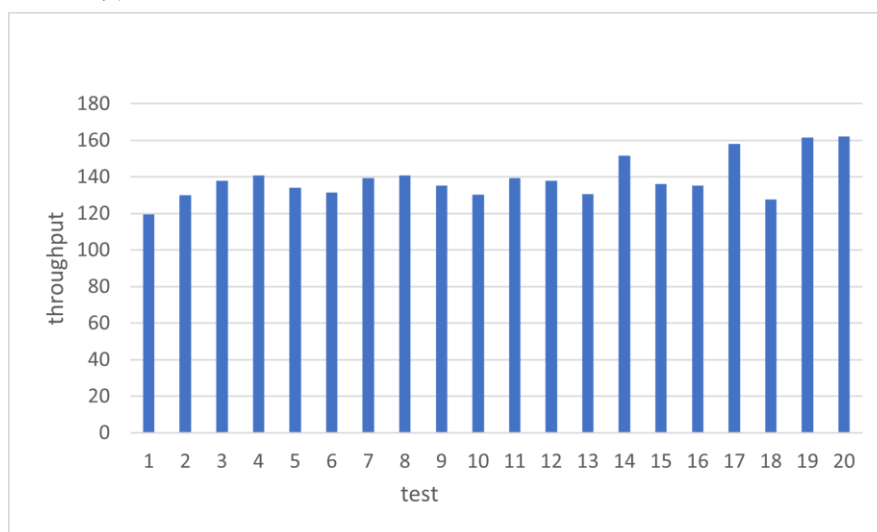


图 5.6 公式计算 2AP 下网络吞吐量

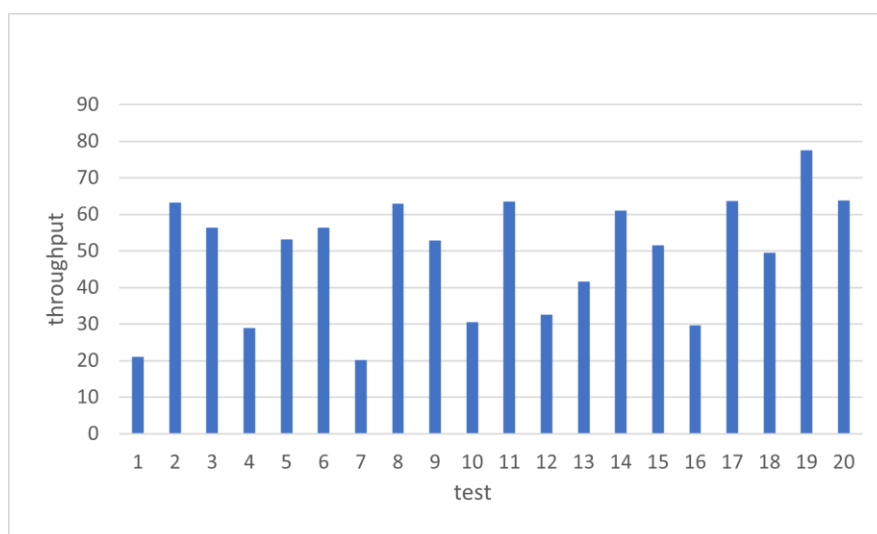


图 5.7 公式计算 3AP 下网络吞吐量

### 5.3.4 支持向量机预测

支持向量机(SVM)是一种强大的监督学习算法，广泛应用于分类和回归问题，特别适合样本较少的情况。SVM 通过寻找一个划分超平面来最大化两类样本的间隔，从而实现分类<sup>[10]</sup>。SVM 有多种变体，能够处理线性可分和非线性问题，因此可以使用 SVM 预测 WLAN 网络吞吐量。

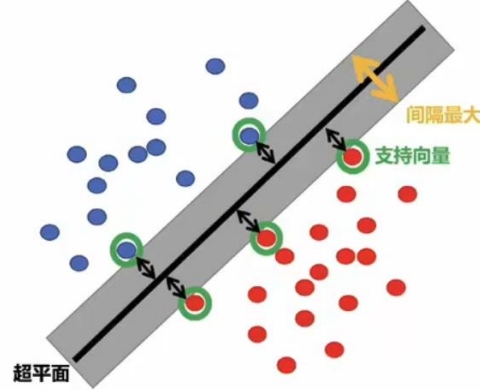


图 5.8 支持向量机示意图

在 WLAN 网络吞吐量的预测中，我们需要处理不同的输入特征，如 AP 发送时间(Seq\_time)、PHY 速率(PHY Rate)、丢包率(PER)，这些特征与网络吞吐量密切相关。SVM 能够通过这些输入特征，预测 WLAN 网络的吞吐量。支持向量机预测流程如下：

**Step1.区分问题线性可分性。**当 AP 的发送时间和 PHY 速率等特征呈现出较好的线性关系时，可以使用线性支持向量机(SVM)来构建模型。当特征与吞吐量的关系较为复杂，呈现非线性，则可以通过引入核函数(如高斯径向基函数 RBF)将问题从低维映射到高维空间，进而使用非线性支持向量机进行建模。核函数的引入能够有效捕捉复杂的特征交互，提升模型的预测性能。

**Step2.间隔最大化与软间隔。**SVM 通过寻找支持向量，即那些距离划分超平面最近的样本点来确定分类边界。在 WLAN 吞吐量预测中，支持向量机能够找到与吞吐量变化趋势最相关的数据点，帮助模型识别关键的影响因素。如果训练数据近似线性可分，但存在少量误分类情况，SVM 会引入软间隔，将误分类控制在一个可接受的范围内。

**Step3.SVM 的优化问题。**SVM 的目标是通过凸优化问题最大化类间隔。对于 WLAN 吞吐量预测问题，模型会通过训练找到最佳超平面来预测吞吐量的不同范围。优化目标可以表示为：

$$\min \frac{1}{2} \|w\|^2 \quad \text{subject to} \quad y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (5-3)$$

其中， $w$ 为权重向量， $x_i$ 为输入特征(Seq\_time、PHY Rate、PER)， $y_i$ 为 WLAN 网络吞吐量， $\xi_i$ 为松弛变量，允许部分数据误分类。

**Step4. SVM 模型预测 WLAN 吞吐量。**完成 SVM 的建模后，可以利用训练数据中 AP 发送时间与 PHY 速率等特征来构建预测模型。通过引入合适的核函数，SVM 能够捕捉非线性关系，进而预测不同网络条件下的吞吐量。

对于非线性问题，核函数将特征空间映射到更高维度，分类决策函数表示为：

$$f(x) = \sum_{i=1}^N a_i y_i K(x_i, x) + b \quad (5-4)$$

其中， $K(x_i, x)$ 为核函数， $a_i$ 为拉格朗日乘子， $x_i$ 和 $x$ 分别为训练样本和测试样本的

特征向量(PHY Rate、Seq\_time、PER)， $y_i$ 为样本对应的网络吞吐量， $b$ 为偏置项。

5.4 模型求解结果

我们采用三种方法进行求解，以下实验模型数据集划分方式均以问题一和问题二方式相同。  
(1) 随机森林模型求解：  
针对随机森林模型求解，我们设置模型针对问题划分 100 棵决策树进行数据的学习和预测，图中展示了随机森林回归模型对验证集中 50 个测试用例的预测吞吐量与真实吞吐量的对比：

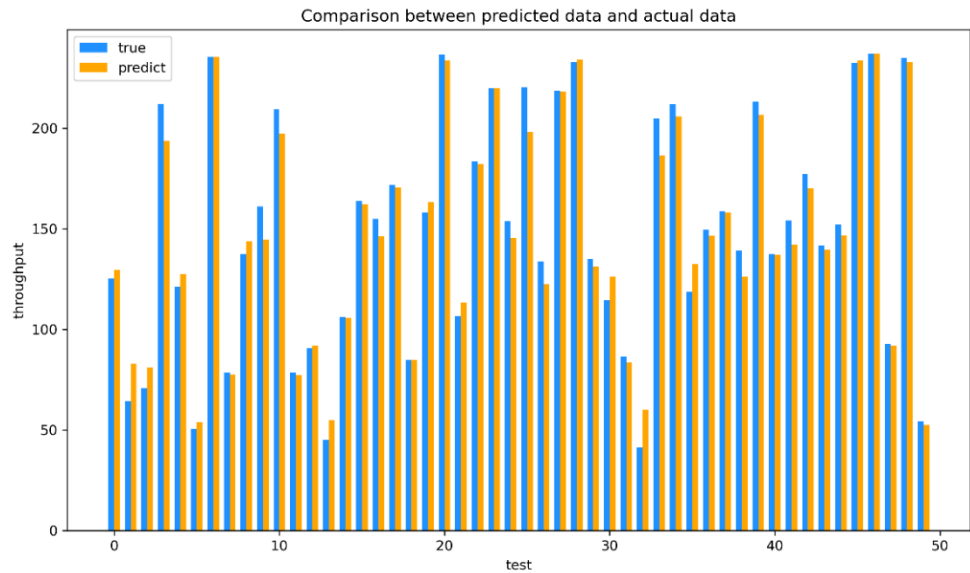


图 5.9 2AP 下随机森林预测与真实吞吐量对比图

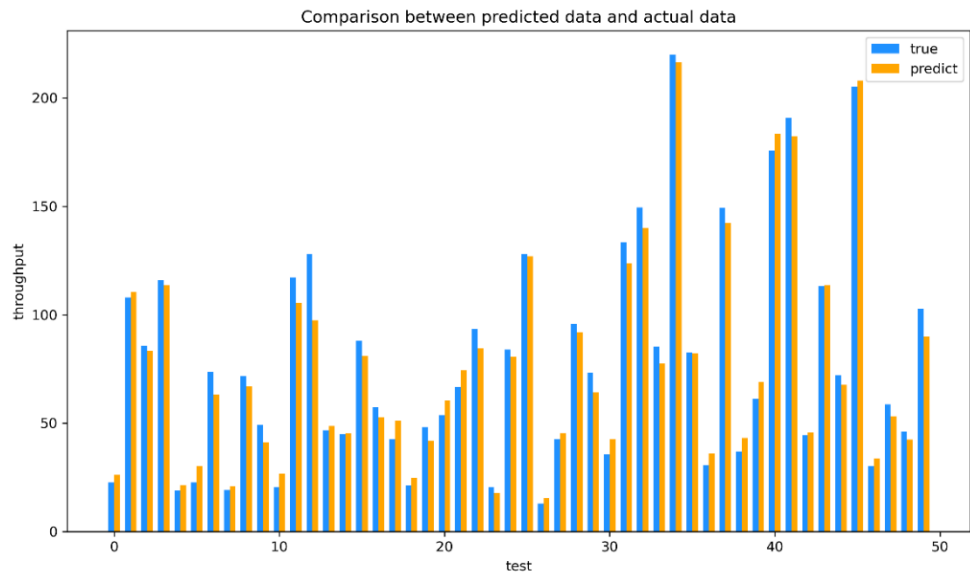


图 5.10 3AP 下随机森林预测与真实吞吐量对比图

表 5.2 随机森林效果分析

| AP 数量 | n  | MSE   | MAE  |
|-------|----|-------|------|
| 2     | 79 | 71.05 | 6.19 |

其中,  $n$  为验证集的样本数, 体现在图上为柱状图中蓝色或橙色的总数量; MSE 为均方误差, 是所有预测误差的平方和的平均值; MAE 为平均绝对误差, 为所有预测误差的绝对值之和的平均值。

## (2) 支持向量机模型求解:

我们通过 python 将根据组合表映射标准化后的整合数据送入三种不同核函数(线性核函数、高斯径向基核函数和多项式核函数)的支持向量机模型(SVM)中训练。随后通过验证集进行模型效果分析。

针对线性核函数 Linear, 我们设置模型复杂度为 10, 模型回归的精度为 0.1。下图中展示了线性核函数的支持向量机模型对 50 个测试用例的预测值与真实值的对比:

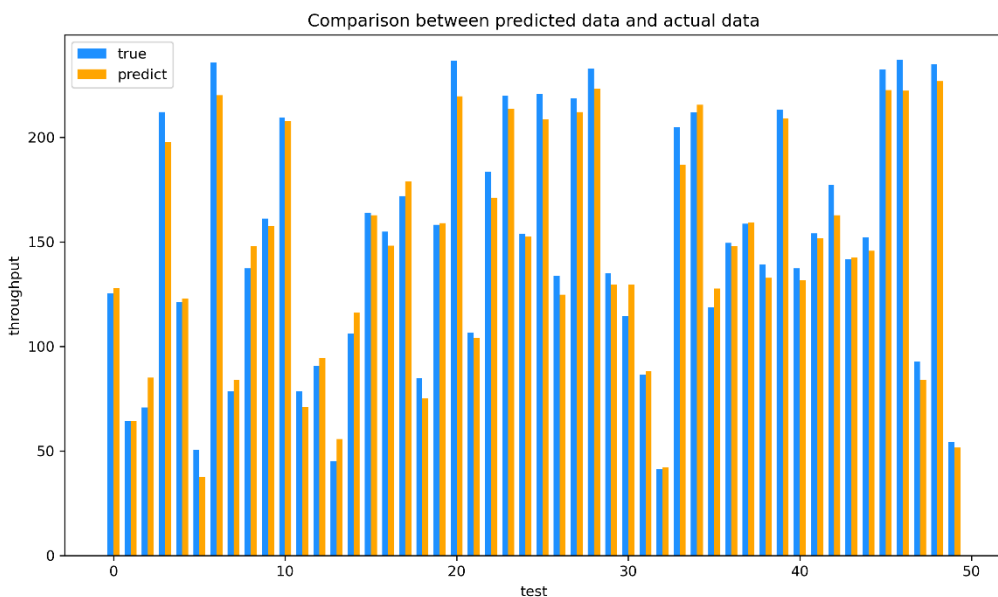


图 5.11 2AP 下线性核函数 SVM 预测与真实 PHY\_Rate 对比图

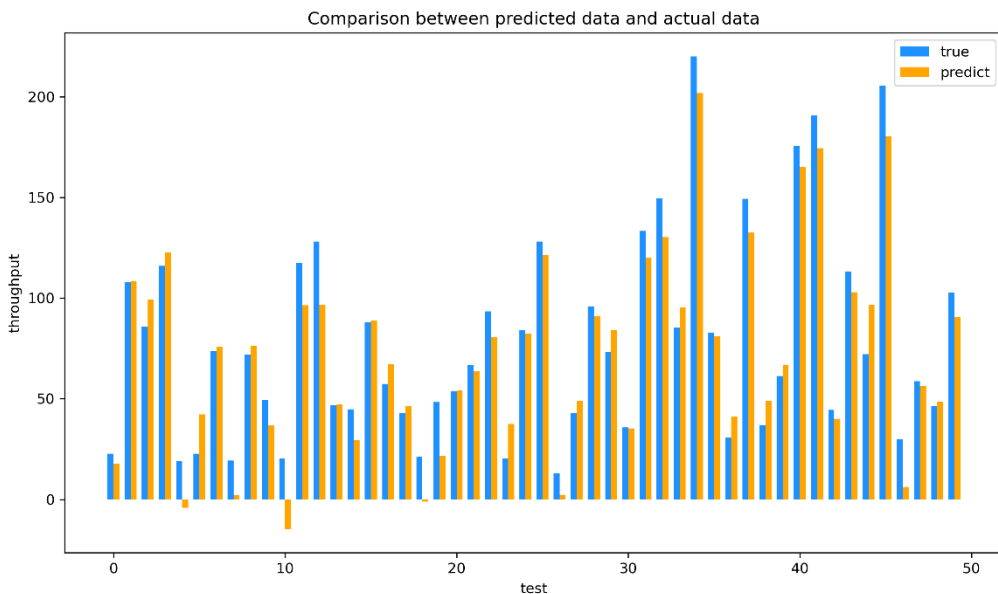


图 5.12 3AP 下线性核函数 SVM 预测与真实 PHY\_Rate 对比图

表 5.3 线性核函数 SVM 效果分析

| AP 数量 | n   | MSE    | MAE   |
|-------|-----|--------|-------|
| 2     | 79  | 124.64 | 8.48  |
| 3     | 172 | 232.74 | 12.24 |

针对高斯径向基核函数 Rbf，我们设置模型复杂度为 10，模型回归的精度为 0.1，高斯分布宽度  $\gamma$  为 0.1。下图中展示了径向基核函数的支持向量机模型对 50 个测试用例的预测值与真实值的对比：

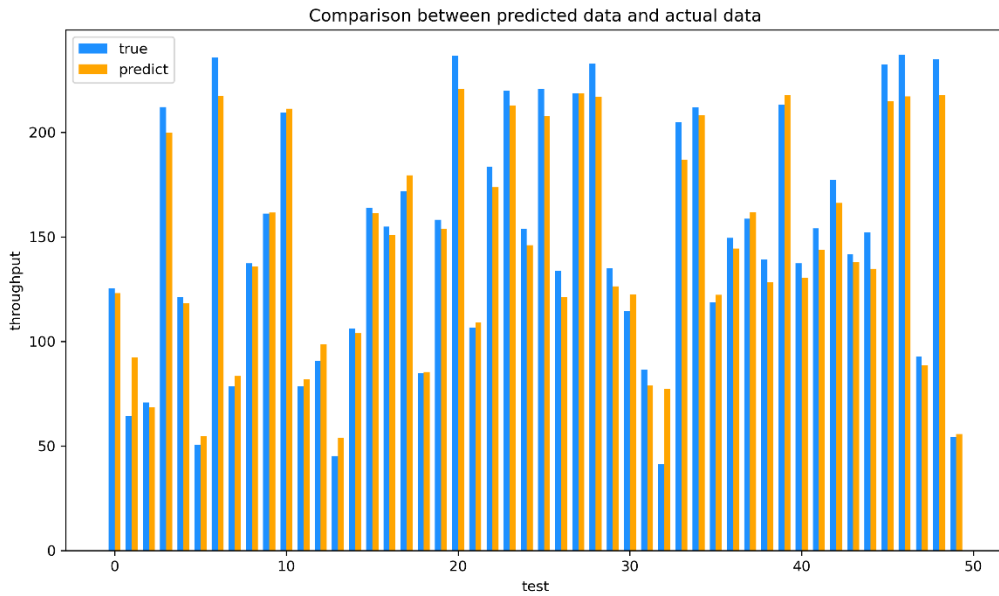


图 5.13 2AP 下径向基核函数 SVM 预测与真实 PHY\_Rate 对比图

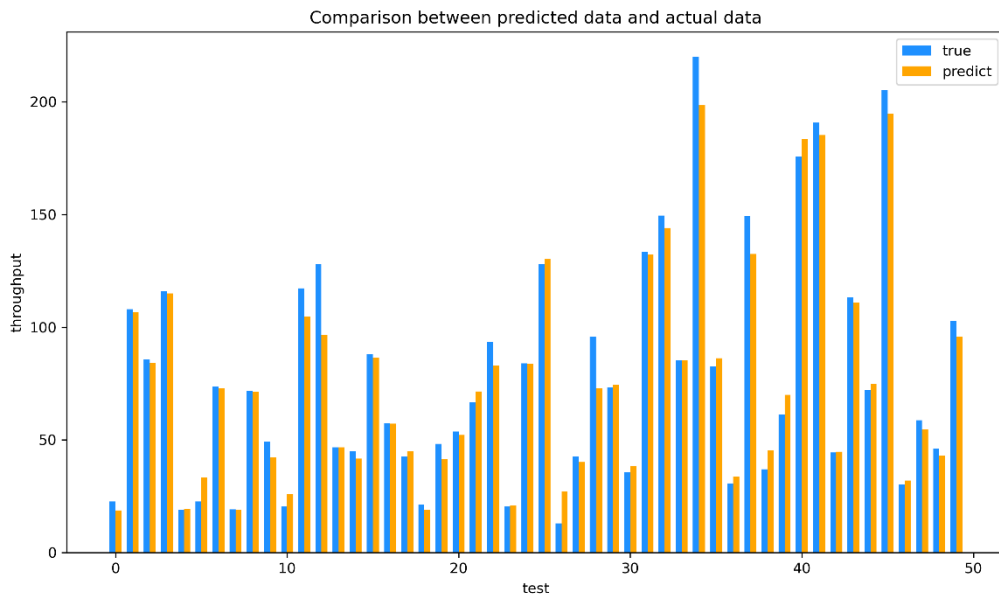


图 5.14 3AP 下径向基核函数 SVM 预测与真实 PHY\_Rate 对比图

表 5.4 径向基核函数 SVM 效果分析

| AP 数量 | n   | MSE    | MAE  |
|-------|-----|--------|------|
| 2     | 79  | 112.97 | 8.21 |
| 3     | 172 | 65.13  | 5.62 |

针对多项式核函数 Poly，我们设置模型复杂度为 10，模型回归的精度为 0.1。下图中展示了多项式核函数的支持向量机模型对 50 个测试用例的预测值与真实值的对比：

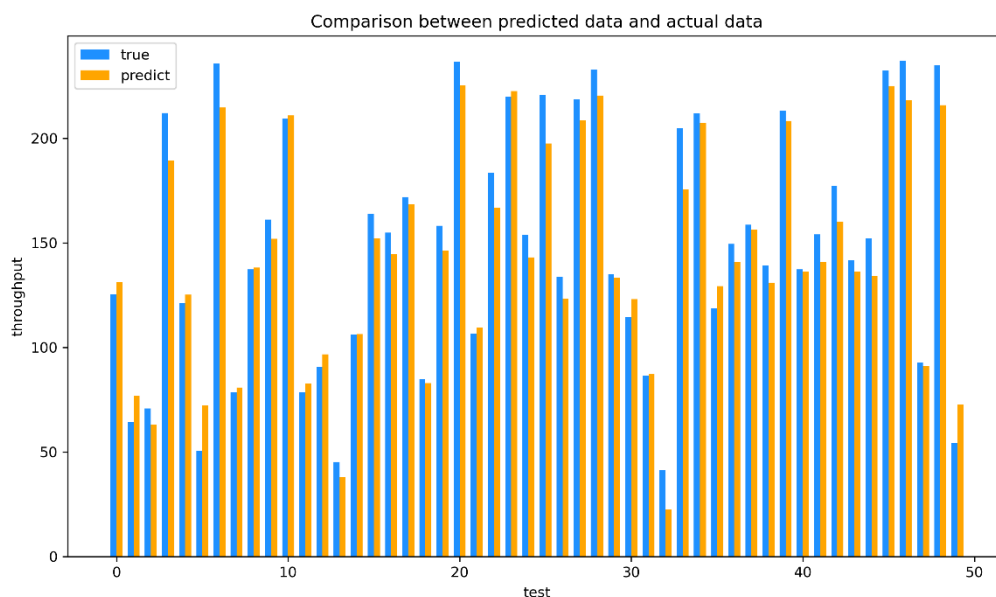


图 5.15 2AP 下多项式核函数 SVM 预测与真实 PHY\_Rate 对比图

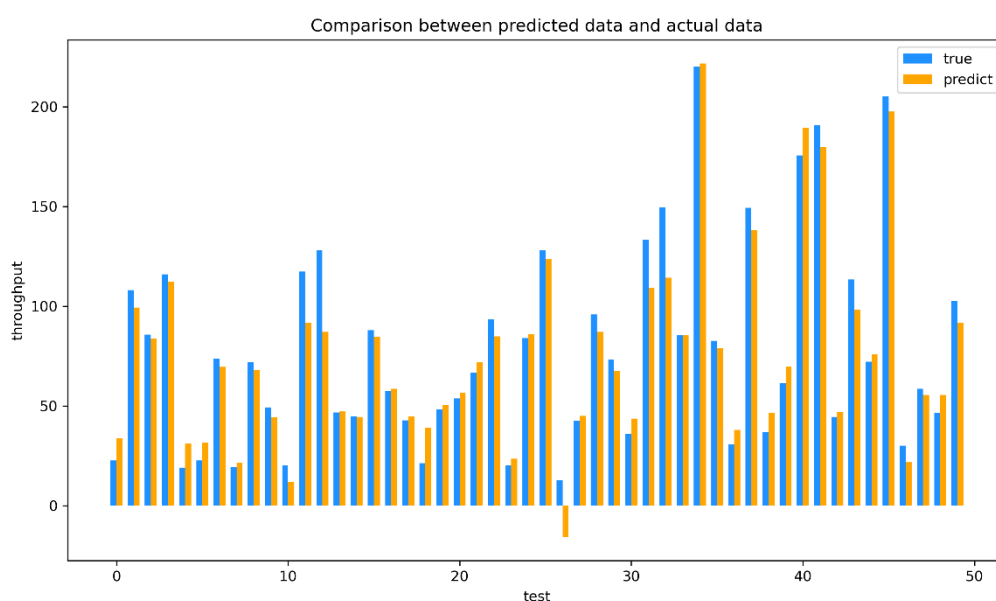


图 5.16 3AP 下多项式核函数 SVM 预测与真实 PHY\_Rate 对比图

表 5.5 多项式核函数 SVM 效果分析

| AP 数量 | n   | MSE    | MAE  |
|-------|-----|--------|------|
| 2     | 79  | 137.23 | 9.60 |
| 3     | 172 | 12.48  | 7.30 |

根据对三种核函数的支持向量机模型的验证集效果分析，我们发现采用高斯径向基核函数下的支持向量机模型在三种模型中取得了最优的效果，因此高斯径向基核函数更适合这种多元线性预测。

### (3) LSTM 模型求解：

我们使用 python 编写 LSTM 模型，在不同层数的 LSTM 模型上进行训练，使用 Adam 优化器进行模型优化，设置学习率为 0.005。通过验证集对 500 趟训练模型进行择优选择及模型评判分析，以下为 50 个验证集数据的模型效果分析：

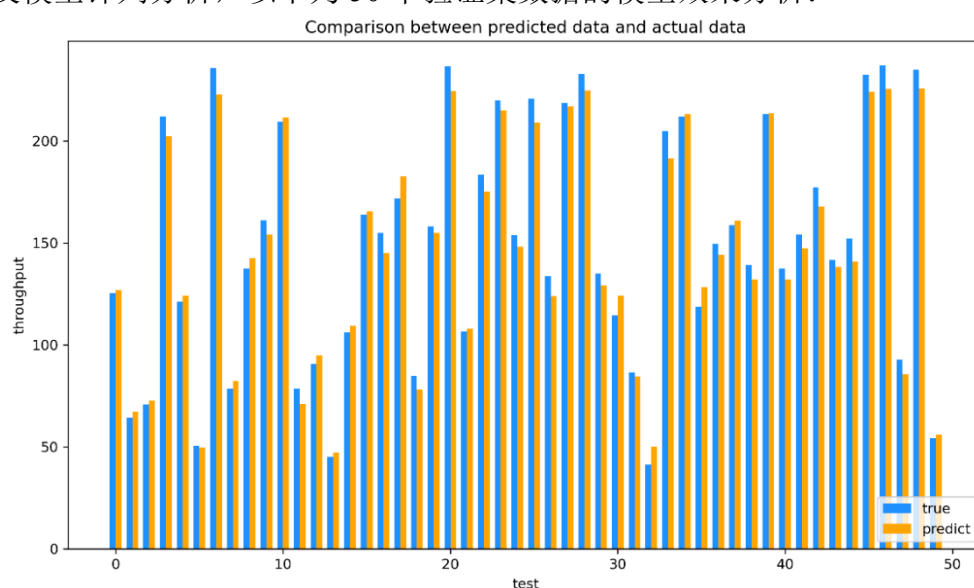


图 5.17 2AP 下 LSTM 预测与真实 PHY\_Rate 对比图

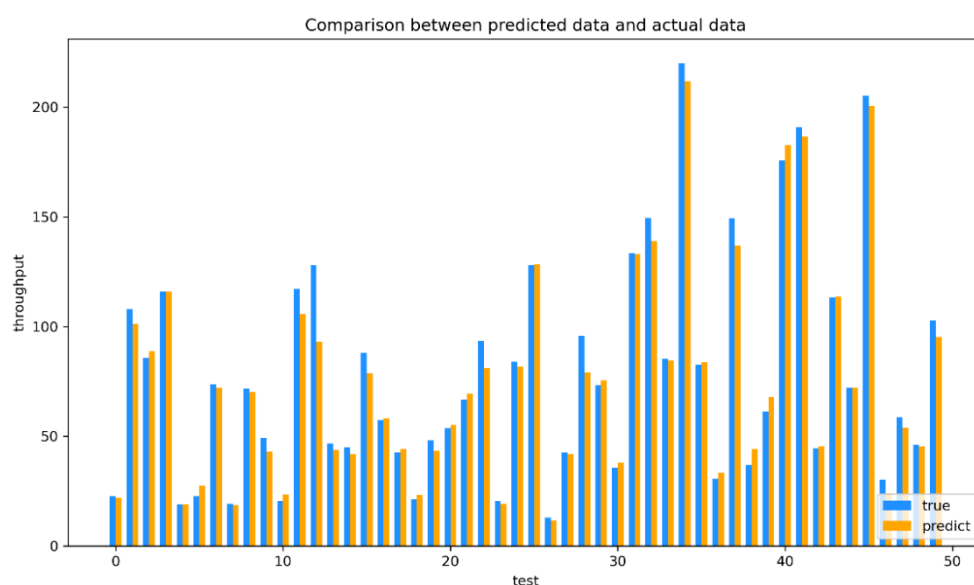


图 5.18 3AP 下 LSTM 预测与真实 PHY\_Rate 对比图

表 5.6 LSTM 效果分析

| AP 数量 | LSTM 层数 | 隐藏神经元 | MSE   |
|-------|---------|-------|-------|
| 2     | 1       | 50    | 53.61 |
|       | 5       | 50    | 68.52 |
|       | 10      | 50    | 85.93 |
| 3     | 1       | 50    | 50.12 |
|       | 5       | 50    | 58.20 |
|       | 10      | 50    | 65.15 |

根据随机森林、支持向量机、LSTM 的图示以及表格分析中 MSE 数值，我们发现在 AP 数量为 2 和 AP 数量为 3 的情况下，基于深度学习的 LSTM 模型在多个方法中具有最好的预测效果。因此，我们已经训练好的具有单层 LSTM 模型对测试集 test\_set\_1\_2ap 和 test\_set\_1\_3ap 中的网络吞吐量 throughput 进行预测。以下为测试集中的部分预测吞吐量数据，全部预测数据请查看本文附录：

表 5.7 test\_set\_1\_2ap 部分预测总时长

| test_id | loc_id | protocol | bss_id | NSS | MCS | PHY_Rate |
|---------|--------|----------|--------|-----|-----|----------|
| 1       | loc4   | tcp      | 1      | 2   | 8   | 212.7    |
| 1       | loc4   | tcp      | 0      | 1   | 7   | 99.2     |
| 2       | loc4   | udp      | 1      | 2   | 8   | 212.7    |
| 2       | loc4   | udp      | 0      | 1   | 7   | 99.2     |
| 3       | loc4   | tcp      | 1      | 2   | 6   | 168.9    |
| 3       | loc4   | tcp      | 0      | 1   | 7   | 99.2     |
| 4       | loc4   | udp      | 1      | 2   | 6   | 168.9    |
| 4       | loc4   | udp      | 0      | 1   | 7   | 99.2     |

表 5.8 test\_set\_1\_3ap 部分预测总时长

| test_id | loc_id | protocol | bss_id | NSS | MCS | PHY_Rate |
|---------|--------|----------|--------|-----|-----|----------|
| 1       | loc34  | tcp      | 2      | 2   | 6   | 159.6    |
| 1       | loc34  | tcp      | 0      | 1   | 7   | 89.3     |

|   |       |     |   |   |   |       |
|---|-------|-----|---|---|---|-------|
| 1 | loc34 | tcp | 1 | 1 | 4 | 63.6  |
| 2 | loc34 | udp | 2 | 2 | 6 | 159.6 |
| 2 | loc34 | udp | 0 | 1 | 7 | 89.3  |
| 2 | loc34 | udp | 1 | 1 | 4 | 63.6  |
| 3 | loc34 | tcp | 2 | 2 | 6 | 165.3 |

其次根据 5.3.3 中公式计算出的两个测试集中的实际吞吐量，我们对预测值和计算值按照以下公式进行计算得出吞吐量预测误差  $er$ ：

$$er = \frac{|th_p - th|}{th} * 100 \quad (5-5)$$

其中  $er$  表示预测误差  $error$ ， $th$  表示实际吞吐量  $throughput$ ， $th_p$  表示预测吞吐量  $predict\ throughput$ ，并汇总绘制出在两种 AP 数量下的吞吐量预测误差图：

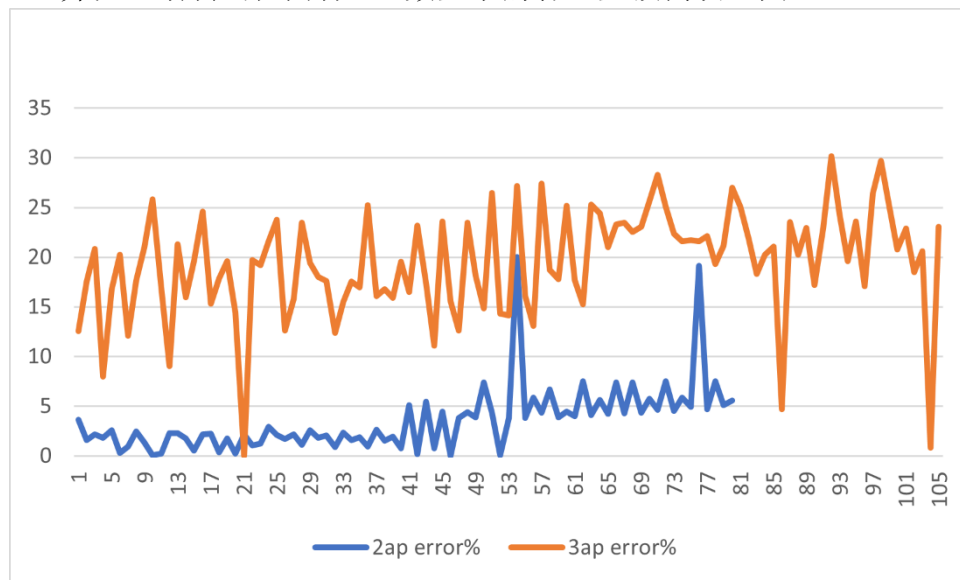


图 5.19 不同 AP 下吞吐量预测误差图

## 六、模型的总结与评价

本文针对 WLAN 组网中的 AP 和 STA 发送信号的相关数据，对 AP 发送信号的各种数据进行预测，建立了关于 AP 发送时间、(MCS, NSS)组合和网络吞吐量预测模型，建立的模型具有一定的优越性和局限性。

### 6.1 模型优缺点分析

#### 6.1.1 模型的优点

本模型具有以下优点：

(1)本文利用多个专业软件进行数据处理与问题求解，例如 $matlab$ 仿真软件， $Spss$ 数

据分析软件, *Python*等, 经过的大量实验数据分析及拟合, 得到的结果较为可靠。

(2)数据是建立模型的基础, 对所给的附件数据均进行了细致的分析, 并进行合理的预处理, 包括填充缺失值, 剔除异常值, 降低异常数据对模型的影响, 使模型稳定性更好, 可信度高。

(3)本文所用的数据均来自现实生活, 在解决问题的过程中尽量结合现实情况, 在建立模型的过程中, 考虑了多种因素对 AP 与 STA 信号传送的影响。对 WLAN 组网的数据预测有着很大的参考价值, 具有很好的实用性和可推广性。

(4) 本文通过皮尔森、斯皮尔曼和肯德尔三大常用相关系数对数据进行分析, 得出输入特征与预测值的影响关系, 确定输入数据信息特征, 确保了特征选择的科学性。。

(5)对于问题三, 我们采用随机森林模型、支持向量机模型和 *LSTM* 时间序列模型三种不同的方法给予解决, 能够很好的预测 WLAN 组网中的网络吞吐量。

### 6.1.2 模型的缺点

(1)由于竞赛时间仓促, 数据处理不够完美, 存在许多未考虑的因素, 具有一定的误差。如果对数据进行近一步的预处理和更加完善的分析, 得到的模型将会更加的准确、可靠。

(2)本文所建立的预测模型为理想情况下构建的, 没有考虑很多无法定量因素的影响, 比如测试范围外设备对实验数据的影响。此外构建模型时有选择性的忽略了部分不变数据对预测数据的影响, 为了进一步提高模型的实用性, 模型还需进一步优化。

(3)在相关因素的拟合时, 部分训练因子的拟合度不够好, 对于进一步的预测, 可能会产生较大的误差。

## 6.2 模型的改进与推广

(1) 测试范围外设备对实验数据的影响是不可忽略的, 因此在以后的研究中, 对测试范围外设备对实验数据的影响定量分析, 加入测试范围外设备对实验数据的影响, 使模型进一步优化, 具有更加完善 WLAN 组网测试。

(2)本文对于 WLAN 组网的数据预测, 仅仅对题目所给数据集具有较强的参考价值, 应该增加更多的实际数据, 使模型具有更强的推广性。

## 参考文献

- [1] Yang M, Li B. Survey and perspective on extremely high throughput (EHT) WLAN—IEEE 802.11 be[J]. *Mobile Networks and Applications*, 2020, 25(5): 1765-1780.
- [2] Rahman M M, Funabiki N, Munene K I, et al. A Throughput Request Satisfaction Method for Concurrently Communicating Multiple Hosts in Wireless Local Area Network[J]. *Sensors*, 2022, 22(22): 8823.
- [3] Kassa L, Deng J, Davis M, et al. Frame size optimization using a machine learning approach in wlan downlink mu-mimo channel[J]. *arXiv preprint arXiv:2209.09687*, 2022.
- [4] Tsuchiya Y, Suga N, Uruma K, et al. WLAN Throughput Prediction Using Deep Learning with Throughput, RSS, and COR[C]//2022 International Symposium on Intelligent Signal Processing and Communication Systems (ISPACS). IEEE, 2022: 1-4.
- [5] Venkatachalam I, Palaniappan S, Ameer John S. Distributed directional cooperative MAC (DD-CoopMAC) protocol for improving VBR throughput in IEEE 802.11 ad WLAN[J]. *Sādhanā*, 2020, 45(1): 245.
- [6] Uddin M F. Downlink performance analysis of a CSMA based WLAN under physical interference model[J]. *Computer Networks*, 2021, 196: 108255.
- [7] Munene K I, Funabiki N, Briantoro H, et al. A throughput drop estimation model for concurrent communications under partially overlapping channels without channel bonding and its application to channel assignment in IEEE 802.11 n WLAN[J]. *IEICE TRANSACTIONS on Information and Systems*, 2021, 104(5): 585-596.
- [8] Wu B, Funabiki N, Roy S C, et al. An Application of Throughput Request Satisfaction Method for Maximizing Concurrent Throughput in WLAN for IoT Application System[J]. *Sensors*, 2024, 24(7): 2173.
- [9] Fieller, E.C.; Hartley, H.O.; Pearson, E.S. (1957) Tests for rank correlation coefficients. I. *Biometrika* 44, pp. 470–481
- [10] 周志华. 机器学习[M]. Qing hua da xue chu ban she, 2016.
- [11] 赵志勇. Python 机器学习算法[M]. BEIJING BOOK CO. INC., 2017.
- [12] Fieller, E.C.; Hartley, H.O.; Pearson, E.S. (1957) Tests for rank correlation coefficients. I. *Biometrika* 44, pp. 470–481

## 附录

| 数据集 test_set_1_2ap |       |        |            | 任务 id 及预测数据         |                       |        |
|--------------------|-------|--------|------------|---------------------|-----------------------|--------|
| test_id            | ap_id | sta_id | throughput | predict<br>seq_time | predict<br>throughput | error% |
| 1                  | ap_1  | sta_1  | 119.42     | 48.69               | 115.07                | 3.64   |
| 1                  | ap_0  | sta_0  | 130        | 47.21               | 127.95                | 1.58   |
| 2                  | ap_1  | sta_1  | 137.97     | 49.59               | 135                   | 2.15   |
| 2                  | ap_0  | sta_0  | 140.82     | 49.59               | 143.4                 | 1.83   |
| 3                  | ap_1  | sta_1  | 133.99     | 48.66               | 130.54                | 2.57   |
| 3                  | ap_0  | sta_0  | 131.52     | 47.27               | 131.13                | 0.3    |
| 4                  | ap_1  | sta_1  | 139.31     | 49.56               | 138.02                | 0.93   |
| 4                  | ap_0  | sta_0  | 140.79     | 49.58               | 144.3                 | 2.49   |
| 5                  | ap_1  | sta_1  | 135.19     | 48.59               | 133.33                | 1.38   |
| 5                  | ap_0  | sta_0  | 130.3      | 47.32               | 130.21                | 0.07   |
| 6                  | ap_1  | sta_1  | 139.26     | 49.54               | 138.93                | 0.24   |
| 6                  | ap_0  | sta_0  | 137.95     | 49.58               | 141.13                | 2.31   |
| 7                  | ap_1  | sta_1  | 130.74     | 48.49               | 127.75                | 2.29   |
| 7                  | ap_0  | sta_0  | 151.65     | 47.38               | 148.93                | 1.79   |
| 8                  | ap_1  | sta_1  | 136.28     | 49.49               | 135.53                | 0.55   |
| 8                  | ap_0  | sta_0  | 135.21     | 49.62               | 138.12                | 2.15   |
| 9                  | ap_1  | sta_1  | 158.15     | 48.37               | 154.58                | 2.26   |
| 9                  | ap_0  | sta_0  | 127.75     | 47.38               | 128.17                | 0.33   |
| 10                 | ap_1  | sta_1  | 161.68     | 49.45               | 158.78                | 1.79   |
| 10                 | ap_0  | sta_0  | 162.3      | 49.64               | 162.65                | 0.22   |
| 11                 | ap_1  | sta_1  | 155.97     | 48.21               | 152.6                 | 2.16   |
| 11                 | ap_0  | sta_0  | 151.36     | 47.29               | 149.72                | 1.08   |
| 12                 | ap_1  | sta_1  | 163.12     | 49.37               | 161.14                | 1.21   |
| 12                 | ap_0  | sta_0  | 133.68     | 49.58               | 137.6                 | 2.93   |
| 13                 | ap_1  | sta_1  | 153.54     | 47.97               | 150.3                 | 2.11   |
| 13                 | ap_0  | sta_0  | 174.42     | 47.52               | 171.43                | 1.71   |
| 14                 | ap_1  | sta_1  | 184.65     | 49.28               | 180.63                | 2.18   |
| 14                 | ap_0  | sta_0  | 183.95     | 49.6                | 181.94                | 1.09   |
| 15                 | ap_1  | sta_1  | 170.96     | 47.57               | 166.49                | 2.61   |
| 15                 | ap_0  | sta_0  | 164.29     | 47.22               | 161.33                | 1.8    |
| 16                 | ap_1  | sta_1  | 182.13     | 49.11               | 178.39                | 2.05   |
| 16                 | ap_0  | sta_0  | 179.94     | 49.54               | 178.31                | 0.91   |
| 17                 | ap_1  | sta_1  | 174.49     | 47.05               | 170.4                 | 2.34   |
| 17                 | ap_0  | sta_0  | 166.24     | 47.26               | 163.62                | 1.58   |
| 18                 | ap_1  | sta_1  | 183.3      | 48.92               | 179.81                | 1.9    |
| 18                 | ap_0  | sta_0  | 185.32     | 49.46               | 183.59                | 0.93   |
| 19                 | ap_1  | sta_1  | 166.04     | 46.2                | 161.66                | 2.64   |

|    |      |       |        |       |        |       |
|----|------|-------|--------|-------|--------|-------|
| 19 | ap_0 | sta_0 | 172.44 | 46.98 | 169.84 | 1.51  |
| 20 | ap_1 | sta_1 | 174.16 | 48.46 | 170.74 | 1.96  |
| 20 | ap_0 | sta_0 | 181.24 | 49.38 | 179.87 | 0.76  |
| 21 | ap_1 | sta_1 | 200.93 | 45.2  | 190.63 | 5.13  |
| 21 | ap_0 | sta_0 | 149.46 | 46.2  | 149.17 | 0.19  |
| 22 | ap_1 | sta_1 | 215.85 | 48.04 | 204.02 | 5.48  |
| 22 | ap_0 | sta_0 | 179.96 | 49.03 | 178.56 | 0.78  |
| 23 | ap_1 | sta_1 | 165.94 | 41.48 | 158.52 | 4.47  |
| 23 | ap_0 | sta_0 | 151.06 | 45.72 | 151.02 | 0.03  |
| 24 | ap_1 | sta_1 | 193.34 | 47.31 | 185.94 | 3.83  |
| 24 | ap_0 | sta_0 | 128.51 | 48.7  | 134.2  | 4.43  |
| 25 | ap_1 | sta_1 | 141.7  | 30.25 | 136.18 | 3.9   |
| 25 | ap_0 | sta_0 | 97.18  | 39.86 | 89.97  | 7.42  |
| 26 | ap_1 | sta_1 | 153.04 | 32.67 | 146.37 | 4.36  |
| 26 | ap_0 | sta_0 | 134.19 | 47.55 | 134.14 | 0.04  |
| 27 | ap_1 | sta_1 | 137.39 | 29.33 | 132.17 | 3.8   |
| 27 | ap_0 | sta_0 | 73.03  | 31.05 | 58.42  | 20.01 |
| 28 | ap_1 | sta_1 | 142.39 | 30.09 | 136.92 | 3.84  |
| 28 | ap_0 | sta_0 | 102.96 | 35.2  | 96.94  | 5.85  |
| 29 | ap_1 | sta_1 | 132.78 | 29.24 | 127.03 | 4.33  |
| 29 | ap_0 | sta_0 | 107    | 29.07 | 99.85  | 6.68  |
| 30 | ap_1 | sta_1 | 141.02 | 29.8  | 135.53 | 3.89  |
| 30 | ap_0 | sta_0 | 96.08  | 30    | 91.78  | 4.48  |
| 31 | ap_1 | sta_1 | 135.99 | 29.03 | 130.58 | 3.98  |
| 31 | ap_0 | sta_0 | 90.06  | 25.12 | 83.29  | 7.52  |
| 32 | ap_1 | sta_1 | 138.05 | 29.47 | 132.41 | 4.09  |
| 32 | ap_0 | sta_0 | 98.96  | 29.16 | 93.4   | 5.62  |
| 33 | ap_1 | sta_1 | 137.58 | 29.37 | 131.8  | 4.2   |
| 33 | ap_0 | sta_0 | 84.16  | 24.12 | 77.96  | 7.37  |
| 34 | ap_1 | sta_1 | 136.55 | 29.15 | 130.67 | 4.31  |
| 34 | ap_0 | sta_0 | 83.69  | 23.66 | 77.49  | 7.41  |
| 35 | ap_1 | sta_1 | 141.4  | 29.88 | 135.24 | 4.36  |
| 35 | ap_0 | sta_0 | 98.05  | 28.49 | 92.41  | 5.75  |
| 36 | ap_1 | sta_1 | 132.8  | 28.94 | 126.61 | 4.66  |
| 36 | ap_0 | sta_0 | 79.3   | 22.12 | 73.32  | 7.54  |
| 37 | ap_1 | sta_1 | 141.87 | 29.98 | 135.41 | 4.55  |
| 37 | ap_0 | sta_0 | 95.44  | 27.73 | 89.83  | 5.88  |
| 38 | ap_1 | sta_1 | 132.28 | 29.13 | 125.74 | 4.94  |
| 38 | ap_0 | sta_0 | 68.28  | 21.26 | 55.21  | 19.14 |
| 39 | ap_1 | sta_1 | 133.93 | 28.59 | 127.63 | 4.7   |
| 39 | ap_0 | sta_0 | 79.83  | 21.41 | 73.85  | 7.49  |
| 40 | ap_1 | sta_1 | 142.36 | 30.39 | 135.09 | 5.11  |
| 40 | ap_0 | sta_0 | 96.3   | 27.98 | 90.92  | 5.59  |

| 数据集 test_set_1_3ap |       |        |            | 任务 id 及预测数据         |                       |        |
|--------------------|-------|--------|------------|---------------------|-----------------------|--------|
| test_id            | ap_id | sta_id | throughput | predict<br>seq_time | predict<br>throughput | error% |
| 1                  | ap_2  | sta_2  | 21.02      | 21.32               | 18.38                 | 12.56  |
| 1                  | ap_1  | sta_1  | 63.15      | 42.2                | 52.07                 | 17.55  |
| 1                  | ap_0  | sta_0  | 56.33      | 38.53               | 44.61                 | 20.81  |
| 2                  | ap_2  | sta_2  | 28.88      | 29.63               | 26.58                 | 7.96   |
| 2                  | ap_1  | sta_1  | 53.15      | 35.52               | 44.23                 | 16.78  |
| 2                  | ap_0  | sta_0  | 56.39      | 38.12               | 44.96                 | 20.27  |
| 3                  | ap_2  | sta_2  | 20.17      | 21.19               | 17.73                 | 12.1   |
| 3                  | ap_1  | sta_1  | 62.94      | 42.55               | 51.82                 | 17.67  |
| 3                  | ap_0  | sta_0  | 52.89      | 37.05               | 41.77                 | 21.02  |
| 4                  | ap_2  | sta_2  | 30.49      | 21.1                | 22.62                 | 25.81  |
| 4                  | ap_1  | sta_1  | 63.58      | 42.98               | 52.82                 | 16.92  |
| 4                  | ap_0  | sta_0  | 32.53      | 33.38               | 29.59                 | 9.04   |
| 5                  | ap_2  | sta_2  | 41.57      | 28.1                | 32.71                 | 21.31  |
| 5                  | ap_1  | sta_1  | 61.04      | 40.79               | 51.28                 | 15.99  |
| 5                  | ap_0  | sta_0  | 51.58      | 35.7                | 41.46                 | 19.62  |
| 6                  | ap_2  | sta_2  | 29.67      | 20.78               | 22.38                 | 24.57  |
| 6                  | ap_1  | sta_1  | 63.69      | 42.56               | 53.95                 | 15.29  |
| 6                  | ap_0  | sta_0  | 49.51      | 33.47               | 40.67                 | 17.85  |
| 7                  | ap_2  | sta_2  | 77.58      | 31.09               | 62.35                 | 19.63  |
| 7                  | ap_1  | sta_1  | 63.84      | 42.66               | 54.64                 | 14.41  |
| 7                  | ap_0  | sta_0  | 0          | 31.85               | 14.94                 | inf    |
| 8                  | ap_2  | sta_2  | 76.83      | 30.79               | 61.66                 | 19.74  |
| 8                  | ap_1  | sta_1  | 80.95      | 40.57               | 65.39                 | 19.22  |
| 8                  | ap_0  | sta_0  | 40.17      | 28.83               | 31.5                  | 21.58  |
| 9                  | ap_2  | sta_2  | 55.27      | 22.15               | 42.12                 | 23.79  |
| 9                  | ap_1  | sta_1  | 64.22      | 42.43               | 56.13                 | 12.6   |
| 9                  | ap_0  | sta_0  | 85.64      | 28.6                | 72.1                  | 15.81  |
| 10                 | ap_2  | sta_2  | 57.65      | 23.37               | 44.1                  | 23.5   |
| 10                 | ap_1  | sta_1  | 99.58      | 40.37               | 80.23                 | 19.43  |
| 10                 | ap_0  | sta_0  | 76.22      | 25.75               | 62.51                 | 17.99  |
| 11                 | ap_2  | sta_2  | 60.82      | 23.3                | 50.1                  | 17.63  |
| 11                 | ap_1  | sta_1  | 63.04      | 42.13               | 55.22                 | 12.4   |
| 11                 | ap_0  | sta_0  | 88.27      | 25.94               | 74.61                 | 15.48  |
| 12                 | ap_2  | sta_2  | 61.1       | 23.41               | 50.39                 | 17.53  |
| 12                 | ap_1  | sta_1  | 78.95      | 39.57               | 65.57                 | 16.95  |
| 12                 | ap_0  | sta_0  | 38.21      | 21.92               | 28.57                 | 25.23  |
| 13                 | ap_2  | sta_2  | 60.79      | 23.29               | 51.01                 | 16.09  |
| 13                 | ap_1  | sta_1  | 81.75      | 41.45               | 68.02                 | 16.8   |
| 13                 | ap_0  | sta_0  | 70.67      | 23.07               | 59.44                 | 15.89  |
| 14                 | ap_2  | sta_2  | 58.24      | 23.34               | 46.86                 | 19.54  |
| 14                 | ap_1  | sta_1  | 72.36      | 36.69               | 60.44                 | 16.47  |

|    |      |       |       |       |       |       |
|----|------|-------|-------|-------|-------|-------|
| 14 | ap_0 | sta_0 | 57.48 | 17.28 | 44.16 | 23.17 |
| 15 | ap_2 | sta_2 | 58.41 | 23.14 | 48.25 | 17.39 |
| 15 | ap_1 | sta_1 | 60.31 | 40.77 | 53.62 | 11.09 |
| 15 | ap_0 | sta_0 | 52.78 | 16.24 | 40.34 | 23.57 |
| 16 | ap_2 | sta_2 | 56.72 | 22.22 | 47.9  | 15.55 |
| 16 | ap_1 | sta_1 | 78.04 | 38.67 | 68.2  | 12.61 |
| 16 | ap_0 | sta_0 | 47.81 | 14.21 | 36.6  | 23.45 |
| 17 | ap_2 | sta_2 | 57.34 | 23.52 | 46.94 | 18.14 |
| 17 | ap_1 | sta_1 | 71.76 | 32.32 | 61.11 | 14.84 |
| 17 | ap_0 | sta_0 | 41.72 | 14.17 | 30.68 | 26.46 |
| 18 | ap_2 | sta_2 | 55.8  | 21.86 | 47.8  | 14.34 |
| 18 | ap_1 | sta_1 | 69.63 | 35.72 | 59.76 | 14.17 |
| 18 | ap_0 | sta_0 | 55.13 | 14.24 | 40.15 | 27.17 |
| 19 | ap_2 | sta_2 | 59.2  | 24    | 49.67 | 16.1  |
| 19 | ap_1 | sta_1 | 60.29 | 30.57 | 52.39 | 13.1  |
| 19 | ap_0 | sta_0 | 51.5  | 14.56 | 37.38 | 27.42 |
| 20 | ap_2 | sta_2 | 48.81 | 21.01 | 39.66 | 18.75 |
| 20 | ap_1 | sta_1 | 112   | 31.37 | 92.05 | 17.81 |
| 20 | ap_0 | sta_0 | 45.78 | 13.14 | 34.26 | 25.16 |
| 21 | ap_2 | sta_2 | 49.23 | 21.19 | 40.54 | 17.65 |
| 21 | ap_1 | sta_1 | 55.95 | 26.11 | 47.41 | 15.26 |
| 21 | ap_0 | sta_0 | 51.73 | 13.04 | 38.63 | 25.32 |
| 22 | ap_2 | sta_2 | 77.66 | 20.83 | 58.71 | 24.4  |
| 22 | ap_1 | sta_1 | 81.19 | 23.89 | 64.12 | 21.02 |
| 22 | ap_0 | sta_0 | 50.5  | 11.87 | 38.75 | 23.27 |
| 23 | ap_2 | sta_2 | 77.51 | 20.27 | 59.32 | 23.47 |
| 23 | ap_1 | sta_1 | 80.35 | 21.55 | 62.25 | 22.53 |
| 23 | ap_0 | sta_0 | 46.36 | 11.41 | 35.67 | 23.06 |
| 24 | ap_2 | sta_2 | 70.11 | 23.28 | 52.14 | 25.63 |
| 24 | ap_1 | sta_1 | 83.47 | 25.68 | 59.86 | 28.29 |
| 24 | ap_0 | sta_0 | 52    | 15.11 | 38.98 | 25.04 |
| 25 | ap_2 | sta_2 | 78    | 19.9  | 60.56 | 22.36 |
| 25 | ap_1 | sta_1 | 81.49 | 21.31 | 63.9  | 21.59 |
| 25 | ap_0 | sta_0 | 44.81 | 11.03 | 35.07 | 21.74 |
| 26 | ap_2 | sta_2 | 77.36 | 19.5  | 60.64 | 21.61 |
| 26 | ap_1 | sta_1 | 70.24 | 18.6  | 54.68 | 22.15 |
| 26 | ap_0 | sta_0 | 46.52 | 11.06 | 37.54 | 19.3  |
| 27 | ap_2 | sta_2 | 52.92 | 22.95 | 41.75 | 21.11 |
| 27 | ap_1 | sta_1 | 81.22 | 23.6  | 59.28 | 27.01 |
| 27 | ap_0 | sta_0 | 54.84 | 16.39 | 41.1  | 25.05 |
| 28 | ap_2 | sta_2 | 71.02 | 18.12 | 55.49 | 21.87 |
| 28 | ap_1 | sta_1 | 61.87 | 17.33 | 50.55 | 18.3  |
| 28 | ap_0 | sta_0 | 44.99 | 11.34 | 35.87 | 20.27 |
| 29 | ap_2 | sta_2 | 48.63 | 22.08 | 38.39 | 21.06 |

|    |      |       |       |       |       |       |
|----|------|-------|-------|-------|-------|-------|
| 29 | ap_1 | sta_1 | 29.53 | 23.2  | 28.15 | 4.67  |
| 29 | ap_0 | sta_0 | 62.02 | 17.3  | 47.41 | 23.56 |
| 30 | ap_2 | sta_2 | 68.1  | 16.76 | 54.33 | 20.22 |
| 30 | ap_1 | sta_1 | 67.1  | 17.77 | 51.71 | 22.94 |
| 30 | ap_0 | sta_0 | 48.54 | 11.54 | 40.2  | 17.18 |
| 31 | ap_2 | sta_2 | 53.81 | 21.99 | 41.45 | 22.97 |
| 31 | ap_1 | sta_1 | 75.64 | 23.27 | 52.81 | 30.18 |
| 31 | ap_0 | sta_0 | 61.59 | 17.65 | 46.62 | 24.31 |
| 32 | ap_2 | sta_2 | 64.72 | 15.93 | 52.04 | 19.59 |
| 32 | ap_1 | sta_1 | 67.9  | 17.98 | 51.89 | 23.58 |
| 32 | ap_0 | sta_0 | 48.86 | 11.75 | 40.52 | 17.07 |
| 33 | ap_2 | sta_2 | 65.23 | 21.66 | 48.02 | 26.38 |
| 33 | ap_1 | sta_1 | 78.26 | 23.39 | 55.04 | 29.67 |
| 33 | ap_0 | sta_0 | 60.76 | 18.16 | 45.48 | 25.15 |
| 34 | ap_2 | sta_2 | 63.85 | 16.29 | 50.59 | 20.77 |
| 34 | ap_1 | sta_1 | 71.16 | 18.38 | 54.89 | 22.86 |
| 34 | ap_0 | sta_0 | 50.71 | 12.48 | 41.33 | 18.5  |
| 35 | ap_2 | sta_2 | 51.24 | 22.22 | 40.68 | 20.61 |
| 35 | ap_1 | sta_1 | 31.06 | 24.08 | 30.81 | 0.8   |
| 35 | ap_0 | sta_0 | 70.34 | 19.11 | 54.12 | 23.06 |

| 数据集 test_set_2_2ap |       |        |     | 任务 id 及预测数据 |                       |  |
|--------------------|-------|--------|-----|-------------|-----------------------|--|
| test_id            | ap_id | sta_id | nss | mcs         | predicted<br>PHY_Rate |  |
| 1                  | ap_1  | sta_1  | 2   | 8           | 212.7                 |  |
| 1                  | ap_0  | sta_0  | 1   | 7           | 99.2                  |  |
| 2                  | ap_1  | sta_1  | 2   | 8           | 212.7                 |  |
| 2                  | ap_0  | sta_0  | 1   | 7           | 99.2                  |  |
| 3                  | ap_1  | sta_1  | 2   | 6           | 168.9                 |  |
| 3                  | ap_0  | sta_0  | 1   | 7           | 99.2                  |  |
| 4                  | ap_1  | sta_1  | 2   | 6           | 168.9                 |  |
| 4                  | ap_0  | sta_0  | 1   | 7           | 99.2                  |  |
| 5                  | ap_1  | sta_1  | 2   | 7           | 172.3                 |  |
| 5                  | ap_0  | sta_0  | 1   | 7           | 99.2                  |  |
| 6                  | ap_1  | sta_1  | 2   | 7           | 172.3                 |  |
| 6                  | ap_0  | sta_0  | 1   | 7           | 99.2                  |  |
| 7                  | ap_1  | sta_1  | 2   | 10          | 260.4                 |  |
| 7                  | ap_0  | sta_0  | 1   | 7           | 99.2                  |  |
| 8                  | ap_1  | sta_1  | 2   | 10          | 260.4                 |  |
| 8                  | ap_0  | sta_0  | 1   | 7           | 99.2                  |  |
| 9                  | ap_1  | sta_1  | 2   | 10          | 264.4                 |  |
| 9                  | ap_0  | sta_0  | 1   | 7           | 99.2                  |  |
| 10                 | ap_1  | sta_1  | 2   | 10          | 264                   |  |
| 10                 | ap_0  | sta_0  | 1   | 7           | 99.2                  |  |

|    |      |       |   |    |       |
|----|------|-------|---|----|-------|
| 11 | ap_1 | sta_1 | 2 | 10 | 264   |
| 11 | ap_0 | sta_0 | 1 | 7  | 99.2  |
| 12 | ap_1 | sta_1 | 2 | 10 | 266.5 |
| 12 | ap_0 | sta_0 | 1 | 7  | 99.2  |
| 13 | ap_1 | sta_1 | 2 | 10 | 269.4 |
| 13 | ap_0 | sta_0 | 2 | 6  | 160.7 |
| 14 | ap_1 | sta_1 | 2 | 10 | 275.7 |
| 14 | ap_0 | sta_0 | 2 | 10 | 270.2 |
| 15 | ap_1 | sta_1 | 2 | 10 | 275.7 |
| 15 | ap_0 | sta_0 | 2 | 10 | 270.2 |
| 16 | ap_1 | sta_1 | 2 | 10 | 280.6 |
| 16 | ap_0 | sta_0 | 2 | 10 | 279.9 |
| 17 | ap_1 | sta_1 | 2 | 10 | 280.6 |
| 17 | ap_0 | sta_0 | 2 | 10 | 279.9 |
| 18 | ap_1 | sta_1 | 2 | 10 | 281.6 |
| 18 | ap_0 | sta_0 | 2 | 10 | 281.9 |
| 19 | ap_1 | sta_1 | 2 | 10 | 281.6 |
| 19 | ap_0 | sta_0 | 2 | 10 | 281.9 |
| 20 | ap_1 | sta_1 | 2 | 10 | 282.1 |
| 20 | ap_0 | sta_0 | 2 | 10 | 282.2 |
| 21 | ap_1 | sta_1 | 2 | 10 | 282.1 |
| 21 | ap_0 | sta_0 | 2 | 10 | 282.2 |
| 22 | ap_1 | sta_1 | 2 | 10 | 282.2 |
| 22 | ap_0 | sta_0 | 2 | 10 | 282.4 |
| 23 | ap_1 | sta_1 | 2 | 10 | 282.4 |
| 23 | ap_0 | sta_0 | 2 | 10 | 282.5 |
| 24 | ap_1 | sta_1 | 2 | 10 | 282.4 |
| 24 | ap_0 | sta_0 | 2 | 10 | 282.5 |
| 25 | ap_1 | sta_1 | 2 | 10 | 282.5 |
| 25 | ap_0 | sta_0 | 2 | 10 | 282.6 |
| 26 | ap_1 | sta_1 | 2 | 10 | 282.6 |
| 26 | ap_0 | sta_0 | 2 | 10 | 282.7 |
| 27 | ap_1 | sta_1 | 2 | 10 | 282.6 |
| 27 | ap_0 | sta_0 | 2 | 10 | 282.8 |
| 28 | ap_1 | sta_1 | 2 | 10 | 282.6 |
| 28 | ap_0 | sta_0 | 2 | 10 | 282.8 |
| 29 | ap_1 | sta_1 | 2 | 10 | 282.7 |
| 29 | ap_0 | sta_0 | 2 | 10 | 282.8 |
| 30 | ap_1 | sta_1 | 2 | 10 | 282.7 |
| 30 | ap_0 | sta_0 | 2 | 10 | 282.8 |
| 31 | ap_1 | sta_1 | 2 | 10 | 282.8 |
| 31 | ap_0 | sta_0 | 2 | 10 | 282.8 |
| 32 | ap_1 | sta_1 | 2 | 10 | 282.8 |
| 32 | ap_0 | sta_0 | 2 | 10 | 282.8 |

| 数据集 test_set_2_3ap |       |        |     | 任务 id 及预测数据 |                       |
|--------------------|-------|--------|-----|-------------|-----------------------|
| test_id            | ap_id | sta_id | nss | mcs         | predicted<br>PHY_Rate |
| 1                  | ap_2  | sta_2  | 2   | 6           | 159.6                 |
| 1                  | ap_0  | sta_0  | 1   | 7           | 89.3                  |
| 1                  | ap_1  | sta_1  | 1   | 4           | 63.6                  |
| 2                  | ap_2  | sta_2  | 2   | 6           | 159.6                 |
| 2                  | ap_0  | sta_0  | 1   | 7           | 89.3                  |
| 2                  | ap_1  | sta_1  | 1   | 4           | 63.6                  |
| 3                  | ap_2  | sta_2  | 2   | 6           | 165.3                 |
| 3                  | ap_0  | sta_0  | 1   | 6           | 85.4                  |
| 3                  | ap_1  | sta_1  | 1   | 4           | 65.2                  |
| 4                  | ap_2  | sta_2  | 2   | 6           | 165.3                 |
| 4                  | ap_0  | sta_0  | 1   | 6           | 85.4                  |
| 4                  | ap_1  | sta_1  | 1   | 4           | 65.2                  |
| 5                  | ap_2  | sta_2  | 2   | 9           | 248.9                 |
| 5                  | ap_0  | sta_0  | 1   | 7           | 94.5                  |
| 5                  | ap_1  | sta_1  | 1   | 5           | 76.9                  |
| 6                  | ap_2  | sta_2  | 2   | 9           | 248.9                 |
| 6                  | ap_0  | sta_0  | 1   | 7           | 94.5                  |
| 6                  | ap_1  | sta_1  | 1   | 5           | 76.9                  |
| 7                  | ap_2  | sta_2  | 2   | 10          | 278                   |
| 7                  | ap_0  | sta_0  | 1   | 8           | 113.1                 |
| 7                  | ap_1  | sta_1  | 1   | 6           | 78.1                  |
| 8                  | ap_2  | sta_2  | 2   | 10          | 278                   |
| 8                  | ap_0  | sta_0  | 1   | 8           | 113.1                 |
| 8                  | ap_1  | sta_1  | 1   | 6           | 78.1                  |
| 9                  | ap_2  | sta_2  | 2   | 6           | 157.4                 |
| 9                  | ap_0  | sta_0  | 1   | 9           | 116.3                 |
| 9                  | ap_1  | sta_1  | 1   | 6           | 84.8                  |
| 10                 | ap_2  | sta_2  | 2   | 6           | 157.4                 |
| 10                 | ap_0  | sta_0  | 1   | 9           | 116.3                 |
| 10                 | ap_1  | sta_1  | 1   | 6           | 84.8                  |
| 11                 | ap_2  | sta_2  | 2   | 10          | 282                   |
| 11                 | ap_0  | sta_0  | 1   | 10          | 131.9                 |
| 11                 | ap_1  | sta_1  | 1   | 7           | 96.7                  |
| 12                 | ap_2  | sta_2  | 2   | 10          | 282                   |
| 12                 | ap_0  | sta_0  | 1   | 10          | 131.9                 |
| 12                 | ap_1  | sta_1  | 1   | 7           | 96.7                  |
| 13                 | ap_2  | sta_2  | 2   | 7           | 172.6                 |
| 13                 | ap_0  | sta_0  | 1   | 10          | 132.4                 |
| 13                 | ap_1  | sta_1  | 1   | 6           | 80                    |
| 14                 | ap_2  | sta_2  | 2   | 7           | 172.6                 |

|    |      |       |   |    |       |
|----|------|-------|---|----|-------|
| 14 | ap_0 | sta_0 | 1 | 10 | 132.4 |
| 14 | ap_1 | sta_1 | 1 | 6  | 80    |
| 15 | ap_2 | sta_2 | 2 | 7  | 190.7 |
| 15 | ap_0 | sta_0 | 2 | 5  | 139.7 |
| 15 | ap_1 | sta_1 | 1 | 10 | 136   |
| 16 | ap_2 | sta_2 | 2 | 7  | 190.7 |
| 16 | ap_0 | sta_0 | 2 | 5  | 139.7 |
| 16 | ap_1 | sta_1 | 1 | 10 | 136   |
| 17 | ap_2 | sta_2 | 2 | 7  | 192.2 |
| 17 | ap_0 | sta_0 | 1 | 11 | 149.3 |
| 17 | ap_1 | sta_1 | 1 | 10 | 135.9 |
| 18 | ap_2 | sta_2 | 2 | 7  | 192.2 |
| 18 | ap_0 | sta_0 | 1 | 11 | 149.3 |
| 18 | ap_1 | sta_1 | 1 | 10 | 135.9 |
| 19 | ap_2 | sta_2 | 2 | 10 | 285.3 |
| 19 | ap_0 | sta_0 | 2 | 6  | 156   |
| 19 | ap_1 | sta_1 | 2 | 6  | 163.8 |
| 20 | ap_2 | sta_2 | 2 | 10 | 285.3 |
| 20 | ap_0 | sta_0 | 2 | 6  | 158.5 |
| 20 | ap_1 | sta_1 | 2 | 6  | 169.4 |
| 21 | ap_2 | sta_2 | 2 | 6  | 167   |
| 21 | ap_0 | sta_0 | 2 | 6  | 158.3 |
| 21 | ap_1 | sta_1 | 1 | 11 | 152.1 |
| 22 | ap_2 | sta_2 | 2 | 7  | 174.7 |
| 22 | ap_0 | sta_0 | 2 | 7  | 179.3 |
| 22 | ap_1 | sta_1 | 2 | 7  | 203.7 |
| 23 | ap_2 | sta_2 | 2 | 8  | 229.2 |
| 23 | ap_0 | sta_0 | 2 | 8  | 214.3 |
| 23 | ap_1 | sta_1 | 2 | 10 | 283.6 |
| 24 | ap_2 | sta_2 | 2 | 8  | 229.2 |
| 24 | ap_0 | sta_0 | 2 | 8  | 214.3 |
| 24 | ap_1 | sta_1 | 2 | 10 | 283.6 |
| 25 | ap_2 | sta_2 | 2 | 9  | 238.5 |
| 25 | ap_0 | sta_0 | 2 | 9  | 229.7 |
| 25 | ap_1 | sta_1 | 2 | 10 | 276.7 |
| 26 | ap_2 | sta_2 | 2 | 10 | 281.7 |
| 26 | ap_0 | sta_0 | 2 | 10 | 258.4 |
| 26 | ap_1 | sta_1 | 2 | 10 | 273.2 |
| 27 | ap_2 | sta_2 | 2 | 10 | 286.6 |
| 27 | ap_0 | sta_0 | 2 | 10 | 276.9 |
| 27 | ap_1 | sta_1 | 2 | 10 | 285.1 |
| 28 | ap_2 | sta_2 | 2 | 8  | 218.5 |
| 28 | ap_0 | sta_0 | 2 | 10 | 279.6 |
| 28 | ap_1 | sta_1 | 2 | 11 | 286.8 |

|    |      |       |   |    |       |
|----|------|-------|---|----|-------|
| 29 | ap_2 | sta_2 | 2 | 10 | 282   |
| 29 | ap_0 | sta_0 | 2 | 10 | 285.8 |
| 29 | ap_1 | sta_1 | 2 | 11 | 286.8 |

| 程序 1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | 随机森林核心代码 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <pre> preprocessor =ColumnTransformer(transformers=[     ('num', StandardScaler(), numeric_features),     ('cat', OneHotEncoder(categories=cate), categorical_features) ])  model = Pipeline(steps=[('preprocessor', preprocessor), ('regressor', RandomForestRegressor(n_estimators=100, random_state=42))]) X = training_data[features] y = pd.to_numeric(training_data[target], errors='coerce')  X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)  param_grid = {}  # model = RandomForestRegressor(random_state=42) grid_search = GridSearchCV(estimator=model, param_grid=param_grid, cv=3, scoring='neg_mean_squared_error', n_jobs=-1) grid_search.fit(X_train, y_train)  best_model = grid_search.best_estimator_  best_model = grid_search.best_estimator_  y_pred_train = best_model.predict(X_test) mse_train = mean_squared_error(y_test, y_pred_train)  print(f' Mean Squared Error: {mse_train}') mae_train = mean_absolute_error(y_test, y_pred_train) print(f' Mean Absolute Error: {mae_train}')  est_set_2a = test_set_2ap[features]  y_pred_2ap = best_model.predict(test_set_2a)  test_set_2ap['predicted_seq_time'] = y_pred_2ap </pre> |          |

| 程序 2                                                        | LSTM 核心代码 |
|-------------------------------------------------------------|-----------|
| <pre> preprocessor =ColumnTransformer(transformers=[ </pre> |           |

```

        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(categories=cate), categorical_features)
    ])
    X_train = training_data[features]

    X = preprocessor.fit_transform(X_train)

    y = pd.to_numeric(training_data[target], errors='coerce')

    X_scaled = X.reshape((X.shape[0], 1, X.shape[1]))

    X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)

    optimizer = Adam(learning_rate=0.0005)
    model = Sequential()
    model.add(LSTM(50, activation='relu', return_sequences=True,
input_shape=(X_train.shape[1], X_train.shape[2])))
    model.add(Dropout(0.2))

    model.add(LSTM(50, activation='relu', return_sequences=True))
    model.add(Dropout(0.2))

    model.add(LSTM(50, activation='relu', return_sequences=True))
    model.add(Dropout(0.2))

    model.add(LSTM(50, activation='relu', return_sequences=True))
    model.add(Dropout(0.2))

    model.add(LSTM(50, activation='relu', return_sequences=True))
    model.add(Dropout(0.2))

    model.add(LSTM(50, activation='relu', return_sequences=True))
    model.add(Dropout(0.2))

    model.add(LSTM(50, activation='relu', return_sequences=True))
    model.add(Dropout(0.2))

    model.add(LSTM(50, activation='relu', return_sequences=True))
    model.add(Dropout(0.2))

```

```

model.add(LSTM(50, activation='relu', return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(1))
model.compile(optimizer=optimizer, loss='mean_squared_error')

checkpoint = ModelCheckpoint(
    'best_model.h5',
    monitor='val_loss',
    save_best_only=True,
    mode='min',
    verbose=1
)

model.fit(X_train, y_train, epochs=500, batch_size=4, validation_split=0.2,
callbacks=[checkpoint])

model.load_weights('best_model.h5')
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print(f" Mean Squared Error: {mse}")

y_pred_train = y_pred.flatten()

test_set_2a = test_set_2ap[features]

X_test_processed = preprocessor.transform(test_set_2a)

test_set_2ap_scaled = X_test_processed.reshape((X_test_processed.shape[0], 1,
X_test_processed.shape[1]))

y_pred_2ap = model.predict(test_set_2ap_scaled)

test_set_2ap['predicted_seq_time'] = y_pred_2ap.flatten()

```

| 程序 3                                                                                                                                                                                                                                                                                                | 多元线性回归核心代码 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <pre> scaler = StandardScaler() X_scaled = scaler.fit_transform(features[model_features])  X_train, X_test, y_train, y_test = train_test_split(X_scaled, target, test_size=0.2, random_state=42)  model = SGDRegressor(max_iter=3000, learning_rate='constant', eta0=0.001, random_state=42) </pre> |            |

```

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
mae_train = mean_absolute_error(y_test, y_pred)
print(f' Mean Absolute Error: {mae_train}')
X_test_scaled = scaler.transform(test_features[model_features])

PHY_Rate_pred = model.predict(X_test_scaled)

```

| 程序 4                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 支持向量机 SVM 核心代码 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| <pre> preprocessor = ColumnTransformer(transformers=[     ('num', StandardScaler(), numeric_features),     ('cat', OneHotEncoder(categories=cate), categorical_features) ]) X_scaled = preprocessor.fit_transform(features)  X_train, X_test, y_train, y_test = train_test_split(X_scaled, target, test_size=0.2, random_state=42)  model = SVR(kernel='linear', C=10, epsilon=0.1) model = SVR(kernel='rbf', gamma=0.1, C=10, epsilon=0.1) model = SVR(kernel='poly', C=10, epsilon=0.1)  model.fit(X_train, y_train)  y_pred = model.predict(X_test)  mse = mean_squared_error(y_test, y_pred) print("Mean Squared Error:", mse) mae_train = mean_absolute_error(y_test, y_pred) print(f' Mean Absolute Error: {mae_train}')  X_test_scaled = preprocessor.transform(test_features)  throughput_pred = model.predict(X_test_scaled) </pre> |                |

| 程序 5                                                                                                | 数据处理核心代码 |
|-----------------------------------------------------------------------------------------------------|----------|
| <pre> def compute_sinr0(row):     if isinstance(row['sta_from_ap_0_mean_ant_rssi'], str) and </pre> |          |

```

isinstance(row['sta_from_ap_1_mean_ant_rssi'], str):
    rssi_0 = np.fromstring(row['sta_from_ap_0_mean_ant_rssi'].strip("[]"),
sep=',')
    rssi_1 = np.fromstring(row['sta_from_ap_1_mean_ant_rssi'].strip("[]"),
sep=',')
    rssi_2 = np.fromstring(row['sta_from_ap_2_mean_ant_rssi'].strip("[]"),
sep=',')

    max_length = max(rssi_0.shape[0], rssi_1.shape[0], rssi_2.shape[0])

    if rssi_0.shape[0] < max_length:
        rssi_0 = np.pad(rssi_0, (0, max_length - rssi_0.shape[0]),
constant_values=-100)

    if rssi_1.shape[0] < max_length:
        rssi_1 = np.pad(rssi_1, (0, max_length - rssi_1.shape[0]),
constant_values=-100)

    if rssi_2.shape[0] < max_length:
        rssi_2 = np.pad(rssi_2, (0, max_length - rssi_2.shape[0]),
constant_values=-100)

    return rssi_0 - rssi_1 - rssi_2
return 0

features.loc[data['sta_id'] == "sta_0", 'sta_0_SINR'] = features[data['sta_id'] ==
"sta_0"].apply(compute_sinr0, axis=1)
features.loc[data['sta_id'] == "sta_0", 'sta_1_SINR'] = 0
features.loc[data['sta_id'] == "sta_0", 'sta_2_SINR'] = 0

training_data['ap_from_ap_0_sum_ant_rssi'] =
training_data['ap_from_ap_0_sum_ant_rssi'].apply(lambda x: np.mean(ast.literal_eval(x))
if x else -100)
training_data['ap_from_ap_0_max_ant_rssi'] =
training_data['ap_from_ap_0_max_ant_rssi'].apply(lambda x: np.mean(ast.literal_eval(x))
if x else -100)
training_data['ap_from_ap_0_mean_ant_rssi'] =
training_data['ap_from_ap_0_mean_ant_rssi'].apply(lambda x: np.mean(ast.literal_eval(x))
if x else -100)

training_data['ap_from_ap_1_sum_ant_rssi'] =
training_data['ap_from_ap_1_sum_ant_rssi'].apply(lambda x: np.mean(ast.literal_eval(x))
if x else -100)
training_data['ap_from_ap_1_max_ant_rssi'] =

```

```

training_data['ap_from_ap_1_max_ant_rssi'].apply(lambda x: np.mean(ast.literal_eval(x))
if x else -100)
    training_data['ap_from_ap_1_mean_ant_rssi'] =
training_data['ap_from_ap_1_mean_ant_rssi'].apply(lambda x: np.mean(ast.literal_eval(x))
if x else -100)

    training_data['ap_from_ap_2_sum_ant_rssi'] =
training_data['ap_from_ap_2_sum_ant_rssi'].apply(lambda x: np.mean(ast.literal_eval(x))
if x else -100)
    training_data['ap_from_ap_2_max_ant_rssi'] =
training_data['ap_from_ap_2_max_ant_rssi'].apply(lambda x: np.mean(ast.literal_eval(x))
if x else -100)
    training_data['ap_from_ap_2_mean_ant_rssi'] =
training_data['ap_from_ap_2_mean_ant_rssi'].apply(lambda x: np.mean(ast.literal_eval(x))
if x else -100)

```

| 程序 6                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 查找及匹配 PHY Rate 核心代码 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| <pre> phy_rate_table = {     (0, 1): 8.6, (0, 2): 17.2,     (1, 1): 17.2, (1, 2): 34.4,     (2, 1): 25.8, (2, 2): 51.6,     (3, 1): 34.4, (3, 2): 68.8,     (4, 1): 51.6, (4, 2): 103.2,     (5, 1): 68.8, (5, 2): 137.6,     (6, 1): 77.4, (6, 2): 154.9,     (7, 1): 86.0, (7, 2): 172.1,     (8, 1): 103.2, (8, 2): 206.5,     (9, 1): 114.7, (9, 2): 229.4,     (10, 1): 129.0, (10, 2): 258.1,     (11, 1): 143.4, (11, 2): 286.8 }  def map_phy_rate(row):     mcs = row['mcs']     nss = row['nss']     return phy_rate_table.get((mcs, nss), 0)  data['PHY_Rate'] = data.apply(map_phy_rate, axis=1)  def find_mcs_nss(predicted_phy_rate):      sorted_phy_rates = sorted(phy_rate_table.items(), key=lambda x: x[1]) </pre> |                     |

```

suitable_mcs_nss = sorted_phy_rates[0][0]

for (mcs_nss, phy_rate) in sorted_phy_rates:
    if predicted_phy_rate >= phy_rate:
        suitable_mcs_nss = mcs_nss
    else:
        break

return suitable_mcs_nss[0], suitable_mcs_nss[1]

```

| 程序 7                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 计算网络吞吐量误差核心代码 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| <pre> def calculate_throughput(row):      mcs = row['mcs']     nss = row['nss']     phy_rate = phy_rate_table.get((mcs, nss), 0)      throughput = (phy_rate * (1 - row['per'])) * (row['seq_time'] / row['test_dur'])     return round(throughput,2)  data['throughput'] = data.apply(calculate_throughput, axis=1)  total_throughput = data['throughput'].sum()  actual_throughput = test_data['throughput'] predicted_throughput = test_data['predict throughput']  test_data['error'] = (abs(predicted_throughput - actual_throughput)).round(2)  test_data['throughput error%'] = ((test_data['error'] / test_data['throughput']) * 100).round(2) </pre> |               |